

基于变物性传热传质与移动边界的药材烘干模型

摘 要

针对圆柱形药材热风烘干中的内部温度、水分分布与干燥时长问题，本文建立由固定域预热模型、变物性温湿耦合模型和实测收缩驱动的移动边界模型组成的统一求解框架。以干基含水率为水分变量，采用圆柱环形有限体积离散、隐式 BDF 时间积分与两级网格 Richardson 外推，计算题目指定的时空分布。

针对问题一，采用常热物性导热方程和浓度相关扩散方程。计算得到 1800 s 时中心、表面温度分别为 33.5753、36.7856 °C，对应干基含水率为 2.5500、1.5102 kg/kg。温度向内部传递较快，失水主要集中于表层。

针对问题二，从初始时刻统一使用附录 3 的经验物性，联立求解温度和含水率。3 h 时中心、表面温度分别为 49.8495、49.9664 °C，含水率分别为 1.7662、1.0081 kg/kg，说明接近热平衡并不意味着干燥完成。表 3、表 4 给出前三小时结果，支撑材料提供到干燥终点后首个整分钟的逐秒完整场。

针对问题三，将附件 1 最后一小时均值作为 4 h 后的恒温干燥环境，以全域最大含水率低于 0.15 kg/kg 为条件。固定半径模型的临界时长为 57.4740 h；若按整分钟停止，应取 57.4833 h。长期环境的不同平台解释产生约 57.24—57.83 h 的情景范围，该范围不属于统计置信区间。

针对问题四，利用附件 2 的半径历史，令归一化坐标为物理半径与当前半径之比，从均匀收缩下的干物质和水分守恒推导移动域方程。临界时长为 51.0920 h，整分钟严格达标时长为 51.1000 h。采用附录 4 相同物性而保持初始半径的对照结果约为 129.8481 h，表明几何收缩在该模型中显著缩短迁移时间。

通过网格加密、通量平衡、独立离散和输入情景比较检验数值结果。两类终点的相邻外推估计差约为 1 s；但经验密度与实测收缩存在相容性限制，一维近似和气固有效边界也尚缺独立实验验证，因此所给时长是明确假设下的模型预测。

关键词：药材烘干；传热传质；有限体积法；移动边界；干燥终点

1 问题重述与分析

1.1 任务及数据

题目研究长 25 cm、初始半径 2 cm 的圆柱形药材，其初始温度为 28 °C，初始干基含水率为 2.55 kg/kg。附件 1 提供烘房初期温度及含湿量，附件 2 提供药材半径随时间的变化。所求结果不仅包含平均干燥趋势，还包括指定物理半径处的局部温度和含水率，因此必须建立具有空间分辨能力的模型 [1]。

四问逐步改变物性与几何条件：第一问计算固定半径的 30 min 预热过程；第二问在整个过程统一使用附录 3 的变物性，列示前三小时的温湿分布；第三问在同一模型中确定各处含水率均低于 0.15 kg/kg 所需时间；第四问同时采用附录 4 物性和实测收缩历史，重新确定终点。第三问承接第二问的方程和初值，而第二问并非第一问在 1800 s 后切换参数的续段。

1.2 模型选择

只描述平均水分比的经验曲线不能直接提供题目要求的径向分布，也不能用平均含水率代替“各处达标”。本文以 Fourier 导热和 Fick 有效扩散作为内部传输模型，以表面对流边界连接烘房环境。圆柱物料干燥的热质耦合属于已有研究方向 [2]；非等温移动边界模型也已用于描述食品干燥与收缩 [3]。本文只借鉴其建模层次，不移用其他物料的参数或实验精度。

药材长径比为 6.25，本文将径向结果解释为圆柱中截面的分布，采用一维轴对称近似。由常物性到变物性，再到移动边界，逐层增加题目提供的信息，避免引入无法从附件识别的力学参数。第四问的半径是观测输入，不再另设需要拟合的收缩预测方程；这与根据局部收缩速度预测几何的文献模型 [3] 有所区别。

2 模型假设与符号

2.1 基本假设

（1）药材为径向各向同性、轴对称连续介质，忽略周向差异及中段轴向梯度。端面效应尚未通过二维计算量化，不用径向网格收敛替代该项验证。

（2）第一至第三问半径不变；第四问长度不变、截面均匀径向收缩，整体半径历史适用于所研究的中截面。

（3）初始温度和干基含水率均匀。热物性与扩散率按各问指定附录取局部状态值，不另行拟合参数。

（4）附件 1、附件 2 均作分段线性插值。4 h 后的烘房环境以最后一小时均值为基准平台，末观测之后用 60 s 过渡到平台；这是长期环境假设。

（5）第二至第四问未另给表面系数，故沿用第一问的传热系数 $25 \text{ W}/(\text{m}^2 \cdot \text{K})$ 和传质系数 $8 \times 10^{-7} \text{ m/s}$ 。气相含湿量与固相干基含水率的质量基准不同，在缺少吸附等温线时，将二者之差解释为有效传质推动力。

(6) 未显式加入蒸发潜热、热湿交叉迁移和力学应力。经验密度与比热的乘积用作有效体积热容，不同时声称它与水分方程构成严格的多组分热力学闭合。

(7) 第四问干物质无损失、初始干物质体积密度均匀；在均匀收缩下，该干密度只随时间变化。此处的守恒干密度与附录经验密度分开定义，二者的相容性在第 8 节检验。

2.2 主要符号

符号	含义	单位
t	自烘干起点计的时间	s
$r, R(t), R_0$	物理半径、当前表面半径、初始表面半径	m
$\xi = r/R(t)$	随材料运动的归一化径向坐标	1
T, T_K	摄氏温度、开尔文温度	°C、K
C	固相干基含水率	kg/kg
T_∞, C_∞	烘房温度、环境含湿量	°C、kg/kg
ρ, c_p, k	经验密度、比热容、导热系数	kg/m ³ 、J/(kg·K)、W/(m·K)
D	有效水分扩散率	m ² /s
h_T, h_m	对流传热、有效传质系数	W/(m ² ·K)、m/s
ρ_d, v_r	当前干物质体积密度、径向骨架速度	kg/m ³ 、m/s
t_*, t_{60}	临界时刻、其后首个严格达标整分钟	s

所有计算距离使用 m，表格按题意显示 cm；内部温度保存为 °C，只在扩散率的 Arrhenius 项中换算为 K。后文的“水分浓度”均指题目定义的干基含水率。

3 数据准备与统一数值方法

3.1 数据核验及环境延拓

附件 1 含 241 组记录，覆盖 0—14400 s，间隔 60 s；附件 2 含 145 组记录，覆盖 0—259200 s，间隔 1800 s。提取后的两份 CSV 与原始工作簿逐值相同。半径从 2.000 cm 降至末段的 1.198 cm；原始观测值保留，不以拟合曲线代替输入。

对相邻观测点，环境或半径变量 f 采用

$$f(t) = f_i + \frac{t - t_i}{t_{i+1} - t_i} (f_{i+1} - f_i), t_i \leq t \leq t_{i+1}. \quad (1)$$

附件 1 最后一小时包含 61 个观测点，温度和含湿量均值分别为 49.998934 °C、0.04998754 kg/kg，标准差分别为 0.154353 °C、0.00015013 kg/kg。对应线性趋势约为 0.002824 °C/h 和 -0.00001288 kg/(kg·h)。尾段接近平台为长期延拓提供了依据，但不能证明未来两天环境保持不变。第 8 节单独比较不同延拓情景。

3.2 环形有限体积离散

在径向节点周围构造环形控制体。省略所有单元共有的轴向长度后，控制体体积与外侧面积分别为 $V_i = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)$ 、 $A_{i+1/2} = 2\pi r_{i+1/2}$ 。对一般传输变量 u ，令蓄积系数为 s 、输运系数为 Γ ，半离散方程为

$$s_i V_i \frac{du_i}{dt} = A_{i+1/2} \Gamma_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r} - A_{i-1/2} \Gamma_{i-1/2} \frac{u_i - u_{i-1}}{\Delta r}. \quad (2)$$

对热传导取 $u = T, s = \rho c_p, \Gamma = k$ ；对有效水分扩散取 $u = C, s = 1, \Gamma = D$ 。界面系数采用调和平均：

$$\Gamma_{i+1/2} = \frac{2\Gamma_i \Gamma_{i+1}}{\Gamma_i + \Gamma_{i+1}}. \quad (3)$$

圆心控制体的内侧面积为零，因此不直接计算 $1/r$ 的奇异表达；表面控制体将外侧项替换为题设 Robin 通量。相邻单元的内部通量大小相等、符号相反，求和后只剩边界通量，便于独立核查离散平衡。

3.3 时间积分、正性与外推

扩散离散后得到刚性常微分方程组，使用 BDF 隐式积分。水分状态设为 $z = \ln C$ ，利用 $z_t = C_t/C$ 更新，再由 $C = \exp z$ 恢复含水率，从状态表达上避免负浓度。该变换不改变原水分方程；时间步长由积分器自适应选择，输出间隔与积分步长是两个不同概念。

在共同的物理输出节点上，对间隔数为 N 、 $2N$ 的两级解作二阶 Richardson 外推：

$$u_{\text{ext}} = \frac{4u_{2N} - u_N}{3}. \quad (4)$$

外推用于减小空间误差；它依赖渐近二阶误差假设，不能保证每一个四位小数都是真值。第一问使用 5120/10240 区间，第二、三问使用 2560/5120 区间，第四问使用 640/1280 区间。第二问按 600 s 分段求解，传递完整节点终态，而非把四舍五入后的输出表当作下一段初值。

4 问题一：固定半径预热模型

4.1 方程、初值与边界

在 $0 < r < R_0 = 0.02$ m 内，采用

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right), \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C) \frac{\partial C}{\partial r} \right). \quad (5)$$

附录 2 给定 $\rho = 820$ kg/m³、 $c_p = 2600$ J/(kg·K)、 $k = 0.36$ W/(m·K)，水分扩散率为

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right) \text{m}^2/\text{s}. \quad (6)$$

初始条件为 $T(r, 0) = 28$ °C、 $C(r, 0) = 2.55$ kg/kg；圆心满足 $T_r(0, t) = C_r(0, t) = 0$ 。径向向外取正，表面边界为

-kT_r(R_0,t) = h_T[T_s - T_\infty(t)], -DC_r(R_0,t) = h_m[C_s - C_\infty(t)]. (7)

当烘房温度高于表面时，向外热通量为负，热量进入药材；当表面含水率高于有效环境值时，水分向外迁移。第一问热物性为常数、扩散率不依赖温度，因此温度与水分可以分别求解。

4.2 指定位置的计算结果

表 1、表 2 按题目时刻和位置列出结果。完整 1—1800 s、径向间隔 0.1 cm 的两类场见支撑材料 outputs/result1.xlsx。所有浓度和温度按题意保留四位小数。

表 1 代表位置温度 T/ °C

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8489
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 2 代表位置水分浓度 C/(kg/kg)

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

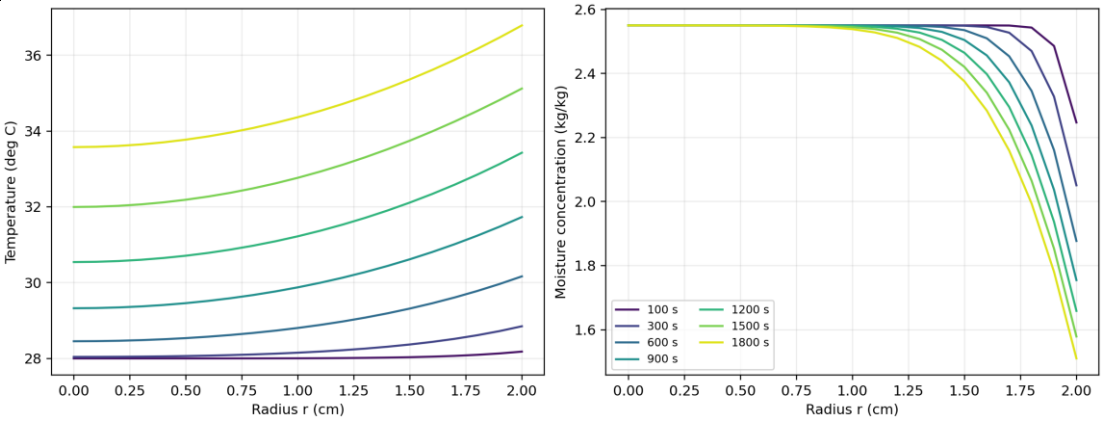


图 1 预热阶段的径向温度与含水率分布；不同曲线对应图例给出的时刻。

1800 s 时表面与中心温差为 3.2103 °C，表面含水率已降至 1.5102 kg/kg，而中心仍约为初值。较小的水分扩散率使早期失水形成明显表层梯度；因此只检查 100 s 之后的少量表格点，不能保证最初几秒的表面计算已经收敛。

5 问题二：变物性温湿耦合模型

5.1 经验物性与耦合关系

从烘干初始时刻统一采用附录 3 物性 [1]：

$$\rho = 650 + 128C, c_p = 1450 + 2736 \frac{C}{C+1}, k = 0.21 + 0.38 \frac{C}{C+1}. \quad (8)$$

$$D(C, T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left(-\frac{3850}{T_K}\right), T_K = T + 273.15. \quad (9)$$

将式（8）—（9）代入式（5），在原初值和式（7）的有效边界下联立求解。含水率通过密度、比热和导热系数影响温度，温度又通过扩散率影响水分迁移，构成非线性系数耦合。不能把局部变化的导热率移出空间导数，也不能将摄氏温度直接代入式（9）。

水分方程采用固定骨架与空间均匀干密度下的有效扩散表达。经验 $\rho(C)$ 在能量方程中提供热容系数，若同时把 $\rho(C)/(1+C)$ 强制解释为静止骨架的守恒干密度，会产生额外闭合矛盾。因此当前模型具有经验有效性质，不等同于完整的多相质量—能量方程组。

5.2 前三小时结果与全程输出

表 3 3 小时内药材温度（°C）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	32.1892	32.3821	32.9659	33.9608	35.4130
1.0	40.3816	40.5536	41.0605	41.8782	42.9976
1.5	45.8468	45.9348	46.1930	46.6051	47.1400
2.0	48.4502	48.4881	48.5984	48.7736	49.0033
2.5	49.4670	49.4792	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

表 4 3 小时内药材水分浓度（kg/kg）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5	2.5499	2.5489	2.5256	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3861	2.3256	2.1344	1.8020	1.3476
2.0	2.1709	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

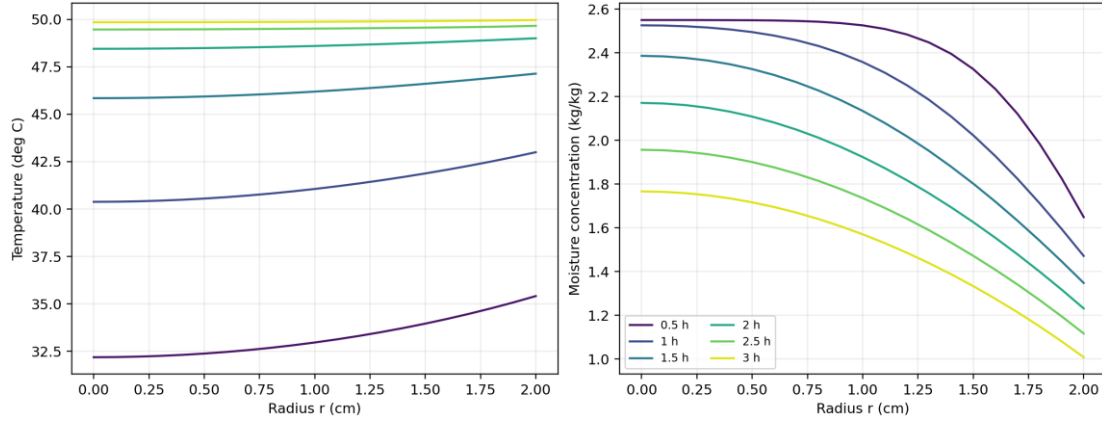


图 2 变物性模型前三小时的径向温湿分布。

3 h 时中心与表面温差仅为 0.1169 °C，但含水率差仍为 0.7581 kg/kg，且各处均未达到 0.15 kg/kg 阈值。升温基本完成后，低含水率区的扩散率继续减小，内部水分向表面的迁移仍需较长时间。这也解释了为什么不能以表面温度稳定作为烘干结束判据。

为兼顾“前三小时列表”与“整个过程完整结果”两项要求，正文只列前 3 h；全程逐秒结果延伸到第三问严格达标整分钟，保留每秒、每 0.1 cm 的温度和含水率。完整 Excel 约 27.7 MB，为满足支撑压缩包 20 MB 上限，包中以 outputs/result2_full_process.npz 无损保存四位小数交付值，并附恢复 Excel 的脚本。支撑材料中的 outputs/result2.xlsx 保留原前三小时文件；包外另提供全程 Excel 方便直接查看。

全程积分保持相同方程与初值，并使用第 3.1 节长期边界。前 4 h 最大积分步长为 2 s，此后为 60 s，相对容差为 2×10^{-10} ；输出仍为每秒。分段续算保留完整温湿节点场，延伸输出不采用已舍入分钟值插值。全程前 3 h 与原第二问交付值完全一致，与第三问独立分钟积分的最大四位小数差为 0.0001 kg/kg，来自积分分段和舍入边界差异，不能宣称两套长期输出逐值相同。

6 问题三：固定半径的干燥终点

6.1 全域阈值与严格不等式

沿用第二问方程，定义

$$g(t) = \max_{0 \leq r \leq R_0} C(r, t) - 0.15, t_* = \inf\{t: g(t) < 0\}. \quad (10)$$

在本算例终点邻域，含水率连续下降，临界时刻满足 $g(t_*) = 0$ ，严格低于在其后成立。计算对所有径向节点取最大值，不预先假定中心控制终点。外推后径向场经检查向外非增，才确认本算例最湿点位于中心。

在包含阈值的时间区间，分别对各位置的未舍入浓度进行插值，取最后达到阈值的位置所对应的时间。若直接插值相邻两行的最大值，在最湿位置改变时可能得出错误终点。按整分钟停止的时间定义为

$$t_{60} = 60 \left(\left\lceil \frac{t_*}{60} \right\rceil + 1 \right). \quad (11)$$

6.2 时长与指定结果

第三问临界时刻为 206906.2447 s（57.4740 h），首个严格达标整分钟为 206940 s（57.4833 h）。

表 5 药材烘干过程的水分浓度（kg/kg）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0548
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
临界终点 57.4740	0.1500	0.1477	0.1402	0.1249	0.0526

表 5 最后一行是连续模型的临界行，其显示值 0.1500 不代表已经满足严格小于。完整 result3.xlsx 同时保留每 60 s 数据、精确临界行与其后首个严格整分钟行，未舍入的终点判断单独保存在诊断文件。

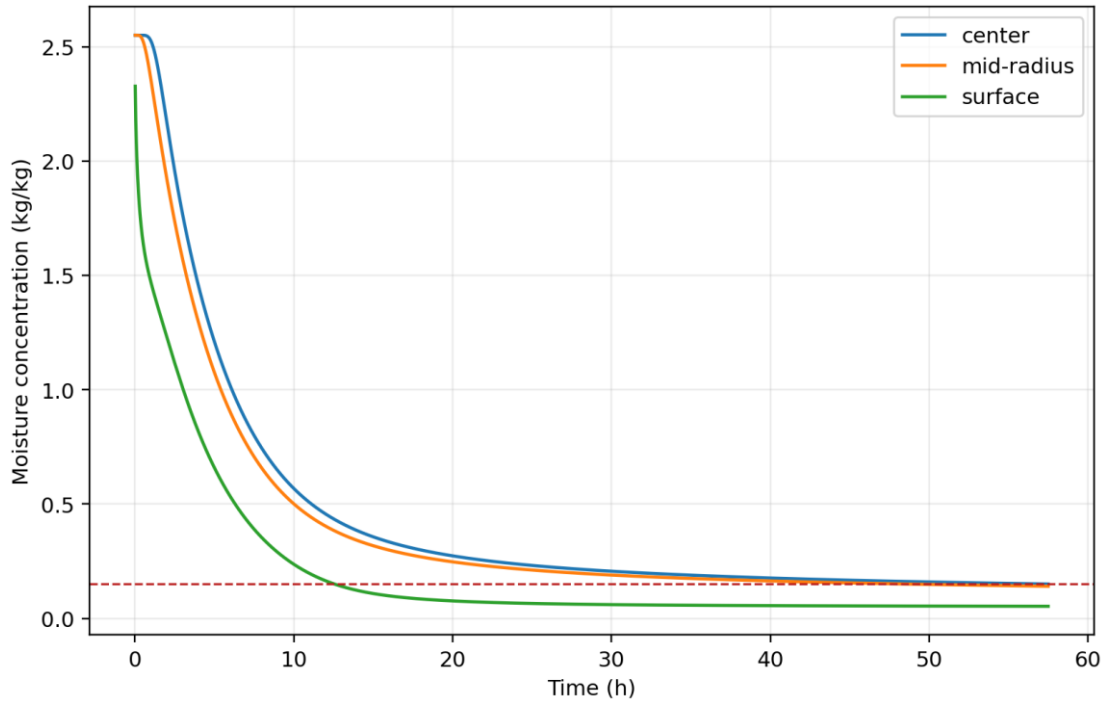


图 3 固定半径下中心、中半径和表面的长期含水率历程；虚线为 0.15 kg/kg 阈值。

表面在中心之前达到较低含水率，后期干燥速度逐渐变慢。因此平均含水率或单一表面值达标

均不足以保证内部满足要求。基准时长同时依赖长期环境平台，不能由前 4 h 数据唯一决定。

7 问题四：收缩移动边界模型

7.1 从干物质与水分守恒出发

附件 2 给出 $R(t)$ ，在均匀径向收缩、长度不变假设下，材料速度为 $v_r = (R/R)r$ 。设当前干物质体积密度为 ρ_d ，相对骨架的水分通量为 $\mathbf{j}_w$ ，则

$$\partial_t \rho_d + \nabla \cdot (\rho_d \mathbf{v}) = 0, \partial_t (\rho_d C) + \nabla \cdot (\rho_d C \mathbf{v} + \mathbf{j}_w) = 0. (12)$$

采用有效 Fick 通量 $\mathbf{j}_w = -\rho_d D \nabla C$ ，从两式消去干物质连续方程，得

$$\frac{DC}{Dt} = \frac{1}{\rho_d} \nabla \cdot (\rho_d D \nabla C). (13)$$

均匀初始干密度与均匀径向收缩给出 $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$ ，故干密度在空间上均匀，才可在式 (13) 的散度内外约去，得到

$$\frac{\partial C}{\partial t} + \frac{R}{r} \frac{\partial C}{\partial r} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial C}{\partial r} \right). (14)$$

这一推导对应干基含水率，而非单位当前体积的水分质量。直接对干基含水率添加一个体积压缩源项会改变变量的物理意义。

7.2 固定域坐标与表面通量

令 $\xi = r/R(t)$ ，则固定 ξ 的网格速度与骨架速度一致，坐标变换项与式 (14) 的对流项相消。在 $0 < \xi < 1$ 内，采用

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} \Big|_\xi = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k(C) \frac{\partial T}{\partial \xi} \right). (15)$$

$$\frac{\partial C}{\partial t} \Big|_\xi = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(C, T) \frac{\partial C}{\partial \xi} \right). (16)$$

热方程沿材料坐标采用有效热容近似，不额外计入变形功及相变热。圆心为 $T_\xi = C_\xi = 0$ ，实际表面的边界条件为

$$-\frac{k}{R} T_\xi(1, t) = h_T (T_s - T_\infty), -\frac{D}{R} C_\xi(1, t) = h_m (C_s - C_\infty). (17)$$

空间算子的 R^{-2} 和表面条件的 R^{-1} 必须同时保留；不能只改变输出半径而仍按固定圆柱求解。

附录 4 的局部物性为

$$\rho = 760 + 90C, c_p = 1850 + 2150 \frac{C}{C+1}, k = 0.12 + 0.20 \frac{C}{C+1}. (18)$$

$$D(C, T) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C}\right) \exp\left(-\frac{3850}{T_K}\right). (19)$$

7.3 固定物理位置的映射与结果

每个输出时刻均以 $\xi = r/R(t)$ 映射题目要求的固定物理位置。当 $r > R(t)$ 时，该点已位于实体之外，工作簿留空而不填零；末列“药材表面”始终取 $\xi = 1$ 的结果，不能以固定 2 cm 列代替实际表面。

第四问临界时刻为 183931.0374 s (51.0920 h)，首个严格达标整分钟为 183960 s (51.1000 h)。

表 6 收缩药材烘干过程的水分浓度 (kg/kg)

时间/h	0 cm	0.5 cm	1.0 cm	药材表面
6	1.7196	1.5377	1.0226	0.4207
12	0.7377	0.6546	0.4083	0.1669
18	0.4088	0.3687	0.2397	0.0892
24	0.2851	0.2611	0.1790	0.0673
30	0.2263	0.2094	0.1493	0.0595
36	0.1928	0.1797	0.1317	0.0560
42	0.1712	0.1604	0.1200	0.0541
48	0.1561	0.1468	0.1116	0.0530
烘干临界时刻 51.091955	0.1500	0.1413	0.1081	0.0526

6 h 时半径已为 1.374 cm，之后继续非增；因此表 6 所列时刻的 1.5 cm 和 2 cm 均在实体外，只保留 0、0.5、1 cm 及实际表面。完整 result4.xlsx 仍保留全部 0.1 cm 固定位置与真实表面列，并以空白标记实体外位置。

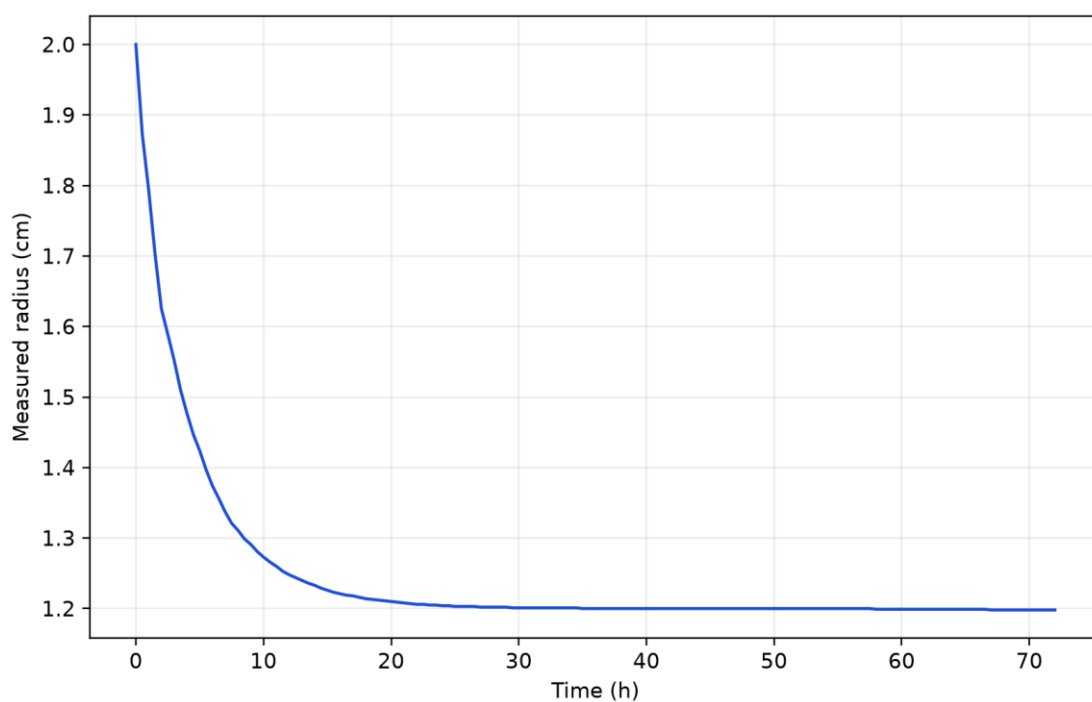


图 4 附件 2 的半径历史及分段线性插值。

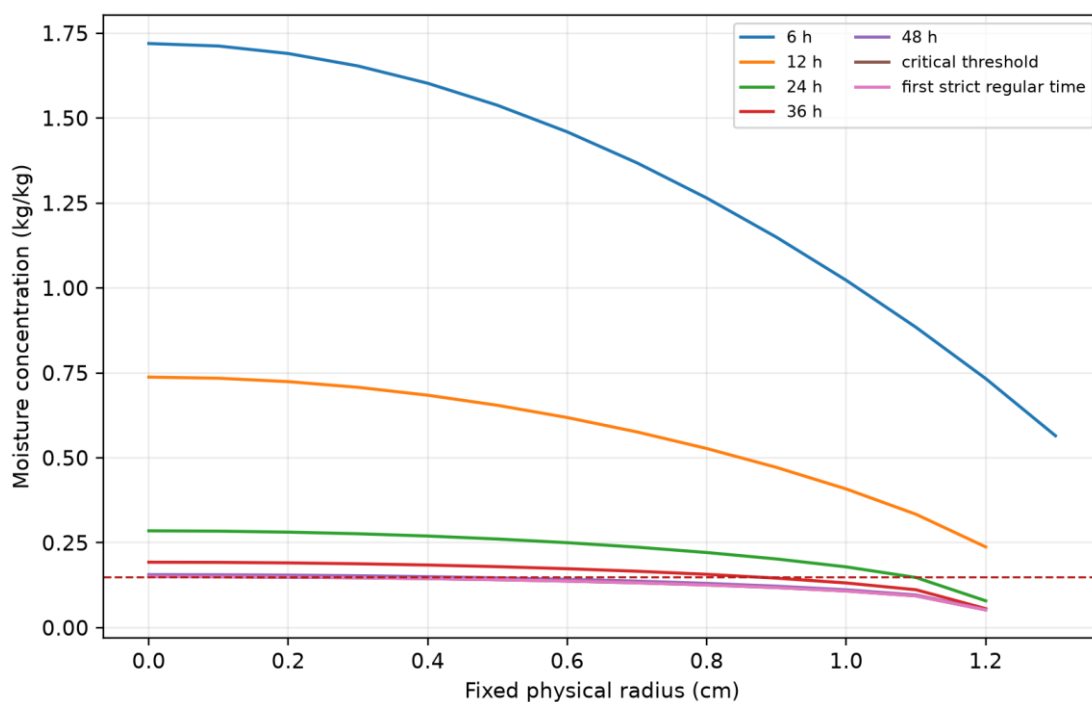


图 5 收缩药材在代表时刻的径向含水率；每条曲线右端是当时实际表面。

第三问与第四问的基准临界时长相差约 6.38 h，但两问同时改变了经验物性和几何条件，不能把该差值全归因于收缩。为分离几何效应，第 8 节采用附录 4 相同物性，另计算保持初始半径的对照模型。

8 数值检验、敏感性与模型边界

8.1 网格与程序一致性

第一问两级网格在全部输出节点的最大温度和含水率差分别约为 1.03×10^{-6} °C、 4.57×10^{-6} kg/kg；第二问前三小时对应最大差约为 8.14×10^{-7} °C、 1.49×10^{-5} kg/kg。部分值靠近四位小数舍入边界，故采用式（4）的外推值，不根据单一网格的舍入稳定性作过强判断。

表 7 两类终点的空间加密结果

模型	径向区间或外推组合	临界时长/h
第三问	1280	57.4881201
第三问	2560	57.4772860
第三问	5120	57.4747882
第三问	2560/5120 外推	57.4739569
第四问	640	51.0996704
第四问	1280	51.0938837
第四问	640/1280 外推	51.0919548

第三、四问相邻外推组合的终点差分别约为 1.02 s、0.98 s，是空间误差的量级参考，不是严格误差上界。第四问另用单元中心有限体积程序复算，1600/3200 单元外推得到 51.0919474 h；代表时刻的表 6 浓度与主程序四位小数一致。但其观测收敛阶仍低于 2，两个外推值接近不能证明真实误差同样小。

恒定平衡场、热干边界方向、固定半径退化、材料公式与 Kelvin 换算、阈值插值、分段续算和实体外空白等均纳入自动化测试。现有 95 项测试通过。原四份工作簿的 700974 个单元格经回读检查，题目表 1—6 与数值载荷一致；第二、三问重叠前三小时的 3780 个分钟含水率值完全相同。最终交付另对第二问全程逐秒工作簿进行回读，其统计见支撑材料诊断文件。

8.2 水分通量平衡

固定半径下，截面平均含水率满足

$$\bar{C}(t) - \bar{C}(0) = -\frac{2h_m}{R_0} \int_0^t [C_s(\tau) - C_\infty(\tau)] d\tau. (20)$$

移动域中定义 $\bar{C} = 2 \int_0^1 C\xi d\xi$ ，由式（16）—（17）得

$$\frac{d\bar{C}}{dt} = -\frac{2h_m}{R(t)} [C_s(t) - C_\infty(t)]. (21)$$

使用前 600 s 的逐秒输出和其后的分钟输出进行通量积分，第三问两级网格相对最大残差约为 1.21×10^{-6} ，第四问约为 1.61×10^{-6} 、 1.69×10^{-6} 。该检验说明当前有效方程与数值通量相符，但不验证未纳入方程的潜热、轴向效应或气固平衡关系。

8.3 长期环境与表面系数情景

表 8 第三问长期平台敏感性（同为 640 区间）

4 h 后环境设定	临界时长/h
最后一小时均值	57.5413
最后半小时均值	57.5151
最后观测值保持	57.2375
温度降低、含湿量升高各一个尾段标准差	57.8259
温度升高、含湿量降低各一个尾段标准差	57.2587

这些指定情景约跨越 0.59 h，明显大于网格外推差。各情景使用相同网格，比较其组内变化；不能把表中的粗网格基准与正式外推值直接混合计算灵敏度。

表 9 第四问表面系数敏感性（同为 320 区间）

变化系数	基准的 0.8 倍/h	基准/h	基准的 1.2 倍/h
传质系数	51.8009	51.1236	50.7400
传热系数	51.1418	51.1236	51.1117

在所考察的范围内，终点对传质系数的变化更明显。以上乘数是人为指定的情景，不表示参数真实误差的概率分布。温度、含湿量与传递系数仍需长期实验进一步约束。

8.4 收缩处理与固定几何对照

保持附件 2 全部观测点不变，将线性插值替换为单调 PCHIP，在相同 320 区间下终点提前约 12.603 s。另将内部半径观测时刻统一提前或推后 15 min，终点相对同组基准改变约 -0.135 h、+0.137 h。前者比较插值方法，后者改变观测时刻，二者不是同一种检验。

使用附录 4 相同物性、但固定 $R = R_0$ 的对照模型，640/1280 区间外推终点为 129.8481 h；实测收缩模型约为 51.0920 h。半径减小通过式 (16) 的扩散尺度与式 (17) 的表面尺度共同影响水分迁移。该对照隔离了所选模型中的几何差异，但并非对真实药材收缩效应的独立实验测量。

8.5 密度与收缩的相容性

如果把附录 4 的 $\rho(C)$ 强制解释为当前真实湿基密度，则应有 $\rho_d = \rho(C)/(1 + C)$ ，相应总干物质量为

$$M_d(t) = 2\pi LR(t)^2 \int_0^1 \frac{760 + 90C(\xi, t)}{1 + C(\xi, t)} \xi d\xi. \quad (22)$$

对非均匀全场积分，在 1280 区间自身临界时刻得到 $M_d(t_*)/M_d(0) \approx 0.890289$ ，隐含偏差约为 10.97%。这表明“给定密度是真实湿密度”“长度不变的实测收缩”与“干物质守恒”不能在当前解中同时严格满足。它是模型闭合诊断，不是观测到药材损失了相同比例的干物质。

本文将附录密度用于经验有效体积热容，水分守恒中的干密度由均匀收缩假设定义。这样明确

了方程的适用解释，但并未通过重新命名消除原经验关系间的不相容。若要求严格湿密度解释，需要补充孔隙率、局部变形、长度变化或其他闭合关系，再建立一致的多组分模型；现有附件不能唯一识别这些关系。

9 模型评价与结论

本文框架能够在同一离散方法下处理常物性、变物性与实测几何收缩，直接给出指定物理位置的结果。以全域最大值判定终点避免平均值掩盖内部未干区域；对实际表面与实体外空白的区分保证了第四问数据的几何含义。逐式核验、网格外推、通量平衡和独立程序比较构成了可复现的数值检查链。

主要局限集中于模型结构与输入解释：一维中截面近似未量化端面影响；长期环境由短期记录延拓；气固传质采用有效浓度差；表面系数在后续问题中沿用；潜热和严格密度一收缩闭合没有纳入。缺少独立内部含水率观测时，不能将网格收敛或不同程序吻合写成物理预测已经准确验证。

在上述条件下，第一问 1800 s 的中心与表面温度分别为 33.5753、36.7856 °C；第二问 3 h 温度接近均匀而水分仍明显不均。第三问固定半径的临界时长为 57.4740 h，第四问实测收缩的临界时长为 51.0920 h；按整分钟停止分别取 57.4833 h、51.1000 h。未来优先补充长期烘房记录、内部温湿测量和气固平衡关系，并开展二维及严格质量闭合验证，其收益可能高于继续加密已有一维网格。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型推导辅助、代码实现与调试、数值复算和结果核查、论文初稿组织及语言润色，详细使用情况见支撑材料。

研究与写作使用 Scientific Agent Skills [4] 协助检查证据来源、模型局限和文档结构；该软件流程不构成结果准确性的证明。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 A 题：药材的烘干问题及附件[Z]. 2026.
- [2] DA SILVA W P, E SILVA C M D P S, GAMA F J A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. Journal of Food Engineering, 2014, 141: 65-73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [3] ADROVER A, VENDITTI C, BRASIELLO A. A non-isothermal moving-boundary model for continuous and intermittent drying of pears[J]. Foods, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.
- [4] KASSIS T, AGARWAL V, HE Y, PATEL D, BRUECKNER A M. Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents[EB/OL]. 2026. DOI: 10.48550/arXiv.2609.00065.

附录 A 支撑材料与复现说明

支撑材料 ZIP 内的文件列表如下。论文电子版从摘要页开始，不含承诺书和编号专用页。文中四位小数用于题设输出，终点判定始终使用未舍入结果。

AI 工具使用详情.pdf

AI 工具使用详情_可编辑版.docx

README_复现说明.md

data/raw/attachment1.csv

data/raw/attachment2.csv

docs/literature-review.md

docs/problem1-formula-audit.md

docs/problem2-formula-audit.md

docs/problem3-formula-audit.md

docs/problem4-formula-audit.md

docs/论文证据索引.md

outputs/result1.xlsx

outputs/result2.xlsx

outputs/result2_full_process.npz

outputs/result3.xlsx

outputs/result4.xlsx

pyproject.toml

reports/data/final_all_questions_audit.json

reports/data/problem2_full_process_audit.json

reports/data/problem4_production_diagnostics.json

reports/data/problem4_review_experiments.json

reports/data/problem4_workbook_audit.json

reports/figures/problem1_radial_profiles.png

reports/figures/problem1_time_histories.png

reports/figures/problem2_radial_profiles.png

reports/figures/problem2_time_histories.png

reports/figures/problem3_radial_profiles.png

reports/figures/problem3_time_histories.png
reports/figures/problem4_radial_profiles.png
reports/figures/problem4_radius_history.png
reports/figures/problem4_time_histories.png
reports/final-review.md
reports/problem1.md
reports/problem2.md
reports/problem3.md
reports/problem4-review-response.md
reports/problem4.md
reports/problem4_independent_validation.json
reports/verification.md
requirements.txt
scripts/run_problem1.py
scripts/run_problem2.py
scripts/run_problem3.py
scripts/run_problem4.py
scripts/verify_problem4.py
scripts/verify_problem4_independent.py
src/drying_model/__init__.py
src/drying_model/problem1.py
src/drying_model/problem2.py
src/drying_model/problem3.py
src/drying_model/problem4.py
src/drying_model/problem4_independent.py
tests/test_final_review_regressions.py
tests/test_problem1.py
tests/test_problem2.py
tests/test_problem3.py
tests/test_problem4.py

```
tests/test_problem4_independent.py
tests/test_result4_workbook.py
tools/audit_all_results.py
tools/build_result4.py
tools/compress_problem2_full_process.py
tools/extend_problem2_delivery.py
tools/rebuild_delivery_workbooks.py
tools/restore_full_result2.py
```

A.1 运行顺序

先在支撑材料根目录安装 requirements.txt 中的依赖。Python 版本须为 3.11 或以上。Windows PowerShell 下设置环境变量后，按下面顺序运行；Linux/macOS 可将环境变量设置语句替换为 export PYTHONPATH=src。

```
python -m pip install -r requirements.txt
$env:PYTHONPATH='src'
python -m pytest -q
python scripts/run_problem1.py
python scripts/run_problem2.py
python scripts/run_problem3.py
python scripts/run_problem4.py
python tools/rebuild_delivery_workbooks.py
python tools/extend_problem2_delivery.py
```

四个求解脚本产生 tmp/problem1_result.json 至 tmp/problem4_result.json。工作簿重建脚本不依赖原始 Excel 模板或专用表格服务；全程扩展脚本独立求解两级网格并逐单元回读。复现所需 CSV 已包含在支撑材料中。

若仅需查看全程第二问 Excel，无须重新求解，在支撑材料根目录执行 python tools/restore_full_result2.py，即可由压缩数组还原 outputs/result2_full_process.xlsx。

A.2 数值记录与人工核验

reports/data 中保存原四问审计、第四问敏感性和密度诊断、全程逐秒结果审计。程序核验与参赛队人工审查是两类不同记录；AI 工具使用详情仅记录可确认的自动检查，不代替或虚构人工核验。

附录 B 完整源程序

下列代码为本次建模、输出与验证所需源文件全文。保留文件相对路径与代码缩进，实际运行应使用支撑材料中的原文件。附录页数不计入正文限制。

scripts/run_problem1.py

```
"""Solve Problem 1 and export workbook-ready values plus diagnostic figures."""
```

```

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from drying_model.problem1 import (
    Problem1Parameters,
    load_chamber_history_csv,
    richardson_extrapolate_solutions,
    result_payload,
    sample_solution,
    solve_problem1,
)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("data/raw/attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("tmp/problem1_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("reports/figures"))
    parser.add_argument("--coarse-radial-intervals", type=int, default=5120)
    parser.add_argument("--fine-radial-intervals", type=int, default=10240)
    return parser

def save_figures(payload: dict[str, list], figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time = np.asarray(payload["time_s"], dtype=float)
    radius = np.asarray(payload["radius_cm"], dtype=float)
    temperature = np.asarray(payload["temperature_c"], dtype=float)
    moisture = np.asarray(payload["moisture_concentration"], dtype=float)

    profile_times = [100, 300, 600, 900, 1200, 1500, 1800]
    colors = plt.cm.viridis(np.linspace(0.05, 0.95, len(profile_times)))
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
    for selected_time, color in zip(profile_times, colors, strict=True):
        row = selected_time - 1
        axes[0].plot(radius, temperature[row], color=color, label=f"{selected_time} s")
        axes[1].plot(radius, moisture[row], color=color, label=f"{selected_time} s")
    axes[0].set(xlabel="Radius r (cm)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Radius r (cm)", ylabel="Moisture concentration (kg/kg)")
    for axis in axes:
        axis.grid(alpha=0.25)
    axes[1].legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem1_radial_profiles.png", dpi=220)
    plt.close(fig)

    locations = [(0, "center"), (10, "mid-radius"), (20, "surface")]
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
    for column, label in locations:
        axes[0].plot(time, temperature[:, column], label=label)
        axes[1].plot(time, moisture[:, column], label=label)
    axes[0].set(xlabel="Time (s)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Time (s)", ylabel="Moisture concentration (kg/kg)")
    for axis in axes:
        axis.grid(alpha=0.25)
        axis.legend()
    fig.savefig(figure_dir / "problem1_time_histories.png", dpi=220)
    plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()

```

```

if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
    raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
history = load_chamber_history_csv(args.input)
output_times = np.arange(1.0, 1801.0)
requested_radii_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
sampled_solutions = []
for radial_intervals in (
    args.coarse_radial_intervals,
    args.fine_radial_intervals,
):
    solution = solve_problem1(
        history,
        radial_intervals,
        output_times,
        parameters=Problem1Parameters(),
        relative_tolerance=5.0e-10,
        max_step_s=1.0,
    )
    sampled_solutions.append(sample_solution(solution, requested_radii_cm))
extrapolated = richardson_extrapolate_solutions(
    sampled_solutions[0],
    sampled_solutions[1],
    order=2,
)
payload = result_payload(extrapolated)
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(
    json.dumps(payload, ensure_ascii=False, separators=(",", ":")),
    encoding="utf-8",
)
save_figures(payload, args.figure_dir)

if __name__ == "__main__":
    main()

```

scripts/run_problem2.py

```

"""Solve Problem 2 and export workbook-ready values plus diagnostic figures."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from drying_model.problem1 import (
    load_chamber_history_csv,
    result_payload,
    richardson_extrapolate_solutions,
)
from drying_model.problem2 import diffusivity_q2, solve_problem2_sampled_in_chunks

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("data/raw/attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("tmp/problem2_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("reports/figures"))
    parser.add_argument("--coarse-radial-intervals", type=int, default=2560)
    parser.add_argument("--fine-radial-intervals", type=int, default=5120)
    parser.add_argument("--chunk-duration-s", type=float, default=600.0)

```

```

return parser

def save_figures(payload: dict[str, list], figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time = np.asarray(payload["time_s"], dtype=float)
    radius = np.asarray(payload["radius_cm"], dtype=float)
    temperature = np.asarray(payload["temperature_c"], dtype=float)
    moisture = np.asarray(payload["moisture_concentration"], dtype=float)

    profile_times = [1800, 3600, 5400, 7200, 9000, 10800]
    colors = plt.cm.viridis(np.linspace(0.05, 0.95, len(profile_times)))
    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.2), constrained_layout=True)
    for selected_time, color in zip(profile_times, colors, strict=True):
        row = selected_time - 1
        label = f"{selected_time / 3600:g} h"
        axes[0].plot(radius, temperature[row], color=color, label=label)
        axes[1].plot(radius, moisture[row], color=color, label=label)
    axes[0].set(xlabel="Radius r (cm)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Radius r (cm)", ylabel="Moisture concentration (kg/kg)")
    for axis in axes:
        axis.grid(alpha=0.25)
    axes[1].legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem2_radial_profiles.png", dpi=220)
    plt.close(fig)

    locations = [(0, "center"), (10, "mid-radius"), (20, "surface")]
    fig, axes = plt.subplots(1, 3, figsize=(14.0, 4.2), constrained_layout=True)
    for column, label in locations:
        axes[0].plot(time / 3600.0, temperature[:, column], label=label)
        axes[1].plot(time / 3600.0, moisture[:, column], label=label)
        axes[2].plot(
            time / 3600.0,
            diffusivity_q2(moisture[:, column], temperature[:, column]),
            label=label,
        )
    axes[0].set(xlabel="Time (h)", ylabel="Temperature (deg C)")
    axes[1].set(xlabel="Time (h)", ylabel="Moisture concentration (kg/kg)")
    axes[2].set(xlabel="Time (h)", ylabel="Diffusivity (m2/s)")
    for axis in axes:
        axis.grid(alpha=0.25)
        axis.legend()
    fig.savefig(figure_dir / "problem2_time_histories.png", dpi=220)
    plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
    history = load_chamber_history_csv(args.input)
    output_times = np.arange(1.0, 10801.0)
    sample_radius_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
    solutions = []
    for radial_intervals in (
        args.coarse_radial_intervals,
        args.fine_radial_intervals,
    ):
        print(f"Solving Problem 2 on {radial_intervals} radial intervals...", flush=True)
        solutions.append(
            solve_problem2_sampled_in_chunks(
                history,
                radial_intervals=radial_intervals,
                output_times_s=output_times,
                sample_radius_cm=sample_radius_cm,
                chunk_duration_s=args.chunk_duration_s,
                relative_tolerance=2.0e-10,
                max_step_s=2.0,
            )
        )

```

```

        print(f"Completed {radial_intervals} radial intervals.", flush=True)
    coarse, fine = solutions
    extrapolated = richardson_extrapolate_solutions(coarse, fine, order=2)
    payload = result_payload(extrapolated)
    payload["numerical_method"] = {
        "coarse_radial_intervals": args.coarse_radial_intervals,
        "fine_radial_intervals": args.fine_radial_intervals,
        "richardson_order": 2,
        "relative_tolerance": 2.0e-10,
        "maximum_time_step_s": 2.0,
        "chunk_duration_s": args.chunk_duration_s,
        "max_abs_temperature_difference_c": float(
            np.max(np.abs(fine.temperature_c - coarse.temperature_c))
        ),
        "max_abs_moisture_difference": float(
            np.max(
                np.abs(
                    fine.moisture_concentration
                    - coarse.moisture_concentration
                )
            )
        ),
        "temperature_four_decimal_mismatches": int(
            np.count_nonzero(
                np.round(fine.temperature_c, 4)
                != np.round(coarse.temperature_c, 4)
            )
        ),
        "moisture_four_decimal_mismatches": int(
            np.count_nonzero(
                np.round(fine.moisture_concentration, 4)
                != np.round(coarse.moisture_concentration, 4)
            )
        ),
    }
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(
        json.dumps(payload, ensure_ascii=False, separators=(",", ":")),
        encoding="utf-8",
    )
    save_figures(payload, args.figure_dir)

if __name__ == "__main__":
    main()

```

scripts/run_problem3.py

```

"""Solve Problem 3, locate the drying endpoint, and export workbook-ready values."""

from __future__ import annotations

import argparse
import gc
import json
from dataclasses import replace
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

from drying_model.problem1 import (
    load_chamber_history_csv,
    richardson_extrapolate_solutions,
    sample_solution,

```

```

)
from drying_model.problem2 import solve_problem2
from drying_model.problem3 import (
    Problem3Result,
    extend_chamber_history_to_plateau,
    moisture_balance_diagnostics,
    problem3_payload,
    richardson_extrapolate_drying_time,
    solve_problem3,
    truncate_solution_at_threshold,
    validate_radially_nonincreasing_moisture,
)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--input", type=Path, default=Path("data/raw/attachment1.csv"))
    parser.add_argument("--output", type=Path, default=Path("tmp/problem3_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("reports/figures"))
    parser.add_argument("--coarse-radial-intervals", type=int, default=2560)
    parser.add_argument("--fine-radial-intervals", type=int, default=5120)
    parser.add_argument("--sensitivity-radial-intervals", type=int, default=640)
    parser.add_argument("--horizon-hours", type=float, default=96.0)
    return parser

def save_figures(result: Problem3Result, figure_dir: Path) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    solution = result.solution
    time_h = solution.time_s / 3600.0
    radius_cm = solution.radius_m * 100.0
    moisture = solution.moisture_concentration

    selected_hours = [6.0, 12.0, 24.0, 36.0, 48.0]
    selected_hours = [hour for hour in selected_hours if hour < time_h[-1]]
    selected_indices = [int(np.argmin(np.abs(time_h - hour))) for hour in selected_hours]
    critical_index = int(
        np.flatnonzero(
            np.isclose(solution.time_s, result.drying_time_s, atol=1.0e-8, rtol=0.0)
        )[0]
    )
    selected_indices.extend([critical_index, len(time_h) - 1])
    selected_indices = list(dict.fromkeys(selected_indices))
    fig, axis = plt.subplots(figsize=(7.2, 4.6), constrained_layout=True)
    for index in selected_indices:
        if index == critical_index:
            label = "critical threshold"
        elif index == len(time_h) - 1:
            label = "first strict regular time"
        else:
            label = f"{time_h[index]:g} h"
        axis.plot(radius_cm, moisture[index], label=label)
    axis.axhline(result.threshold, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(
        xlabel="Radius r (cm)",
        ylabel="Moisture concentration (kg/kg)",
    )
    axis.grid(alpha=0.25)
    axis.legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem3_radial_profiles.png", dpi=220)
    plt.close(fig)

    fig, axis = plt.subplots(figsize=(7.2, 4.6), constrained_layout=True)
    for column, label in ((0, "center"), (10, "mid-radius"), (20, "surface")):
        axis.plot(time_h, moisture[:, column], label=label)
    axis.axhline(result.threshold, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(
        xlabel="Time (h)",
        ylabel="Moisture concentration (kg/kg)",
    )

```

```

axis.grid(alpha=0.25)
axis.legend()
fig.savefig(figure_dir / "problem3_time_histories.png", dpi=220)
plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Fine radial grid must have twice as many intervals as the coarse grid")
    history = load_chamber_history_csv(args.input)
    horizon_s = args.horizon_hours * 3600.0
    solver_options = {"relative_tolerance": 2.0e-9, "max_step_s": 60.0}

    endpoint_results = []
    for radial_intervals in (
        args.coarse_radial_intervals,
        args.fine_radial_intervals,
    ):
        endpoint = solve_problem3(
            history,
            radial_intervals,
            horizon_s,
            output_interval_s=3600.0,
            **solver_options,
        )
        endpoint_results.append(endpoint)
        print(
            f"endpoint grid={radial_intervals}: ",
            f"{endpoint.drying_time_s:.6f} s",
            flush=True,
        )
    extrapolated_time = richardson_extrapolate_drying_time(
        endpoint_results[0].drying_time_s,
        endpoint_results[1].drying_time_s,
        order=2,
    )

    common_end_time = extrapolated_time + 60.0
    extended_history = extend_chamber_history_to_plateau(history, common_end_time)
    regular_times = np.arange(60.0, common_end_time + 1.0e-9, 60.0)
    common_times = np.unique(
        np.append(
            regular_times,
            [extrapolated_time - 60.0, extrapolated_time + 60.0],
        )
    )
    diagnostic_times = np.unique(
        np.concatenate([np.arange(0.0, 601.0), common_times])
    )
    requested_radai_cm = np.arange(0.0, 2.0 + 0.05, 0.1)
    sampled_solutions = []
    moisture_balance: dict[str, dict[str, float]] = {}
    radial_monotonicity: dict[str, float] = {}
    for radial_intervals in (
        args.coarse_radial_intervals,
        args.fine_radial_intervals,
    ):
        full_solution = solve_problem2(
            extended_history,
            radial_intervals,
            diagnostic_times,
            **solver_options,
        )
        moisture_balance[str(radial_intervals)] = moisture_balance_diagnostics(
            full_solution,
            extended_history,
        )
        radial_monotonicity[str(radial_intervals)] = (
            validate_radially_nonincreasing_moisture(full_solution)

```

```

    )
    sampled_full_solution = sample_solution(
        full_solution,
        requested_radii_cm,
    )
    common_indices = np.searchsorted(diagnostic_times, common_times)
    sampled_solutions.append(
        replace(
            sampled_full_solution,
            time_s=sampled_full_solution.time_s[common_indices],
            temperature_c=sampled_full_solution.temperature_c[common_indices],
            moisture_concentration=(
                sampled_full_solution.moisture_concentration[common_indices]
            ),
        )
    )
    del full_solution
    del sampled_full_solution
    gc.collect()
    print(f"sampld field grid={radial_intervals}", flush=True)
    extrapolated_solution = richardson_extrapolate_solutions(
        sampled_solutions[0],
        sampled_solutions[1],
        order=2,
    )
    maximum_outward_increase = validate_radially_nonincreasing_moisture(
        extrapolated_solution
    )
    output_solution = truncate_solution_at_threshold(
        extrapolated_solution,
        threshold=0.15,
        output_interval_s=60.0,
    )
    critical_index = output_solution.time_s.size - 2
    critical_moisture = output_solution.moisture_concentration[critical_index]
    maximum_index = int(np.argmax(critical_moisture))
    result = Problem3Result(
        solution=output_solution,
        drying_time_s=float(output_solution.time_s[critical_index]),
        strict_completion_time_s=float(output_solution.time_s[-1]),
        maximum_moisture=float(critical_moisture[maximum_index]),
        maximum_radius_m=float(output_solution.radius_m[maximum_index]),
        plateau_temperature_c=float(extended_history.temperature_c[-1]),
        plateau_moisture_concentration=float(
            extended_history.moisture_concentration[-1]
        ),
        threshold=0.15,
        output_interval_s=60.0,
    )
)

tail_half_hour = history.time_s >= history.time_s[-1] - 1800.0
tail_hour = history.time_s >= history.time_s[-1] - 3600.0
temperature_mean = float(np.mean(history.temperature_c[tail_hour]))
moisture_mean = float(np.mean(history.moisture_concentration[tail_hour]))
temperature_std = float(np.std(history.temperature_c[tail_hour], ddof=1))
moisture_std = float(np.std(history.moisture_concentration[tail_hour], ddof=1))
scenarios = {
    "last_hour_mean": (temperature_mean, moisture_mean),
    "last_half_hour_mean": (
        float(np.mean(history.temperature_c[tail_half_hour])),
        float(np.mean(history.moisture_concentration[tail_half_hour])),
    ),
    "last_observation": (
        float(history.temperature_c[-1]),
        float(history.moisture_concentration[-1]),
    ),
    "cool_wet_one_standard_deviation": (
        temperature_mean - temperature_std,
        moisture_mean + moisture_std,
    ),
}

```

```

        "warm_dry_one_standard_deviation": (
            temperature_mean + temperature_std,
            max(0.0, moisture_mean - moisture_std),
        ),
    }
}
sensitivity: dict[str, dict[str, float]] = {}
for name, (plateau_temperature, plateau_moisture) in scenarios.items():
    scenario = solve_problem3(
        history,
        args.sensitivity_radial_intervals,
        horizon_s,
        output_interval_s=3600.0,
        plateau_temperature_c=plateau_temperature,
        plateau_moisture_concentration=plateau_moisture,
        **solver_options,
    )
    sensitivity[name] = {
        "plateau_temperature_c": plateau_temperature,
        "plateau_moisture_concentration": plateau_moisture,
        "drying_time_s": scenario.drying_time_s,
        "drying_time_h": scenario.drying_time_s / 3600.0,
    }
    print(
        f"sensitivity {name}: {scenario.drying_time_s / 3600.0:.6f} h",
        flush=True,
    )

payload = problem3_payload(result)
payload["grid_endpoint_s"] = {
    str(args.coarse_radial_intervals): endpoint_results[0].drying_time_s,
    str(args.fine_radial_intervals): endpoint_results[1].drying_time_s,
    "richardson": result.drying_time_s,
}
payload["moisture_balance"] = moisture_balance
payload["radial_monotonicity"] = {
    "center_is_wettest_at_all_output_times": True,
    "maximum_outward_increase_by_grid": radial_monotonicity,
    "extrapolated_sampled_maximum_outward_increase": maximum_outward_increase,
}
payload["sensitivity"] = sensitivity
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(
    json.dumps(payload, ensure_ascii=False, separators=(",", ":")),
    encoding="utf-8",
)
save_figures(result, args.figure_dir)

if __name__ == "__main__":
    main()

```

scripts/run_problem4.py

```

"""Solve Problem 4, perform grid checks, and create workbook-ready payloads."""

from __future__ import annotations

import argparse
import hashlib
import json
from pathlib import Path

import matplotlib
import numpy as np

matplotlib.use("Agg")
import matplotlib.pyplot as plt

```

```

from drying_model.problem1 import load_chamber_history_csv
from drying_model.problem3 import extend_chamber_history_to_plateau
from drying_model.problem4 import (
    RadiusHistory,
    load_radius_history_csv,
    moisture_balance_diagnostics,
    sample_moving_solution,
    sample_surface,
    solve_problem4,
    validate_radially_nonincreasing_moisture,
)

def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--chamber", type=Path, default=Path("data/raw/attachment1.csv"))
    parser.add_argument("--radius", type=Path, default=Path("data/raw/attachment2.csv"))
    parser.add_argument("--output", type=Path, default=Path("tmp/problem4_result.json"))
    parser.add_argument("--figure-dir", type=Path, default=Path("reports/figures"))
    parser.add_argument("--diagnostics-output", type=Path,
    default=Path("reports/data/problem4_production_diagnostics.json"))
    parser.add_argument("--coarse-radial-intervals", type=int, default=640)
    parser.add_argument("--fine-radial-intervals", type=int, default=1280)
    parser.add_argument("--horizon-hours", type=float, default=72.0)
    return parser

def richardson(coarse: np.ndarray, fine: np.ndarray, order: int = 2) -> np.ndarray:
    """Second-order Richardson extrapolation for equal output grids."""
    if coarse.shape != fine.shape:
        raise ValueError("Richardson inputs must have equal shapes")
    return fine + (fine - coarse) / (2.0*order - 1.0)

def interpolate_rows(values: np.ndarray, times: np.ndarray, query: float) -> np.ndarray:
    """Linearly interpolate each spatial column at one time."""
    if query < times[0] or query > times[-1]:
        raise ValueError("Interpolation query lies outside the solution")
    index = int(np.searchsorted(times, query))
    if index == 0:
        return values[0].copy()
    if index == times.size:
        return values[-1].copy()
    if np.isclose(times[index], query, atol=1.0e-10, rtol=0.0):
        return values[index].copy()
    fraction = (query - times[index - 1]) / (times[index] - times[index - 1])
    return values[index - 1] + fraction * (values[index] - values[index - 1])

def write_diagnostics(payload: dict, path: Path, chamber_path: Path, radius_path: Path) -> None:
    """Retain compact production evidence alongside the generated workbook."""
    omitted = {"time_s", "radius_cm", "moisture_concentration", "surface_moisture_concentration"}
    evidence = {key: value for key, value in payload.items() if key not in omitted}
    evidence["input_sha256"] = {p.as_posix(): hashlib.sha256(p.read_bytes()).hexdigest() for p in
(chamber_path, radius_path)}
    evidence["data_rows"] = len(payload["time_s"])
    evidence["table6_columns"] = ["time_h", "center", "0.5_cm", "1.0_cm", "surface"]
    evidence["table6"] = []
    for hour in range(6, 49, 6):
        if hour * 3600 not in payload["time_s"]:
            continue
        index = payload["time_s"].index(hour * 3600)
        row = payload["moisture_concentration"][index]
        evidence["table6"].append([hour, row[0], row[5], row[10],
payload["surface_moisture_concentration"][index]])
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(evidence, indent=2, ensure_ascii=False, allow_nan=False) + "\n",
encoding="utf-8")

```

```

def save_figures(
    times_s: np.ndarray,
    radius_cm: np.ndarray,
    moisture: np.ndarray,
    surface: np.ndarray,
    radius_history,
    drying_time_s: float,
    figure_dir: Path,
) -> None:
    figure_dir.mkdir(parents=True, exist_ok=True)
    time_h = times_s / 3600.0
    critical_index = int(np.argmax(np.abs(times_s - drying_time_s)))
    selected_hours = [6.0, 12.0, 24.0, 36.0, 48.0]
    selected = [int(np.argmax(np.abs(time_h - value))) for value in selected_hours if value <= time_h[-
1]]
    selected.extend([critical_index, len(times_s) - 1])
    selected = list(dict.fromkeys(selected))
    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    for index in selected:
        label = f"{time_h[index]:g} h"
        if np.isclose(times_s[index], drying_time_s, atol=1.0e-6, rtol=0.0):
            label = "critical threshold"
        elif index == len(times_s) - 1:
            label = "first strict regular time"
        axis.plot(radius_cm, moisture[index], marker="", label=label)
    axis.axhline(0.15, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(xlabel="Fixed physical radius (cm)", ylabel="Moisture concentration (kg/kg)")
    axis.grid(alpha=0.25)
    axis.legend(ncol=2, fontsize=8)
    fig.savefig(figure_dir / "problem4_radial_profiles.png", dpi=220)
    plt.close(fig)

    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    center = moisture[:, 0]
    axis.plot(time_h, center, label="center")
    axis.plot(time_h, surface, label="actual surface")
    axis.axhline(0.15, color="#B91C1C", linestyle="--", linewidth=1.0)
    axis.set(xlabel="Time (h)", ylabel="Moisture concentration (kg/kg)")
    axis.grid(alpha=0.25)
    axis.legend()
    fig.savefig(figure_dir / "problem4_time_histories.png", dpi=220)
    plt.close(fig)

    fig, axis = plt.subplots(figsize=(7.4, 4.7), constrained_layout=True)
    axis.plot(radius_history.time_s / 3600.0, radius_history.radius_m * 100.0, color="#1D4ED8")
    axis.set(xlabel="Time (h)", ylabel="Measured radius (cm)")
    axis.grid(alpha=0.25)
    fig.savefig(figure_dir / "problem4_radius_history.png", dpi=220)
    plt.close(fig)

def main() -> None:
    args = build_parser().parse_args()
    chamber = load_chamber_history_csv(args.chamber)
    radius_history = load_radius_history_csv(args.radius)
    if args.fine_radial_intervals != 2 * args.coarse_radial_intervals:
        raise ValueError("Second-order Richardson requires fine intervals = 2 * coarse intervals")
    horizon_s = float(args.horizon_hours * 3600.0)
    extended_chamber = extend_chamber_history_to_plateau(chamber, horizon_s)
    threshold = 0.15
    solver_options = {"relative_tolerance": 2.0e-8, "max_step_s": 120.0}
    endpoint_times: dict[str, float] = {}
    for grid in (args.coarse_radial_intervals, args.fine_radial_intervals):
        probe_times = np.unique(np.append(np.arange(3600.0, horizon_s, 3600.0), horizon_s))
        endpoint = solve_problem4(
            extended_chamber,
            radius_history,
            grid,
            probe_times,
            maximum_moisture_threshold=threshold,

```

```

        **solver_options,
    )
    endpoint_times[str(grid)] = float(endpoint.time_s[-1])
    print(f"endpoint grid={grid}: {endpoint.time_s[-1]:.6f} s", flush=True)
    drying_time_s = float(
        endpoint_times[str(args.fine_radial_intervals)]
        + (endpoint_times[str(args.fine_radial_intervals)] -
        endpoint_times[str(args.coarse_radial_intervals)]) / 3.0
    )
    timing_sensitivity: dict[str, float] = {}
    sensitivity_grid = max(160, args.coarse_radial_intervals // 2)
    for name, shift_s in (("half_interval_early", -900.0), ("baseline", 0.0), ("half_interval_late",
900.0)):
        shifted_times = radius_history.time_s.copy()
        shifted_times[1:-1] += shift_s
        shifted_radius = RadiusHistory(shifted_times, radius_history.radius_m.copy())
        sensitivity_solution = solve_problem4(
            extended_chamber,
            shifted_radius,
            sensitivity_grid,
            probe_times,
            maximum_moisture_threshold=threshold,
            **solver_options,
        )
        timing_sensitivity[name] = float(sensitivity_solution.time_s[-1])
    if not (0.0 < drying_time_s < horizon_s):
        raise RuntimeError("Extrapolated drying endpoint is outside the supplied chamber horizon")
    strict_time_s = float(np.floor(drying_time_s / 60.0 + 1.0) * 60.0)
    if strict_time_s <= drying_time_s + 1.0e-8:
        strict_time_s += 60.0

    regular_times = np.arange(60.0, strict_time_s + 1.0e-9, 60.0)
    bracket = np.array([drying_time_s - 60.0, drying_time_s + 60.0])
    integration_times = np.unique(np.concatenate([regular_times, bracket]))
    fixed_radius_cm = np.round(np.arange(0.0, 2.0 + 0.0001, 0.1), 10)
    sampled_by_grid: list[np.ndarray] = []
    surface_by_grid: list[np.ndarray] = []
    balance_by_grid: dict[str, dict[str, float]] = {}
    monotonicity_by_grid: dict[str, float] = {}
    diagnostic_times = np.unique(np.concatenate([np.arange(0.0, 601.0), integration_times]))
    for grid in (args.coarse_radial_intervals, args.fine_radial_intervals):
        full = solve_problem4(
            extended_chamber,
            radius_history,
            grid,
            diagnostic_times,
            **solver_options,
        )
        balance_by_grid[str(grid)] = moisture_balance_diagnostics(
            full, extended_chamber, radius_history
        )
        monotonicity_by_grid[str(grid)] = validate_radially_nonincreasing_moisture(full)
        sampled_full = sample_moving_solution(full, radius_history, fixed_radius_cm)
        surface_full = sample_surface(full)
        retained = np.searchsorted(diagnostic_times, integration_times)
        sampled_by_grid.append(sampled_full[retained])
        surface_by_grid.append(surface_full[retained])
        del full
        print(f"full output grid={grid} complete", flush=True)

    extrapolated = richardson(sampled_by_grid[0], sampled_by_grid[1])
    extrapolated_surface = richardson(surface_by_grid[0], surface_by_grid[1])
    critical_fixed = interpolate_rows(extrapolated, integration_times, drying_time_s)
    critical_surface = float(interpolate_rows(extrapolated_surface[:, None], integration_times,
drying_time_s)[0])
    strict_index = int(np.where(np.isclose(integration_times, strict_time_s, atol=1.0e-9,
rtol=0.0))[0][0])
    strict_fixed = extrapolated[strict_index]
    strict_surface = float(extrapolated_surface[strict_index])
    regular_before = regular_times[regular_times < drying_time_s - 1.0e-8]

```

```

output_times = np.concatenate([regular_before, [drying_time_s, strict_time_s]])
output_fixed = np.vstack([
    extrapolated[np.searchsorted(integration_times, regular_before)],
    critical_fixed,
    strict_fixed,
])
output_surface = np.concatenate([
    extrapolated_surface[np.searchsorted(integration_times, regular_before)],
    [critical_surface, strict_surface],
])

endpoint_moisture = float(np.nanmax(critical_fixed))
strict_maximum = float(np.nanmax(strict_fixed))
if not np.isclose(endpoint_moisture, threshold, atol=1.0e-7, rtol=0.0):
    raise RuntimeError(f"Critical moisture does not equal threshold: {endpoint_moisture}")
if not strict_maximum < threshold:
    raise RuntimeError(f"Strict regular row is not below threshold: {strict_maximum}")
rounded_fixed = np.round(output_fixed, 4)
workbook_matrix = [
    [None if not np.isfinite(value) else float(value) for value in row]
    for row in rounded_fixed
]
payload = {
    "time_s": [int(round(t)) if np.isclose(t, round(t), rtol=0.0, atol=1.0e-8) else round(float(t),
6) for t in output_times],
    "radius_cm": fixed_radius_cm.tolist(),
    "moisture_concentration": workbook_matrix,
    "surface_moisture_concentration": np.round(output_surface, 4).tolist(),
    "drying_time_s": round(drying_time_s, 6),
    "drying_time_h": round(drying_time_s / 3600.0, 10),
    "strict_completion_time_s": int(round(strict_time_s)),
    "strict_completion_time_h": round(strict_time_s / 3600.0, 10),
    "threshold": threshold,
    "critical_maximum_moisture_unrounded": endpoint_moisture,
    "strict_completion_maximum_moisture_unrounded": strict_maximum,
    "radius_at_drying_time_cm": float(np.interp(drying_time_s, radius_history.time_s,
radius_history.radius_m) * 100.0),
    "grid_endpoint_s": endpoint_times | {"richardson": drying_time_s},
    "outside_radius_cells_are_blank": True,
    "moisture_balance": balance_by_grid,
    "radial_monotonicity": monotonicity_by_grid,
    "radius_timing_sensitivity_s": timing_sensitivity,
}
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(
    json.dumps(payload, ensure_ascii=False, separators=(",", ":"), allow_nan=False),
    encoding="utf-8",
)
write_diagnostics(payload, args.diagnostics_output, args.chamber, args.radius)
save_figures(output_times, fixed_radius_cm, output_fixed, output_surface, radius_history,
drying_time_s, args.figure_dir)
print(f"drying endpoint: {drying_time_s / 3600.0:.6f} h; strict row: {strict_time_s:.0f} s",
flush=True)

if __name__ == "__main__":
    main()

```

scripts/verify_problem4.py

"""Reproduce the Q4 review experiments; no external review values are inputs."""

from __future__ import annotations

```

import argparse
from dataclasses import replace
import hashlib
import json

```

```

from pathlib import Path
import platform

import numpy as np
import scipy

from drying_model.problem1 import Problem1Parameters, load_chamber_history_csv
from drying_model.problem3 import extend_chamber_history_to_plateau
from drying_model.problem4 import RadiusHistory, density_shrinkage_compatibility,
load_radius_history_csv, solve_problem4

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output", type=Path,
default=Path("reports/data/problem4_review_experiments.json"))
    parser.add_argument("--sensitivity-grid", type=int, default=320)
    args = parser.parse_args()
    chamber_path = Path("data/raw/attachment1.csv")
    radius_path = Path("data/raw/attachment2.csv")
    chamber = extend_chamber_history_to_plateau(load_chamber_history_csv(chamber_path), 144 * 3600.0)
    radius = load_radius_history_csv(radius_path)
    parameters = Problem1Parameters()
    results = {
        "schema_version": 1,
        "runtime": {"python": platform.python_version(), "numpy": np.__version__, "scipy":
scipy.__version__},
        "input_sha256": {str(p): hashlib.sha256(p.read_bytes()).hexdigest() for p in (chamber_path,
radius_path)},
        "threshold": 0.15,
        "default_solver": {"relative_tolerance": 2e-8, "max_step_s": 120.0},
        "sensitivity_radial_intervals": args.sensitivity_grid,
    }

    def save() -> None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(json.dumps(results, indent=2, ensure_ascii=False, allow_nan=False) +
"\n", encoding="utf-8")

    def endpoint(grid, radius_input=radius, params=parameters, rtol=2e-8, step=120.0):
        solution = solve_problem4(chamber, radius_input, grid, [0.0, 144 * 3600.0], parameters=params,
relative_tolerance=rtol, max_step_s=step,
maximum_moisture_threshold=0.15)
        return solution

    fixed = RadiusHistory(radius.time_s, np.full_like(radius.radius_m, parameters.radius_m))
    fixed_endpoints = {}
    results["fixed_radius_endpoint_s"] = fixed_endpoints
    for grid in (160, 320, 640, 1280):
        fixed_endpoints[str(grid)] = float(endpoint(grid, fixed).time_s[-1])
        print(f"fixed radius N={grid}: {fixed_endpoints[str(grid)] / 3600:.9f} h", flush=True)
        save()
    fixed_endpoints["richardson_640_1280"] = fixed_endpoints["1280"] + (fixed_endpoints["1280"] -
fixed_endpoints["640"]) / 3
    fixed_endpoints["observed_order_320_640_1280"] = float(np.log2((fixed_endpoints["320"] -
fixed_endpoints["640"]) / (fixed_endpoints["640"] - fixed_endpoints["1280"])))

    time_checks = []
    results["time_integration"] = time_checks
    for rtol, step in ((2e-8, 120.0), (2e-9, 60.0), (5e-10, 30.0)):
        solution = endpoint(args.sensitivity_grid, rtol=rtol, step=step)
        time_checks.append({"relative_tolerance": rtol, "max_step_s": step, "endpoint_s":
float(solution.time_s[-1])})
        print(f"time convergence: {time_checks[-1]}", flush=True)
        save()
    baseline = time_checks[0]["endpoint_s"]
    results["time_integration_endpoint_spread_s"] = max(row["endpoint_s"] for row in time_checks) -
min(row["endpoint_s"] for row in time_checks)

    pchip = RadiusHistory(radius.time_s, radius.radius_m, interpolation="pchip")

```

```

pchip_time = float(endpoint(args.sensitivity_grid, pchip).time_s[-1])
results["radius_interpolation"] = {"linear_endpoint_s": baseline, "pchip_endpoint_s": pchip_time,
                                   "pchip_minus_linear_s": pchip_time - baseline,
"observation_times_unchanged": True}
    save()
    print(f"PCHIP minus linear: {pchip_time - baseline:.6f} s", flush=True)

    for name, field in (("mass_transfer", "mass_transfer_coefficient_m_s"), ("heat_transfer",
"heat_transfer_coefficient_w_m2_k")):
        rows = []
        results[name + "_sensitivity"] = rows
        for multiplier in (0.8, 1.0, 1.2):
            value = getattr(parameters, field) * multiplier
            time = baseline if multiplier == 1.0 else float(endpoint(args.sensitivity_grid,
params=replace(parameters, **{field: value})).time_s[-1])
            rows.append({"multiplier": multiplier, "coefficient": value, "endpoint_s": time,
"change_from_baseline_s": time - baseline})
            print(f"{name} x{multiplier}: {time / 3600:.9f} h", flush=True)
            save()

        density_checks = {}
        results["density_shrinkage_compatibility"] = density_checks
        for grid in (640, 1280):
            event_solution = endpoint(grid)
            times = np.append(np.arange(0.0, 49 * 3600.0, 6 * 3600.0), event_solution.time_s[-1])
            full = solve_problem4(chamber, radius, grid, times, relative_tolerance=2e-8, max_step_s=120.0)
            density_checks[str(grid)] = density_shrinkage_compatibility(full, radius)
            density_checks[str(grid)]["center_moisture_at_48h"] = float(full.moisture_concentration[-2, 0])
            print(f"density compatibility N={grid}: final mass ratio
{density_checks[str(grid)]['final_mass_ratio']:.9f}", flush=True)
            save()
        save()

if __name__ == "__main__":
    main()

```

scripts/verify_problem4_independent.py

```

"""Reproduce a separately implemented cell-centred discretization of Problem 4."""

from __future__ import annotations

import argparse
import json
from pathlib import Path
import platform
import time

import numpy as np
import scipy

from drying_model.problem1 import load_chamber_history_csv
from drying_model.problem3 import extend_chamber_history_to_plateau
from drying_model.problem4 import load_radius_history_csv
from drying_model.problem4_independent import (
    sample_independent_moisture,
    solve_independent_problem4,
)

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--cells", nargs="+", type=int, default=[100, 200, 400, 800, 1600, 3200])
    parser.add_argument("--output", type=Path,
default=Path("reports/problem4_independent_validation.json"))
    args = parser.parse_args()
    raw_chamber = load_chamber_history_csv("data/raw/attachment1.csv")

```



```

extrapolations = {}
for coarse in args.cells:
    fine = 2 * coarse
    if str(fine) not in grids:
        continue
    coarse_value, fine_value = grids[str(coarse)], grids[str(fine)]
    extrap_time = fine_value["drying_time_s"] + (
        fine_value["drying_time_s"] - coarse_value["drying_time_s"]
    ) / 3.0
    extrap_table = np.asarray(fine_value["table_unrounded"]) + (
        np.asarray(fine_value["table_unrounded"]) - np.asarray(coarse_value["table_unrounded"])
    ) / 3.0
    rounded = np.round(extrap_table, 4)
    mismatches = np.argwhere(rounded != reference_table)
    extrapolations[f"{coarse}/{fine}"] = {
        "assumed_order": 2,
        "drying_time_s": extrap_time,
        "drying_time_h": extrap_time / 3600.0,
        "delta_from_baseline_s": extrap_time - 51.091955 * 3600.0,
        "delta_from_unverified_docx_reference_s": extrap_time - 51.091999 * 3600.0,
        "table_unrounded": extrap_table.tolist(),
        "table_rounded_4dp": rounded.tolist(),
        "maximum_absolute_difference_from_rounded_table_6": float(np.max(np.abs(extrap_table -
reference_table))),
        "rounded_table_mismatch_count": int(len(mismatches)),
        "rounded_table_mismatches": [
            {"time_h": float(table_hours[row]), "column": output["table_columns"][col],
             "baseline": float(reference_table[row, col]), "independent": float(rounded[row, col])}
            for row, col in mismatches
        ],
    }
output["second_order_richardson"] = extrapolations
orders = {}
for coarse in args.cells:
    if str(2 * coarse) in grids and str(4 * coarse) in grids:
        a, b, c = [grids[str(k)]["drying_time_s"] for k in (coarse, 2 * coarse, 4 * coarse)]
        orders[f"{coarse}/{2 * coarse}/{4 * coarse}"] = float(np.log2(abs((a - b) / (b - c))))
output["observed_endpoint_orders"] = orders
output["interpretation"] = (
    "The endpoint differences decrease with refinement, but observed orders remain below two. "
    "The second-order Richardson values are diagnostic extrapolations, not certified error bounds.
"
    "Agreement with a production-solver extrapolation does not validate the shared physical
assumptions. "
    "Four-decimal table agreement is assessed against the rounded published table; its raw
differences "
    "therefore include reference rounding and are not discretization error estimates."
)
args.output.parent.mkdir(parents=True, exist_ok=True)
args.output.write_text(json.dumps(output, indent=2, ensure_ascii=False, allow_nan=False) + "\n",
encoding="utf-8")
print(json.dumps({"extrapolated_endpoints_h": {key: row["drying_time_h"] for key, row in
extrapolations.items()}},
"observed_orders": orders}, ensure_ascii=False), flush=True)

if __name__ == "__main__":
    main()

```

src/drying_model/__init__.py

"""Numerical models for Problem A."""

src/drying_model/problem1.py

"""Problem 1 material laws and chamber boundary interpolation."""

```

from __future__ import annotations

import csv
from dataclasses import dataclass
from pathlib import Path

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import diags

@dataclass(frozen=True)
class ChamberHistory:
    """Observed chamber boundary conditions."""

    time_s: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        temperature = np.asarray(self.temperature_c, dtype=float)
        moisture = np.asarray(self.moisture_concentration, dtype=float)
        if time.ndim != 1 or temperature.shape != time.shape or moisture.shape != time.shape:
            raise ValueError("Chamber history columns must be one-dimensional and equal-sized")
        if any(np.any(~np.isfinite(values)) for values in (time, temperature, moisture)):
            raise ValueError("Chamber history must contain only finite values")
        if time.size < 2 or np.any(np.diff(time) <= 0.0):
            raise ValueError("Chamber times must be strictly increasing")
        if np.any(moisture < 0.0):
            raise ValueError("Chamber moisture concentration must be non-negative")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "temperature_c", temperature)
        object.__setattr__(self, "moisture_concentration", moisture)

@dataclass(frozen=True)
class Problem1Parameters:
    radius_m: float = 0.02
    initial_temperature_c: float = 28.0
    initial_moisture_concentration: float = 2.55
    density_kg_m3: float = 820.0
    heat_capacity_j_kg_k: float = 2600.0
    thermal_conductivity_w_m_k: float = 0.36
    heat_transfer_coefficient_w_m2_k: float = 25.0
    mass_transfer_coefficient_m_s: float = 8.0e-7

@dataclass(frozen=True)
class Problem1Solution:
    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture_concentration: NDArray[np.float64]

def load_chamber_history_csv(path: str | Path) -> ChamberHistory:
    """Load the three columns supplied in Attachment 1 from a UTF-8 CSV."""
    rows: list[tuple[float, float, float]] = []
    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        required = {"时间", "温度", "水分浓度"}
        if reader.fieldnames is None or not required.issubset(reader.fieldnames):
            raise ValueError("Attachment 1 must contain 时间, 温度, 水分浓度 columns")
        for line_number, row in enumerate(reader, start=2):
            try:
                rows.append((float(row["时间"]), float(row["温度"]), float(row["水分浓度"])))
            except (TypeError, ValueError) as exc:
                raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc

```

```

    if not rows:
        raise ValueError("Attachment 1 contains no data rows")
    data = np.asarray(rows, dtype=float)
    if not np.all(np.isfinite(data)):
        raise ValueError("Attachment 1 contains non-finite values")
    return ChamberHistory(data[:, 0], data[:, 1], data[:, 2])

def diffusivity_q1(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return  $D=7e-9 \exp(-0.89/C)$  in square metres per second."""
    values = np.asarray(concentration, dtype=float)
    if np.any(values <= 0.0):
        raise ValueError("Moisture concentration must be positive")
    return  $7.0e-9 * \exp(-0.89 / \text{values})$ 

def interpolate_history(
    query_time_s: float,
    time_s: ArrayLike,
    values: ArrayLike,
) -> float:
    """Linearly interpolate an observed chamber history."""
    return float(np.interp(query_time_s, time_s, values))

def nodal_control_volumes(node_count: int, radius_m: float) -> NDArray[np.float64]:
    """Return annular control volumes per unit cylinder length for nodal radii."""
    if node_count < 2:
        raise ValueError("At least two radial nodes are required")
    if radius_m <= 0.0:
        raise ValueError("Cylinder radius must be positive")
    spacing = radius_m / (node_count - 1)
    radii = np.linspace(0.0, radius_m, node_count)
    inner = np.maximum(radii - 0.5 * spacing, 0.0)
    outer = np.minimum(radii + 0.5 * spacing, radius_m)
    return np.pi * (outer**2 - inner**2)

def radial_flux_divergence(
    state: ArrayLike,
    coefficient: ArrayLike,
    radius_m: float,
    exchange_coefficient: float,
    ambient_value: float,
) -> NDArray[np.float64]:
    """Conservative cylindrical divergence with a convective outer boundary.

    ``coefficient`` is k for heat or D for moisture. At the outer surface the
    imposed flux is ``-exchange_coefficient * (surface - ambient)``.
    """
    values = np.asarray(state, dtype=float)
    conductance = np.asarray(coefficient, dtype=float)
    if values.ndim != 1 or conductance.shape != values.shape:
        raise ValueError("State and coefficient must be one-dimensional and equal-sized")
    if np.any(conductance < 0.0) or exchange_coefficient < 0.0:
        raise ValueError("Transport coefficients must be non-negative")

    node_count = values.size
    spacing = radius_m / (node_count - 1)
    face_radii = (np.arange(node_count - 1, dtype=float) + 0.5) * spacing
    denominator = conductance[:-1] + conductance[1:]
    face_coefficient = np.divide(
        2.0 * conductance[:-1] * conductance[1:],
        denominator,
        out=np.zeros_like(denominator),
        where=denominator > 0.0,
    )
    face_gradient = np.diff(values) / spacing
    face_rate = 2.0 * np.pi * face_radii * face_coefficient * face_gradient

```

```

integrated_rate = np.zeros_like(values)
integrated_rate[:-1] += face_rate
integrated_rate[1:] -= face_rate
surface_flux = -exchange_coefficient * (values[-1] - ambient_value)
integrated_rate[-1] += 2.0 * np.pi * radius_m * surface_flux

return integrated_rate / nodal_control_volumes(node_count, radius_m)

def solve_problem1(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve the fixed-radius preheating model at requested output times."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    if output_times[0] < history.time_s[0] or output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times must lie within the chamber history")

    node_count = radial_intervals + 1
    radii = np.linspace(0.0, parameters.radius_m, node_count)
    sparsity = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1],
        shape=(node_count, node_count),
        format="csr",
    )

    thermal_coefficient = np.full(node_count, parameters.thermal_conductivity_w_m_k)

    def temperature_rhs(time_s: float, temperature_c: NDArray[np.float64]) -> NDArray[np.float64]:
        ambient = interpolate_history(time_s, history.time_s, history.temperature_c)
        divergence = radial_flux_divergence(
            temperature_c,
            thermal_coefficient,
            parameters.radius_m,
            parameters.heat_transfer_coefficient_w_m2_k,
            ambient,
        )
        return divergence / (parameters.density_kg_m3 * parameters.heat_capacity_j_kg_k)

    def log_moisture_rhs(time_s: float, log_concentration: NDArray[np.float64]) -> NDArray[np.float64]:
        concentration = np.exp(log_concentration)
        ambient = interpolate_history(time_s, history.time_s, history.moisture_concentration)
        divergence = radial_flux_divergence(
            concentration,
            diffusivity_q1(concentration),
            parameters.radius_m,
            parameters.mass_transfer_coefficient_m_s,
            ambient,
        )
        return divergence / concentration

    time_span = (float(history.time_s[0]), float(output_times[-1]))
    temperature_result = solve_ivp(
        temperature_rhs,
        time_span,
        np.full(node_count, parameters.initial_temperature_c),
        method="BDF",
        t_eval=output_times,

```

```

        rtol=relative_tolerance,
        atol=1.0e-9,
        max_step=max_step_s,
        jac_sparsity=sparsity,
    )
moisture_result = solve_ivp(
    log_moisture_rhs,
    time_span,
    np.full(node_count, np.log(parameters.initial_moisture_concentration)),
    method="BDF",
    t_eval=output_times,
    rtol=relative_tolerance,
    atol=1.0e-10,
    max_step=max_step_s,
    jac_sparsity=sparsity,
)
if not temperature_result.success:
    raise RuntimeError(f"Temperature solver failed: {temperature_result.message}")
if not moisture_result.success:
    raise RuntimeError(f"Moisture solver failed: {moisture_result.message}")

return Problem1Solution(
    time_s=output_times,
    radius_m=radii,
    temperature_c=temperature_result.y.T,
    moisture_concentration=np.exp(moisture_result.y.T),
)

def sample_solution(solution: Problem1Solution, radius_cm: ArrayLike) -> Problem1Solution:
    """Interpolate a solution to requested physical radii in centimetres."""
    requested_radius_m = np.asarray(radius_cm, dtype=float) * 0.01
    if requested_radius_m.ndim != 1 or requested_radius_m.size == 0:
        raise ValueError("Requested radii must be a non-empty one-dimensional array")
    if np.any(np.diff(requested_radius_m) <= 0.0):
        raise ValueError("Requested radii must be strictly increasing")
    if requested_radius_m[0] < 0.0 or requested_radius_m[-1] > solution.radius_m[-1]:
        raise ValueError("Requested radii lie outside the cylinder")

    temperature = np.vstack(
        [np.interp(requested_radius_m, solution.radius_m, row) for row in solution.temperature_c]
    )
    moisture = np.vstack(
        [
            np.interp(requested_radius_m, solution.radius_m, row)
            for row in solution.moisture_concentration
        ]
    )
    return Problem1Solution(
        time_s=solution.time_s.copy(),
        radius_m=requested_radius_m,
        temperature_c=temperature,
        moisture_concentration=moisture,
    )

def richardson_extrapolate_solutions(
    coarse: Problem1Solution,
    fine: Problem1Solution,
    *,
    order: int = 2,
) -> Problem1Solution:
    """Extrapolate two solutions whose spatial mesh widths differ by a factor of two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if (
        coarse.temperature_c.shape != fine.temperature_c.shape
        or coarse.moisture_concentration.shape
        != fine.moisture_concentration.shape
    ):

```

```

        raise ValueError("Solutions must have matching field shapes")
    if not np.array_equal(coarse.time_s, fine.time_s) or not np.array_equal(
        coarse.radius_m,
        fine.radius_m,
    ):
        raise ValueError("Solutions must use identical output times and radii")
    denominator = 2.0**order - 1.0
    return Problem1Solution(
        time_s=fine.time_s.copy(),
        radius_m=fine.radius_m.copy(),
        temperature_c=fine.temperature_c
        + (fine.temperature_c - coarse.temperature_c) / denominator,
        moisture_concentration=fine.moisture_concentration
        + (fine.moisture_concentration - coarse.moisture_concentration) / denominator,
    )

def result_payload(solution: Problem1Solution) -> dict[str, list]:
    """Convert a sampled solution to the numeric schema used by result1.xlsx."""
    if np.any(~np.isfinite(solution.time_s)) or not np.allclose(solution.time_s,
np.round(solution.time_s), atol=1.0e-10, rtol=0.0):
        raise ValueError("Workbook output times must be whole seconds")
    return {
        "time_s": np.round(solution.time_s).astype(int).tolist(),
        "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
        "temperature_c": np.round(solution.temperature_c, 4).tolist(),
        "moisture_concentration": np.round(solution.moisture_concentration, 4).tolist(),
    }

```

src/drying_model/problem2.py

```

"""Coupled variable-property model for Problem 2."""

from __future__ import annotations

from collections.abc import Callable

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import bmat, diags

from drying_model.problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    interpolate_history,
    radial_flux_divergence,
    sample_solution,
)

ScalarLaw = Callable[[NDArray[np.float64]], NDArray[np.float64]]
CoupledLaw = Callable[
    [NDArray[np.float64], NDArray[np.float64]],
    NDArray[np.float64],
]

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:
    values = np.asarray(concentration, dtype=float)
    if np.any(~np.isfinite(values)) or np.any(values < 0.0):
        raise ValueError("Moisture concentration must be finite and non-negative")
    return values

def density_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 density in kilograms per cubic metre."""
    values = _concentration_array(concentration)

```

```

return 650.0 + 128.0 * values

def heat_capacity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 specific heat capacity in joules per kilogram-kelvin."""
    values = _concentration_array(concentration)
    return 1450.0 + 2736.0 * values / (values + 1.0)

def thermal_conductivity_q2(concentration: ArrayLike) -> NDArray[np.float64]:
    """Return Appendix 3 thermal conductivity in watts per metre-kelvin."""
    values = _concentration_array(concentration)
    return 0.21 + 0.38 * values / (values + 1.0)

def diffusivity_q2(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:
    """Return Appendix 3 moisture diffusivity using absolute temperature."""
    moisture = _concentration_array(concentration)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
        raise ValueError("Temperature must be finite and above absolute zero")
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must be positive for diffusivity")
    moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
    return 2.4e-3 * np.exp(-0.45 / moisture) * np.exp(-3850.0 / temperature_k)

def solve_variable_property_fixed_cylinder(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    density_law: ScalarLaw,
    heat_capacity_law: ScalarLaw,
    conductivity_law: ScalarLaw,
    diffusivity_law: CoupledLaw,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve coupled heat and moisture transport on a fixed cylindrical radius."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if (
        not np.isfinite(start_time)
        or start_time < history.time_s[0]
        or start_time > output_times[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if maximum_moisture_threshold is not None and (
        not np.isfinite(maximum_moisture_threshold)
        or maximum_moisture_threshold <= 0.0
        or maximum_moisture_threshold >= parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

```

```

node_count = radial_intervals + 1
radii = np.linspace(0.0, parameters.radius_m, node_count)
tridiagonal = diags(
    [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
    offsets=[-1, 0, 1],
    shape=(node_count, node_count),
    format="csr",
)
sparsity = bmat(
    [[tridiagonal, tridiagonal], [tridiagonal, tridiagonal]],
    format="csr",
)

def coupled_rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
    temperature_c = state[:node_count]
    concentration = np.exp(state[node_count:])
    density = np.asarray(density_law(concentration), dtype=float)
    heat_capacity = np.asarray(heat_capacity_law(concentration), dtype=float)
    conductivity = np.asarray(conductivity_law(concentration), dtype=float)
    diffusivity = np.asarray(
        diffusivity_law(concentration, temperature_c),
        dtype=float,
    )
    expected_shape = concentration.shape
    for name, values in (
        ("density", density),
        ("heat capacity", heat_capacity),
        ("thermal conductivity", conductivity),
        ("diffusivity", diffusivity),
    ):
        if values.shape != expected_shape or np.any(~np.isfinite(values)) or np.any(values <= 0.0):
            raise ValueError(f"{name} law must return finite positive nodal values")

    ambient_temperature = interpolate_history(
        time_s,
        history.time_s,
        history.temperature_c,
    )
    temperature_rate = radial_flux_divergence(
        temperature_c,
        conductivity,
        parameters.radius_m,
        parameters.heat_transfer_coefficient_w_m2_k,
        ambient_temperature,
    ) / (density * heat_capacity)

    ambient_moisture = interpolate_history(
        time_s,
        history.time_s,
        history.moisture_concentration,
    )
    concentration_rate = radial_flux_divergence(
        concentration,
        diffusivity,
        parameters.radius_m,
        parameters.mass_transfer_coefficient_m_s,
        ambient_moisture,
    )
    log_concentration_rate = concentration_rate / concentration
    return np.concatenate([temperature_rate, log_concentration_rate])

supplied_initial_fields = (
    initial_temperature_c is not None,
    initial_moisture_concentration is not None,
)
if any(supplied_initial_fields) and not all(supplied_initial_fields):
    raise ValueError("Both initial fields must be supplied together")
if all(supplied_initial_fields):
    temperature_initial = np.asarray(initial_temperature_c, dtype=float)
    moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)

```

```

    if (
        temperature_initial.shape != (node_count,)
        or moisture_initial.shape != (node_count,)
        or np.any(~np.isfinite(temperature_initial))
        or np.any(~np.isfinite(moisture_initial))
        or np.any(moisture_initial <= 0.0)
    ):
        raise ValueError("Initial fields must be finite positive nodal arrays")
    else:
        temperature_initial = np.full(node_count, parameters.initial_temperature_c)
        moisture_initial = np.full(
            node_count,
            parameters.initial_moisture_concentration,
        )
    initial_state = np.concatenate(
        [temperature_initial, np.log(moisture_initial)]
    )
    absolute_tolerance = np.concatenate(
        [np.full(node_count, 1.0e-9), np.full(node_count, 1.0e-10)]
    )

    threshold_event = None
    if maximum_moisture_threshold is not None:

        def threshold_event(time_s: float, state: NDArray[np.float64]) -> float:
            del time_s
            return float(
                np.max(np.exp(state[node_count:])) - maximum_moisture_threshold
            )

        threshold_event.terminal = True
        threshold_event.direction = -1.0

    result = solve_ivp(
        coupled_rhs,
        (start_time, float(output_times[-1])),
        initial_state,
        method="BDF",
        t_eval=output_times,
        rtol=relative_tolerance,
        atol=absolute_tolerance,
        max_step=max_step_s,
        jac_sparsity=sparsity,
        events=threshold_event,
    )
    if not result.success:
        raise RuntimeError(f"Coupled solver failed: {result.message}")
    result_time = result.t
    result_state = result.y
    if maximum_moisture_threshold is not None:
        if not result.t_events or result.t_events[0].size == 0:
            raise RuntimeError("Maximum moisture threshold was not reached")
        event_time = float(result.t_events[0][0])
        event_state = result.y_events[0][0]
        if result_time.size and np.isclose(
            result_time[-1],
            event_time,
            rtol=0.0,
            atol=1.0e-9,
        ):
            result_time = result_time.copy()
            result_state = result_state.copy()
            result_time[-1] = event_time
            result_state[:, -1] = event_state
        else:
            result_time = np.append(result_time, event_time)
            result_state = np.column_stack([result_state, event_state])
    return Problem1Solution(
        time_s=result_time,
        radius_m=rad_ii,

```

```

        temperature_c=result_state[:node_count].T,
        moisture_concentration=np.exp(result_state[node_count:].T),
    )

def solve_problem2(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem1Solution:
    """Solve Problem 2 with the four Appendix 3 material-property laws."""
    return solve_variable_property_fixed_cylinder(
        history,
        radial_intervals,
        output_times_s,
        density_law=density_q2,
        heat_capacity_law=heat_capacity_q2,
        conductivity_law=thermal_conductivity_q2,
        diffusivity_law=diffusivity_q2,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=maximum_moisture_threshold,
        initial_time_s=initial_time_s,
        initial_temperature_c=initial_temperature_c,
        initial_moisture_concentration=initial_moisture_concentration,
    )

def solve_problem2_sampled_in_chunks(
    history: ChamberHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    sample_radius_cm: ArrayLike,
    *,
    chunk_duration_s: float = 600.0,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 5.0,
) -> Problem1Solution:
    """Solve Problem 2 in bounded-memory chunks and retain sampled radii only."""
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0:
        raise ValueError("Output times must be a non-empty one-dimensional array")
    if np.any(~np.isfinite(output_times)) or np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be finite and strictly increasing")
    if (
        output_times[0] < history.time_s[0]
        or output_times[-1] > history.time_s[-1]
    ):
        raise ValueError("Output times must lie within the chamber history")
    if not np.isfinite(chunk_duration_s) or chunk_duration_s <= 0.0:
        raise ValueError("Chunk duration must be finite and positive")

    requested_radius_cm = np.asarray(sample_radius_cm, dtype=float)
    node_count = radial_intervals + 1
    nodal_radius = np.linspace(0.0, parameters.radius_m, node_count)
    initial_solution = Problem1Solution(
        time_s=np.array([history.time_s[0]]),
        radius_m=nodal_radius,
        temperature_c=np.full((1, node_count), parameters.initial_temperature_c),
        moisture_concentration=np.full(

```

```

        (1, node_count),
        parameters.initial_moisture_concentration,
    ),
)
sampled_initial = sample_solution(initial_solution, requested_radius_cm)
sampled_temperature = np.empty(
    (output_times.size, sampled_initial.radius_m.size),
    dtype=float,
)
sampled_moisture = np.empty_like(sampled_temperature)

next_output_index = 0
current_time = float(history.time_s[0])
current_temperature: NDArray[np.float64] | None = None
current_moisture: NDArray[np.float64] | None = None
if np.isclose(output_times[0], current_time, rtol=0.0, atol=1.0e-12):
    sampled_temperature[0] = sampled_initial.temperature_c[0]
    sampled_moisture[0] = sampled_initial.moisture_concentration[0]
    next_output_index = 1

final_time = float(output_times[-1])
while current_time < final_time:
    chunk_end = min(current_time + chunk_duration_s, final_time)
    chunk_stop = int(np.searchsorted(output_times, chunk_end, side="right"))
    requested_times = output_times[next_output_index:chunk_stop]
    if requested_times.size and requested_times[0] <= current_time:
        raise RuntimeError("Chunked output indexing did not advance")
    append_chunk_end = (
        requested_times.size == 0
        or not np.isclose(requested_times[-1], chunk_end, rtol=0.0, atol=1.0e-12)
    )
    integration_times = (
        np.append(requested_times, chunk_end)
        if append_chunk_end
        else requested_times
    )
    chunk_solution = solve_problem2(
        history,
        radial_intervals=radial_intervals,
        output_times_s=integration_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        initial_time_s=None if current_temperature is None else current_time,
        initial_temperature_c=current_temperature,
        initial_moisture_concentration=current_moisture,
    )
    sampled_chunk = sample_solution(chunk_solution, requested_radius_cm)
    retained_count = requested_times.size
    if retained_count:
        sampled_temperature[next_output_index:chunk_stop] = (
            sampled_chunk.temperature_c[:retained_count]
        )
        sampled_moisture[next_output_index:chunk_stop] = (
            sampled_chunk.moisture_concentration[:retained_count]
        )
    current_temperature = chunk_solution.temperature_c[-1].copy()
    current_moisture = chunk_solution.moisture_concentration[-1].copy()
    current_time = chunk_end
    next_output_index = chunk_stop

if next_output_index != output_times.size:
    raise RuntimeError("Chunked solver did not populate every requested output time")
return Problem1Solution(
    time_s=output_times.copy(),
    radius_m=sampled_initial.radius_m,
    temperature_c=sampled_temperature,
    moisture_concentration=sampled_moisture,
)

```

src/drying_model/problem3.py

```
"""Fixed-radius drying endpoint model for Problem 3."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from scipy.integrate import cumulative_simpson

from drying_model.problem1 import (
    ChamberHistory,
    Problem1Parameters,
    Problem1Solution,
    nodal_control_volumes,
)
from drying_model.problem2 import solve_problem2

@dataclass(frozen=True)
class Problem3Result:
    """Problem 3 solution together with its unrounded drying endpoint."""

    solution: Problem1Solution
    drying_time_s: float
    strict_completion_time_s: float | None
    maximum_moisture: float
    maximum_radius_m: float
    plateau_temperature_c: float
    plateau_moisture_concentration: float
    threshold: float
    output_interval_s: float

def extend_chamber_history_to_plateau(
    history: ChamberHistory,
    end_time_s: float,
    *,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,
) -> ChamberHistory:
    """Extend Attachment 1 with a short transition to a constant tail plateau."""
    if not np.isfinite(end_time_s) or end_time_s <= history.time_s[-1]:
        raise ValueError("Extension end time must be later than the observed history")
    if not np.isfinite(averaging_window_s) or averaging_window_s <= 0.0:
        raise ValueError("Averaging window must be positive")
    if not np.isfinite(transition_s) or transition_s <= 0.0:
        raise ValueError("Plateau transition must be positive")

    tail = history.time_s >= history.time_s[-1] - averaging_window_s
    if np.count_nonzero(tail) < 2:
        raise ValueError("Averaging window must contain at least two observations")
    temperature = (
        float(np.mean(history.temperature_c[tail]))
        if plateau_temperature_c is None
        else float(plateau_temperature_c)
    )
    moisture = (
        float(np.mean(history.moisture_concentration[tail]))
        if plateau_moisture_concentration is None
        else float(plateau_moisture_concentration)
    )
    if not np.isfinite(temperature):
        raise ValueError("Plateau temperature must be finite")
    if not np.isfinite(moisture) or moisture < 0.0:
        raise ValueError("Plateau moisture concentration must be finite and non-negative")
```

```

transition_time = min(history.time_s[-1] + transition_s, end_time_s)
appended_times = [transition_time]
if end_time_s > transition_time:
    appended_times.append(end_time_s)
return ChamberHistory(
    time_s=np.concatenate([history.time_s, np.asarray(appended_times)]),
    temperature_c=np.concatenate(
        [history.temperature_c, np.full(len(appended_times), temperature)]
    ),
    moisture_concentration=np.concatenate(
        [history.moisture_concentration, np.full(len(appended_times), moisture)]
    ),
)

def solve_problem3(
    history: ChamberHistory,
    radial_intervals: int,
    horizon_s: float,
    *,
    output_interval_s: float = 60.0,
    threshold: float = 0.15,
    averaging_window_s: float = 3600.0,
    transition_s: float = 60.0,
    plateau_temperature_c: float | None = None,
    plateau_moisture_concentration: float | None = None,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 1.0e-8,
    max_step_s: float = 60.0,
) -> Problem3Result:
    """Continue the Problem 2 model until the full field reaches the threshold."""
    if not np.isfinite(output_interval_s) or output_interval_s <= 0.0:
        raise ValueError("Output interval must be positive")
    extended = extend_chamber_history_to_plateau(
        history,
        horizon_s,
        averaging_window_s=averaging_window_s,
        transition_s=transition_s,
        plateau_temperature_c=plateau_temperature_c,
        plateau_moisture_concentration=plateau_moisture_concentration,
    )
    regular_times = np.arange(
        output_interval_s,
        horizon_s + 0.5 * output_interval_s,
        output_interval_s,
    )
    regular_times = regular_times[regular_times <= horizon_s]
    if regular_times.size == 0 or regular_times[-1] < horizon_s:
        output_times = np.append(regular_times, horizon_s)
    else:
        output_times = regular_times
    solution = solve_problem2(
        extended,
        radial_intervals,
        output_times,
        parameters=parameters,
        relative_tolerance=relative_tolerance,
        max_step_s=max_step_s,
        maximum_moisture_threshold=threshold,
    )
    terminal_moisture = solution.moisture_concentration[-1]
    maximum_index = int(np.argmax(terminal_moisture))
    return Problem3Result(
        solution=solution,
        drying_time_s=float(solution.time_s[-1]),
        strict_completion_time_s=None,
        maximum_moisture=float(terminal_moisture[maximum_index]),
        maximum_radius_m=float(solution.radius_m[maximum_index]),
        plateau_temperature_c=float(extended.temperature_c[-1]),
    )

```

```

        plateau_moisture_concentration=float(extended.moisture_concentration[-1]),
        threshold=float(threshold),
        output_interval_s=float(output_interval_s),
    )

def richardson_extrapolate_drying_time(
    coarse_time_s: float,
    fine_time_s: float,
    *,
    order: int = 2,
) -> float:
    """Extrapolate drying times from meshes whose spacing ratio is two."""
    if order <= 0:
        raise ValueError("Richardson order must be positive")
    if not np.isfinite(coarse_time_s) or not np.isfinite(fine_time_s):
        raise ValueError("Drying times must be finite")
    return float(fine_time_s + (fine_time_s - coarse_time_s) / (2.0*order - 1.0))

def validate_radially_nonincreasing_moisture(
    solution: Problem1Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require every stored profile to be wettest at the centre and nonincreasing."""
    if not np.isfinite(tolerance) or tolerance < 0.0:
        raise ValueError("Radial monotonicity tolerance must be finite and non-negative")
    if solution.radius_m.size < 2:
        raise ValueError("Radial monotonicity requires at least two positions")
    maximum_outward_increase = float(
        np.max(np.diff(solution.moisture_concentration, axis=1))
    )
    if maximum_outward_increase > tolerance:
        raise ValueError(
            "Moisture profile contains an outward increase; centre-wettest claim failed"
        )
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem1Solution,
    history: ChamberHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Compare total moisture change with integrated convective surface flux."""
    if not np.isclose(solution.time_s[0], history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time solution row")
    if solution.time_s[-1] > history.time_s[-1]:
        raise ValueError("Moisture balance times must lie within the chamber history")
    if not np.isclose(solution.radius_m[-1], parameters.radius_m, atol=1.0e-12, rtol=0.0):
        raise ValueError("Solution radius and model radius must match")

    volumes = nodal_control_volumes(solution.radius_m.size, parameters.radius_m)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient_moisture = np.interp(
        solution.time_s,
        history.time_s,
        history.moisture_concentration,
    )
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / parameters.radius_m
        * (solution.moisture_concentration[:, -1] - ambient_moisture)
    )
    cumulative_surface_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,

```

```

        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - cumulative_surface_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(
            mean_moisture[-1] - mean_moisture[0]
        ),
        "integrated_surface_change": float(cumulative_surface_change[-1]),
    }

def truncate_solution_at_threshold(
    solution: Problem1Solution,
    *,
    threshold: float,
    output_interval_s: float,
) -> Problem1Solution:
    """Insert the threshold crossing and retain the first later strict regular row."""
    maximum = np.max(solution.moisture_concentration, axis=1)
    crossing_indices = np.flatnonzero(maximum <= threshold)
    if crossing_indices.size == 0 or crossing_indices[0] == 0:
        raise ValueError("Solution must bracket the first moisture threshold crossing")
    upper = int(crossing_indices[0])
    lower = upper - 1
    # Interpolate each nodal field, then take the last nodal crossing. Merely
    # interpolating the two maxima is wrong when the wettest node changes.
    above = solution.moisture_concentration[lower] > threshold
    nodal_drop = solution.moisture_concentration[lower, above] - solution.moisture_concentration[upper,
above]
    fraction = float(np.max((solution.moisture_concentration[lower, above] - threshold) / nodal_drop))
    crossing_time = solution.time_s[lower] + fraction * (
        solution.time_s[upper] - solution.time_s[lower]
    )
    crossing_temperature = solution.temperature_c[lower] + fraction * (
        solution.temperature_c[upper] - solution.temperature_c[lower]
    )
    crossing_moisture = solution.moisture_concentration[lower] + fraction * (
        solution.moisture_concentration[upper]
        - solution.moisture_concentration[lower]
    )
    on_regular_interval = np.isclose(
        np.mod(solution.time_s, output_interval_s),
        0.0,
        atol=1.0e-8,
    )
    regular_before = on_regular_interval & (
        solution.time_s < crossing_time - 1.0e-8
    )
    strict_candidates = np.flatnonzero(
        on_regular_interval
        & (solution.time_s > crossing_time + 1.0e-8)
        & (maximum < threshold)
    )
    if strict_candidates.size == 0:
        raise ValueError("Solution must include a later regular row strictly below threshold")
    strict_index = int(strict_candidates[0])
    return Problem1Solution(
        time_s=np.concatenate(
            [solution.time_s[regular_before], [crossing_time, solution.time_s[strict_index]]]
        ),
        radius_m=solution.radius_m.copy(),
        temperature_c=np.vstack(
            [
                solution.temperature_c[regular_before],

```

```

        crossing_temperature,
        solution.temperature_c[strict_index],
    ]
),
moisture_concentration=np.vstack(
    [
        solution.moisture_concentration[regular_before],
        crossing_moisture,
        solution.moisture_concentration[strict_index],
    ]
),
)

def problem3_payload(result: Problem3Result) -> dict[str, object]:
    """Convert a sampled Problem 3 result to the result3.xlsx data schema."""
    solution = result.solution
    if solution.time_s.size == 0 or np.any(np.diff(solution.time_s) <= 0.0):
        raise ValueError("Problem 3 output times must be non-empty and increasing")
    if result.strict_completion_time_s is None:
        raise ValueError("Strict completion time requires a verified post-threshold row")
    if not np.isclose(
        solution.time_s[-1],
        result.strict_completion_time_s,
        rtol=0.0,
        atol=1.0e-8,
    ):
        raise ValueError("The final solution row must be the strict completion time")
    critical_indices = np.flatnonzero(
        np.isclose(
            solution.time_s,
            result.drying_time_s,
            rtol=0.0,
            atol=1.0e-8,
        )
    )
    if critical_indices.size != 1:
        raise ValueError("The solution must contain exactly one critical threshold row")
    critical_index = int(critical_indices[0])
    regular_rows = np.ones(solution.time_s.size, dtype=bool)
    regular_rows[critical_index] = False
    if not np.allclose(
        np.mod(solution.time_s[regular_rows], result.output_interval_s),
        0.0,
        atol=1.0e-8,
    ):
        raise ValueError("All non-critical output rows must lie on the regular interval")
    critical_maximum = float(
        np.max(solution.moisture_concentration[critical_index])
    )
    strict_maximum = float(np.max(solution.moisture_concentration[-1]))
    if not np.isclose(critical_maximum, result.threshold, atol=1.0e-8, rtol=0.0):
        raise ValueError("Critical maximum moisture must equal the unrounded threshold")
    if not strict_maximum < result.threshold:
        raise ValueError("Strict completion maximum moisture must be below threshold")

    time_values: list[int | float] = []
    for value in solution.time_s:
        rounded_integer = round(float(value))
        if np.isclose(value, rounded_integer, rtol=0.0, atol=1.0e-9):
            time_values.append(int(rounded_integer))
        else:
            time_values.append(round(float(value), 6))
    return {
        "time_s": time_values,
        "radius_cm": np.round(solution.radius_m * 100.0, 10).tolist(),
        "moisture_concentration": np.round(
            solution.moisture_concentration,
            4,
        ).tolist(),
    }

```

```

"drying_time_s": round(result.drying_time_s, 6),
"drying_time_h": round(result.drying_time_s / 3600.0, 10),
"strict_completion_time_s": round(result.strict_completion_time_s, 6),
"strict_completion_time_h": round(
    result.strict_completion_time_s / 3600.0,
    10,
),
"threshold": result.threshold,
"critical_maximum_moisture_unrounded": critical_maximum,
"strict_completion_maximum_moisture_unrounded": strict_maximum,
"maximum_radius_m": result.maximum_radius_m,
"plateau_temperature_c": result.plateau_temperature_c,
"plateau_moisture_concentration": result.plateau_moisture_concentration,
}

```

src/drying_model/problem4.py

```

"""Moving-radius drying model for Problem 4.

```

```

The measured radius history is used as a prescribed moving boundary. The
radial coordinate is mapped to  $\xi=r/R(t)$ ; for dry-basis concentration the
solid motion then cancels the grid velocity and leaves a diffusion equation on
the fixed interval  $0 \leq \xi \leq 1$  under spatially uniform dry-solid density.
Appendix 4 density is used as an effective thermal-storage law; compatibility
with a literal wet-bulk-density interpretation is audited separately.
"""

```

```

from __future__ import annotations

```

```

import csv
from dataclasses import dataclass, field
from pathlib import Path

```

```

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import cumulative_simpson, solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.sparse import bmat, diags

```

```

from drying_model.problem1 import (
    ChamberHistory,
    Problem1Parameters,
    interpolate_history,
    nodal_control_volumes,
    radial_flux_divergence,
)

```

```

@dataclass(frozen=True)

```

```

class RadiusHistory:

```

```

    """Measured radius history in seconds and centimetres converted to metres."""

```

```

    time_s: NDArray[np.float64]
    radius_m: NDArray[np.float64]
    interpolation: str = "linear"
    _pchip: PchipInterpolator | None = field(init=False, repr=False, compare=False)

```

```

    def __post_init__(self) -> None:
        time = np.asarray(self.time_s, dtype=float)
        radius = np.asarray(self.radius_m, dtype=float)
        if time.ndim != 1 or radius.shape != time.shape or time.size < 2:
            raise ValueError("Radius history columns must be one-dimensional and equal-sized")
        if np.any(~np.isfinite(time)) or np.any(~np.isfinite(radius)):
            raise ValueError("Radius history must be finite")
        if np.any(np.diff(time) <= 0.0):
            raise ValueError("Radius times must be strictly increasing")
        if np.any(radius <= 0.0) or np.any(np.diff(radius) > 1.0e-12):
            raise ValueError("Radius must remain positive and non-increasing")

```

```

        if self.interpolation not in ("linear", "pchip"):
            raise ValueError("Radius interpolation must be linear or pchip")
        object.__setattr__(self, "time_s", time)
        object.__setattr__(self, "radius_m", radius)
        object.__setattr__(self, "_pchip", PchipInterpolator(time, radius) if self.interpolation ==
"pchip" else None)

```

```

@dataclass(frozen=True)

```

```

class Problem4Solution:

```

```

    """Full solution on the dimensionless moving-boundary coordinate."""

```

```

    time_s: NDArray[np.float64]

```

```

    xi: NDArray[np.float64]

```

```

    temperature_c: NDArray[np.float64]

```

```

    moisture_concentration: NDArray[np.float64]

```

```

def load_radius_history_csv(path: str | Path) -> RadiusHistory:

```

```

    """Load Attachment 2 exported as UTF-8 CSV."""

```

```

    rows: list[tuple[float, float]] = []

```

```

    with Path(path).open("r", encoding="utf-8-sig", newline="") as handle:

```

```

        reader = csv.DictReader(handle)

```

```

        if reader.fieldnames is None:

```

```

            raise ValueError("Attachment 2 has no header")

```

```

        names = set(reader.fieldnames)

```

```

        time_name = next((name for name in ("时间", "time", "time_s", "t") if name in names), None)

```

```

        radius_name = next((name for name in ("半径", "radius", "radius_cm", "R") if name in names),

```

```

None)

```

```

        # The official workbook may be exported with a locale-specific or

```

```

        # mojibake header. Attachment 2 has exactly two columns, so a

```

```

        # positional fallback is unambiguous and keeps the data auditable.

```

```

        if time_name is None or radius_name is None:

```

```

            time_name, radius_name = reader.fieldnames[:2]

```

```

        for line_number, row in enumerate(reader, start=2):

```

```

            try:

```

```

                rows.append((float(row[time_name]), float(row[radius_name])))

```

```

            except (TypeError, ValueError) as exc:

```

```

                raise ValueError(f"Invalid numeric value on CSV line {line_number}") from exc

```

```

        if not rows:

```

```

            raise ValueError("Attachment 2 contains no data rows")

```

```

        values = np.asarray(rows, dtype=float)

```

```

        return RadiusHistory(values[:, 0], values[:, 1] * 0.01)

```

```

def interpolate_radius(time_s: ArrayLike, history: RadiusHistory) -> NDArray[np.float64]:

```

```

    """Interpolate radius with the selected method and hold the measured tail."""

```

```

    query = np.asarray(time_s, dtype=float)

```

```

    if np.any(~np.isfinite(query)) or np.any(query < history.time_s[0]):

```

```

        raise ValueError("Requested radius time precedes Attachment 2")

```

```

    if history._pchip is not None:

```

```

        return history._pchip(np.minimum(query, history.time_s[-1]))

```

```

    return np.interp(query, history.time_s, history.radius_m)

```

```

def _concentration_array(concentration: ArrayLike) -> NDArray[np.float64]:

```

```

    values = np.asarray(concentration, dtype=float)

```

```

    if np.any(~np.isfinite(values)) or np.any(values < 0.0):

```

```

        raise ValueError("Moisture concentration must be finite and non-negative")

```

```

    return values

```

```

def density_q4(concentration: ArrayLike) -> NDArray[np.float64]:

```

```

    values = _concentration_array(concentration)

```

```

    return 760.0 + 90.0 * values

```

```

def heat_capacity_q4(concentration: ArrayLike) -> NDArray[np.float64]:

```

```

    values = _concentration_array(concentration)

```

```

    return 1850.0 + 2150.0 * values / (values + 1.0)

```

```

def thermal_conductivity_q4(concentration: ArrayLike) -> NDArray[np.float64]:
    values = _concentration_array(concentration)
    return 0.12 + 0.20 * values / (values + 1.0)

def diffusivity_q4(
    concentration: ArrayLike,
    temperature_c: ArrayLike,
) -> NDArray[np.float64]:
    moisture = _concentration_array(concentration)
    temperature_k = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(~np.isfinite(temperature_k)) or np.any(temperature_k <= 0.0):
        raise ValueError("Temperature must be finite and above absolute zero")
    if np.any(moisture <= 0.0):
        raise ValueError("Moisture concentration must be positive for diffusivity")
    moisture, temperature_k = np.broadcast_arrays(moisture, temperature_k)
    return 4.2e-4 * np.exp(-0.30 / moisture) * np.exp(-3850.0 / temperature_k)

def solve_problem4(
    history: ChamberHistory,
    radius_history: RadiusHistory,
    radial_intervals: int,
    output_times_s: ArrayLike,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
    relative_tolerance: float = 2.0e-8,
    max_step_s: float = 60.0,
    maximum_moisture_threshold: float | None = None,
    initial_time_s: float | None = None,
    initial_temperature_c: ArrayLike | None = None,
    initial_moisture_concentration: ArrayLike | None = None,
) -> Problem4Solution:
    """Integrate the coupled Appendix 4 equations on ``xi=r/R(t)``."""
    if radial_intervals < 2:
        raise ValueError("At least two radial intervals are required")
    output_times = np.asarray(output_times_s, dtype=float)
    if output_times.ndim != 1 or output_times.size == 0 or np.any(~np.isfinite(output_times)):
        raise ValueError("Output times must be a non-empty finite array")
    if np.any(np.diff(output_times) <= 0.0):
        raise ValueError("Output times must be strictly increasing")
    start_time = history.time_s[0] if initial_time_s is None else float(initial_time_s)
    if not np.isfinite(start_time) or start_time < max(history.time_s[0], radius_history.time_s[0]) or
    start_time > output_times[0]:
        raise ValueError("Initial time must not be after the first output time")
    if output_times[-1] > history.time_s[-1]:
        raise ValueError("Output times exceed the supplied chamber history")
    if maximum_moisture_threshold is not None and not (
        0.0 < maximum_moisture_threshold < parameters.initial_moisture_concentration
    ):
        raise ValueError("Maximum moisture threshold must lie between zero and the initial value")

    node_count = radial_intervals + 1
    xi = np.linspace(0.0, 1.0, node_count)
    tri = diags(
        [np.ones(node_count - 1), np.ones(node_count), np.ones(node_count - 1)],
        offsets=[-1, 0, 1], shape=(node_count, node_count), format="csr",
    )
    sparsity = bmat([[tri, tri], [tri, tri]], format="csr")

    supplied_initial = (initial_temperature_c is not None, initial_moisture_concentration is not None)
    if any(supplied_initial) and not all(supplied_initial):
        raise ValueError("Both initial fields must be supplied together")
    if all(supplied_initial):
        temperature_initial = np.asarray(initial_temperature_c, dtype=float)
        moisture_initial = np.asarray(initial_moisture_concentration, dtype=float)
        if temperature_initial.shape != (node_count,) or moisture_initial.shape != (node_count,):
            raise ValueError("Initial fields have the wrong nodal shape")

```

```

        if np.any(~np.isfinite(temperature_initial)) or np.any(~np.isfinite(moisture_initial)) or
np.any(moisture_initial <= 0.0):
            raise ValueError("Initial fields must be finite and moisture must be positive")
        else:
            temperature_initial = np.full(node_count, parameters.initial_temperature_c)
            moisture_initial = np.full(node_count, parameters.initial_moisture_concentration)

def rhs(time_s: float, state: NDArray[np.float64]) -> NDArray[np.float64]:
    temperature = state[:node_count]
    concentration = np.exp(state[node_count:])
    radius = float(interpolate_radius(time_s, radius_history))
    density = density_q4(concentration)
    heat_capacity = heat_capacity_q4(concentration)
    conductivity = thermal_conductivity_q4(concentration)
    diffusivity = diffusivity_q4(concentration, temperature)
    ambient_temperature = interpolate_history(time_s, history.time_s, history.temperature_c)
    ambient_moisture = interpolate_history(time_s, history.time_s, history.moisture_concentration)
    temperature_div = radial_flux_divergence(
        temperature, conductivity, 1.0, radius * parameters.heat_transfer_coefficient_w_m2_k,
        ambient_temperature,
    ) / radius**2
    moisture_div = radial_flux_divergence(
        concentration, diffusivity, 1.0, radius * parameters.mass_transfer_coefficient_m_s,
        ambient_moisture,
    ) / radius**2
    return np.concatenate([temperature_div / (density * heat_capacity), moisture_div /
concentration])

initial_state = np.concatenate([temperature_initial, np.log(moisture_initial)])
atol = np.concatenate([np.full(node_count, 1.0e-8), np.full(node_count, 1.0e-10)])
event = None
if maximum_moisture_threshold is not None:
    def event(time_s: float, state: NDArray[np.float64]) -> float:
        del time_s
        return float(np.max(np.exp(state[node_count:])) - maximum_moisture_threshold)
    event.terminal = True
    event.direction = -1.0

result = solve_ivp(
    rhs, (start_time, float(output_times[-1])), initial_state, method="BDF", t_eval=output_times,
    rtol=relative_tolerance, atol=atol, max_step=max_step_s, jac_sparsity=sparsity, events=event,
)
if not result.success:
    raise RuntimeError(f"Problem 4 solver failed: {result.message}")
result_time = result.t
result_state = result.y
if maximum_moisture_threshold is not None:
    if not result.t_events or result.t_events[0].size == 0:
        raise RuntimeError("Maximum moisture threshold was not reached")
    event_time = float(result.t_events[0][0])
    event_state = result.y_events[0][0]
    if result_time.size and np.isclose(result_time[-1], event_time, atol=1.0e-9, rtol=0.0):
        result_time = result_time.copy(); result_state = result_state.copy()
        result_time[-1] = event_time; result_state[:, -1] = event_state
    else:
        result_time = np.append(result_time, event_time)
        result_state = np.column_stack([result_state, event_state])
return Problem4Solution(
    time_s=np.asarray(result_time), xi=xi,
    temperature_c=result_state[:node_count].T,
    moisture_concentration=np.exp(result_state[node_count:].T),
)

def sample_moving_solution(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
    radius_cm: ArrayLike,
) -> NDArray[np.float64]:
    """Sample moisture at fixed physical radii; outside cells are NaN."""

```

```

        requested = np.asarray(radius_cm, dtype=float)
        if requested.ndim != 1 or requested.size == 0 or np.any(~np.isfinite(requested)) or
np.any(requested < 0.0) or np.any(np.diff(requested) <= 0.0):
            raise ValueError("Requested radii must be strictly increasing")
        radii_m = requested * 0.01
        sampled = np.full((solution.time_s.size, requested.size), np.nan, dtype=float)
        current_radius = interpolate_radius(solution.time_s, radius_history)
        for row, radius in enumerate(current_radius):
            valid = radii_m <= radius + 1.0e-12
            if np.any(valid):
                coordinates = radii_m[valid] / radius
                sampled[row, valid] = np.interp(coordinates, solution.xi,
solution.moisture_concentration[row])
        return sampled

def sample_surface(solution: Problem4Solution) -> NDArray[np.float64]:
    """Return the actual moving-surface concentration."""
    return solution.moisture_concentration[:, -1].copy()

def validate_radially_nonincreasing_moisture(
    solution: Problem4Solution,
    *,
    tolerance: float = 1.0e-10,
) -> float:
    """Require the stored moving-coordinate profiles to decrease outward."""
    if not np.isfinite(tolerance) or tolerance < 0.0:
        raise ValueError("Tolerance must be finite and non-negative")
    maximum_outward_increase = float(np.max(np.diff(solution.moisture_concentration, axis=1)))
    if maximum_outward_increase > tolerance:
        raise ValueError("Moisture profile contains an outward increase")
    return maximum_outward_increase

def moisture_balance_diagnostics(
    solution: Problem4Solution,
    chamber_history: ChamberHistory,
    radius_history: RadiusHistory,
    *,
    parameters: Problem1Parameters = Problem1Parameters(),
) -> dict[str, float]:
    """Check normalized-domain moisture change against the moving surface flux."""
    if not np.isclose(solution.time_s[0], chamber_history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Moisture balance requires the initial-time row")
    volumes = nodal_control_volumes(solution.xi.size, 1.0)
    mean_moisture = solution.moisture_concentration @ volumes / np.sum(volumes)
    ambient = np.interp(
        solution.time_s,
        chamber_history.time_s,
        chamber_history.moisture_concentration,
    )
    current_radius = interpolate_radius(solution.time_s, radius_history)
    mean_rate_from_surface = (
        -2.0
        * parameters.mass_transfer_coefficient_m_s
        / current_radius
        * (solution.moisture_concentration[:, -1] - ambient)
    )
    integrated_change = cumulative_simpson(
        mean_rate_from_surface,
        x=solution.time_s,
        initial=0.0,
    )
    residual = (mean_moisture - mean_moisture[0]) - integrated_change
    total_change = abs(float(mean_moisture[-1] - mean_moisture[0]))
    maximum_residual = float(np.max(np.abs(residual)))
    return {
        "maximum_absolute_residual": maximum_residual,
        "final_residual": float(residual[-1]),
    }

```

```

        "relative_max_residual": maximum_residual / max(total_change, 1.0e-15),
        "total_mean_moisture_change": float(mean_moisture[-1] - mean_moisture[0]),
        "integrated_surface_change": float(integrated_change[-1]),
    }

def density_shrinkage_compatibility(
    solution: Problem4Solution,
    radius_history: RadiusHistory,
) -> dict[str, object]:
    """Audit the hypothetical interpretation of Appendix 4 rho as wet density.

    This is separate from the moisture PDE flux balance. With fixed length,
    dry mass per unit length would be  $2\pi R^2 \int \rho(C)/(1+C) \xi d\xi$ .
    The first field must be the initial field; no conservation is imposed here.
    """
    if not np.isclose(solution.time_s[0], radius_history.time_s[0], atol=1.0e-12, rtol=0.0):
        raise ValueError("Density compatibility requires the initial-time row")
    volumes = nodal_control_volumes(solution.xi.size, 1.0)
    concentration = solution.moisture_concentration
    implied_dry_density = density_q4(concentration) / (1.0 + concentration)
    radii = interpolate_radius(solution.time_s, radius_history)
    # Unit-radius annular volumes already include pi (their sum is pi).
    mass_per_length = radii**2 * (implied_dry_density @ volumes)
    ratio = mass_per_length / mass_per_length[0]
    return {
        "interpretation": "hypothetical wet bulk density; constant cylinder length",
        "time_s": solution.time_s.tolist(),
        "implied_dry_mass_per_length_kg_m": mass_per_length.tolist(),
        "mass_ratio_to_initial": ratio.tolist(),
        "final_mass_ratio": float(ratio[-1]),
        "maximum_absolute_ratio_deviation": float(np.max(np.abs(ratio - 1.0))),
    }

```

src/drying_model/problem4_independent.py

```

"""Independent cell-centred finite-volume check of the Problem 4 model.

```

```

This implements the same continuum assumptions with different spatial unknowns
and boundary closure. It does not call the production solver, its material-law
functions, or its flux-divergence routine. Unknowns approximate point values at
the midpoints of N complete annular cells; neither the axis nor the surface is
a state node. Centre values are reconstructed by even quadratic extrapolation.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.integrate import solve_ivp
from scipy.sparse import bmat, diags

```

```

from drying_model.problem1 import ChamberHistory, Problem1Parameters
from drying_model.problem4 import RadiusHistory

```

```

@dataclass(frozen=True)
class IndependentSolution:
    time_s: NDArray[np.float64]
    xi_centres: NDArray[np.float64]
    temperature_c: NDArray[np.float64]
    moisture: NDArray[np.float64]
    surface_temperature_c: NDArray[np.float64]
    surface_moisture: NDArray[np.float64]
    centre_moisture: NDArray[np.float64]
    drying_time_s: float | None

```

```

endpoint_cell_maximum: float | None
endpoint_reconstructed_maximum: float | None

def appendix4_coefficients(moisture: ArrayLike, temperature_c: ArrayLike):
    """Transcribe Appendix 4 separately from the production implementation."""
    concentration = np.asarray(moisture, dtype=float)
    kelvin = np.asarray(temperature_c, dtype=float) + 273.15
    if np.any(concentration <= 0.0) or np.any(kelvin <= 0.0):
        raise ValueError("Positive concentration and absolute temperature required")
    rho_cp = (760.0 + 90.0 * concentration) * (
        1850.0 + 2150.0 * concentration / (concentration + 1.0)
    )
    conductivity = 0.12 + 0.20 * concentration / (concentration + 1.0)
    diffusion = 4.2e-4 * np.exp(-0.30 / concentration - 3850.0 / kelvin)
    return rho_cp, conductivity, diffusion

def reconstruct_centre(values: ArrayLike) -> NDArray[np.float64]:
    """Even quadratic continuation from xi=dx/2 and 3dx/2 to xi=0.

    These are point-centred unknowns, not volume averages. For a smooth even
    exact profile this formula's reconstruction error is O(dx**4); the solved
    point values still carry the spatial scheme's discretization error.
    """
    field = np.asarray(values, dtype=float)
    if field.shape[-1] < 2:
        raise ValueError("At least two cells are needed for axis reconstruction")
    return (9.0 * field[..., 0] - field[..., 1]) / 8.0

def surface_from_series_resistance(
    last_value: ArrayLike,
    last_coefficient: ArrayLike,
    radius_m: ArrayLike,
    cell_count: int,
    exchange: float,
    ambient: ArrayLike,
) -> tuple[NDArray[np.float64], NDArray[np.float64]]:
    """Return actual-surface value and outward flux using a half-cell resistor.

     $J = (u_{\text{last}} - u_{\text{inf}}) / (R \cdot dx / (2 \cdot a_{\text{last}}) + 1/h)$ ,  $u_{\text{surface}} = u_{\text{inf}} + J/h$ .
    The coefficient is frozen at the last cell centre. That local closure is
    a truncation approximation, checked by spatial convergence, not an exact
    nonlinear surface solve.
    """
    last = np.asarray(last_value, dtype=float)
    coefficient = np.asarray(last_coefficient, dtype=float)
    ambient = np.asarray(ambient, dtype=float)
    if cell_count < 2 or exchange < 0.0 or np.any(coefficient <= 0.0):
        raise ValueError("Invalid boundary conductance or cell count")
    if exchange == 0.0:
        return last.copy(), np.zeros_like(last)
    resistance_ratio = exchange * np.asarray(radius_m) / (2.0 * cell_count * coefficient)
    surface = (last + resistance_ratio * ambient) / (1.0 + resistance_ratio)
    return surface, exchange * (surface - ambient)

def cell_flux_divergence(
    values: ArrayLike,
    coefficient: ArrayLike,
    radius_m: float,
    exchange: float,
    ambient: float,
) -> tuple[NDArray[np.float64], float]:
    """Annular flux balance in xi with arithmetic internal-face coefficients."""
    field = np.asarray(values, dtype=float)
    transport = np.asarray(coefficient, dtype=float)
    if field.ndim != 1 or field.shape != transport.shape or radius_m <= 0.0:
        raise ValueError("Mismatched fields or nonpositive radius")

```

```

cell_count = field.size
dx = 1.0 / cell_count
faces = np.arange(cell_count + 1, dtype=float) * dx
volumes = 0.5 * (faces[1:] ** 2 - faces[:-1] ** 2)
flux = np.zeros(cell_count + 1)
flux[1:-1] = (
    faces[1:-1]
    * 0.5 * (transport[:-1] + transport[1:])
    * np.diff(field) / dx
)
surface, outward = surface_from_series_resistance(
    field[-1], transport[-1], radius_m, cell_count, exchange, ambient
)
# flux stores  $\xi a u_{\xi}$ . At  $\xi=1$  this equals  $-R^*J$ .
flux[-1] = -radius_m * float(outward)
return np.diff(flux) / (radius_m**2 * volumes), float(surface)

def solve_independent_problem4(
    history: ChamberHistory,
    radius_history: RadiusHistory,
    cell_count: int,
    output_times_s: ArrayLike,
    *,
    threshold: float | None = 0.15,
    parameters: Problem1Parameters = Problem1Parameters(),
    rtol: float = 2.0e-9,
    max_step_s: float = 60.0,
) -> IndependentSolution:
    """Solve directly for T,C and terminate on the reconstructed global maximum.

    The checked set includes all cell-centred values, the even-reconstructed
    axis and the surface. In this monotone drying case its maximum is the
    reconstructed axis. This avoids mistaking  $\xi=dx/2$  for the actual centre.
    """
    times = np.asarray(output_times_s, dtype=float)
    if cell_count < 2 or times.ndim != 1 or times.size == 0:
        raise ValueError("At least two cells and nonempty output times required")
    if np.any(np.diff(times) <= 0.0) or times[0] < history.time_s[0]:
        raise ValueError("Invalid output-time ordering")
    if times[-1] > min(history.time_s[-1], radius_history.time_s[-1]):
        raise ValueError("Output horizon exceeds supplied history")
    if threshold is not None and not (0.0 < threshold < parameters.initial_moisture_concentration):
        raise ValueError("Invalid moisture threshold")
    n = cell_count
    xi = (np.arange(n, dtype=float) + 0.5) / n
    tri = diags([np.ones(n - 1), np.ones(n), np.ones(n - 1)], [-1, 0, 1], format="csr")
    sparsity = bmat([[tri, tri], [tri, tri]], format="csr")

    def surroundings(time_s):
        radius = float(np.interp(time_s, radius_history.time_s, radius_history.radius_m))
        ambient_t = float(np.interp(time_s, history.time_s, history.temperature_c))
        ambient_c = float(np.interp(time_s, history.time_s, history.moisture_concentration))
        return radius, ambient_t, ambient_c

    def rhs(time_s, state):
        temperature, moisture = state[:n], state[n:]
        capacity, conductivity, diffusivity = appendix4_coefficients(moisture, temperature)
        radius, ambient_t, ambient_c = surroundings(time_s)
        heat_div, _ = cell_flux_divergence(
            temperature, conductivity, radius,
            parameters.heat_transfer_coefficient_w_m2_k, ambient_t,
        )
        mass_div, _ = cell_flux_divergence(
            moisture, diffusivity, radius,
            parameters.mass_transfer_coefficient_m_s, ambient_c,
        )
        return np.concatenate([heat_div / capacity, mass_div])

    def reconstructed_maximum(time_s, state):

```

```

        temperature, moisture = state[:n], state[n:]
        _, _, diffusion = appendix4_coefficients(moisture[-1], temperature[-1])
        radius, _, ambient_c = surroundings(time_s)
        surface, _ = surface_from_series_resistance(
            moisture[-1], diffusion, radius, n,
            parameters.mass_transfer_coefficient_m_s, ambient_c,
        )
        return max(float(np.max(moisture)), float(reconstruct_centre(moisture)), float(surface))

def event(time_s, state):
    return reconstructed_maximum(time_s, state) - threshold

event.terminal = True
event.direction = -1.0
initial_state = np.concatenate([
    np.full(n, parameters.initial_temperature_c),
    np.full(n, parameters.initial_moisture_concentration),
])
result = solve_ivp(
    rhs, (float(history.time_s[0]), float(times[-1])), initial_state,
    t_eval=times, method="BDF", rtol=rtol,
    atol=np.concatenate([np.full(n, 1.0e-9), np.full(n, 1.0e-11)]),
    max_step=max_step_s, jac_sparsity=sparsity,
    events=event if threshold is not None else None,
)
if not result.success:
    raise RuntimeError(result.message)
endpoint = cell_maximum = reconstructed_max = None
result_times, states = result.t, result.y
if threshold is not None:
    if not result.t_events[0].size:
        raise RuntimeError("Threshold not reached within supplied horizon")
    endpoint = float(result.t_events[0][0])
    endpoint_state = result.y_events[0][0]
    cell_maximum = float(np.max(endpoint_state[n:]))
    reconstructed_max = reconstructed_maximum(endpoint, endpoint_state)
    result_times = np.append(result_times, endpoint)
    states = np.column_stack([states, endpoint_state])
temperatures, moistures = states[:n].T, states[n:].T
radii = np.interp(result_times, radius_history.time_s, radius_history.radius_m)
ambient_ts = np.interp(result_times, history.time_s, history.temperature_c)
ambient_cs = np.interp(result_times, history.time_s, history.moisture_concentration)
_, last_k, last_d = appendix4_coefficients(moistures[:, -1], temperatures[:, -1])
surface_t, _ = surface_from_series_resistance(
    temperatures[:, -1], last_k, radii, n,
    parameters.heat_transfer_coefficient_w_m2_k, ambient_ts,
)
surface_c, _ = surface_from_series_resistance(
    moistures[:, -1], last_d, radii, n,
    parameters.mass_transfer_coefficient_m_s, ambient_cs,
)
return IndependentSolution(
    result_times, xi, temperatures, moistures, surface_t, surface_c,
    reconstruct_centre(moistures), endpoint, cell_maximum, reconstructed_max,
)

def sample_independent_moisture(
    solution: IndependentSolution,
    radius_history: RadiusHistory,
    physical_radii_cm: ArrayLike,
) -> NDArray[np.float64]:
    """Piecewise-linear interpolation including separately reconstructed ends."""
    radii_m = np.asarray(physical_radii_cm, dtype=float) * 0.01
    if radii_m.ndim != 1 or np.any(radii_m < 0.0):
        raise ValueError("Nonnegative one-dimensional physical radii required")
    result = np.full((solution.time_s.size, radii_m.size), np.nan)
    xi = np.concatenate([[0.0], solution.xi_centres, [1.0]])
    for row, time_s in enumerate(solution.time_s):
        radius = float(np.interp(time_s, radius_history.time_s, radius_history.radius_m))

```

```

        field = np.concatenate([
            [solution.centre_moisture[row]], solution.moisture[row],
            [solution.surface_moisture[row]],
        ])
        valid = radii_m <= radius + 1.0e-12
        result[row, valid] = np.interp(radii_m[valid] / radius, xi, field)
    return result

```

tests/test_final_review_regressions.py

```

"""Regressions found while reviewing the four-question delivery."""

import importlib.util
from pathlib import Path
import sys

import numpy as np
import pytest

from drying_model.problem1 import ChamberHistory, Problem1Solution, result_payload
from drying_model.problem3 import Problem3Result, problem3_payload, truncate_solution_at_threshold

def test_fractional_second_is_not_silently_rounded_at_large_times():
    solution = Problem1Solution(np.array([10800.01]), np.array([0.0, 0.02]),
                                np.full((1, 2), 50.0), np.full((1, 2), 0.2))
    with pytest.raises(ValueError, match="whole seconds"):
        result_payload(solution)

@pytest.mark.parametrize("column", [0, 1, 2])
def test_history_rejects_nonfinite_observation_columns(column):
    columns = [np.array([0.0, 60.0]), np.array([28.0, 30.0]), np.array([0.02, 0.03])]
    columns[column][1] = np.nan
    with pytest.raises(ValueError, match="finite"):
        ChamberHistory(*columns)

def test_threshold_interpolation_when_wettest_node_changes():
    solution = Problem1Solution(
        np.array([60.0, 120.0, 180.0]), np.array([0.0, 0.02]), np.full((3, 2), 50.0),
        np.array([[0.20, 0.19], [0.14, 0.149], [0.13, 0.14]]),
    )
    actual = truncate_solution_at_threshold(solution, threshold=0.15, output_interval_s=60.0)
    expected = 60.0 + 60.0 * (0.19 - 0.15) / (0.19 - 0.149)
    assert actual.time_s[-2] == pytest.approx(expected, abs=1e-10)
    assert np.max(actual.moisture_concentration[-2]) == pytest.approx(0.15, abs=1e-14)
    assert actual.time_s[-1] == 120.0

def test_critical_moisture_validation_uses_absolute_tolerance_only():
    solution = Problem1Solution(
        np.array([60.0, 90.0, 120.0]), np.array([0.0, 0.02]), np.full((3, 2), 50.0),
        np.array([[0.16, 0.10], [0.150001, 0.09], [0.14999, 0.08]]),
    )
    result = Problem3Result(solution, 90.0, 120.0, 0.150001, 0.0, 50.0, 0.05, 0.15, 60.0)
    with pytest.raises(ValueError, match="unrounded threshold"):
        problem3_payload(result)

@pytest.mark.parametrize("question", [1, 3])
def test_runner_rejects_invalid_richardson_grid_ratio(question, monkeypatch):
    path = Path(__file__).resolve().parents[1] / "scripts" / f"run_problem{question}.py"
    spec = importlib.util.spec_from_file_location(f"run_problem{question}", path)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    monkeypatch.setattr(sys, "argv", [str(path), "--coarse-radial-intervals", "8", "--fine-radial-intervals", "24"])

```

```

with pytest.raises(ValueError, match="twice"):
    module.main()

```

tests/test_problem1.py

```

import math
from pathlib import Path

import numpy as np

from drying_model.problem1 import (
    ChamberHistory,
    Problem1Solution,
    diffusivity_q1,
    richardson_extrapolate_solutions,
    nodal_control_volumes,
    radial_flux_divergence,
    interpolate_history,
    load_chamber_history_csv,
    result_payload,
    sample_solution,
    solve_problem1,
)

def test_q1_diffusivity_uses_entire_minus_089_over_c_exponent():
    concentration = np.array([2.55, 1.0, 0.5])

    actual = diffusivity_q1(concentration)

    expected = 7.0e-9 * np.exp(-0.89 / concentration)
    np.testing.assert_allclose(actual, expected, rtol=1e-14, atol=0.0)

def test_chamber_history_is_linearly_interpolated_in_seconds():
    time = np.array([0.0, 60.0, 120.0])
    values = np.array([28.0, 30.0, 29.0])

    assert math.isclose(interpolate_history(30.0, time, values), 29.0)
    assert math.isclose(interpolate_history(90.0, time, values), 29.5)

def test_radial_flux_divergence_preserves_uniform_equilibrium():
    state = np.full(9, 2.55)

    rhs = radial_flux_divergence(
        state,
        coefficient=np.full_like(state, 4.0e-9),
        radius_m=0.02,
        exchange_coefficient=8.0e-7,
        ambient_value=2.55,
    )

    np.testing.assert_allclose(rhs, 0.0, atol=1e-18)

def test_closed_radial_domain_conserves_volume_weighted_state():
    state = np.array([2.6, 2.5, 2.4, 2.2, 2.0], dtype=float)
    radius_m = 0.02

    rhs = radial_flux_divergence(
        state,
        coefficient=np.full_like(state, 5.0e-9),
        radius_m=radius_m,
        exchange_coefficient=0.0,
        ambient_value=0.0,
    )

    volumes = nodal_control_volumes(state.size, radius_m)

```

```

assert abs(float(np.dot(rhs, volumes))) < 1e-18

def test_surface_exchange_removes_exact_integrated_amount():
    state = np.full(5, 2.55)
    radius_m = 0.02
    transfer = 8.0e-7
    ambient = 0.02

    rhs = radial_flux_divergence(
        state,
        coefficient=np.full_like(state, 5.0e-9),
        radius_m=radius_m,
        exchange_coefficient=transfer,
        ambient_value=ambient,
    )
    volumes = nodal_control_volumes(state.size, radius_m)
    expected_per_length = -2.0 * math.pi * radius_m * transfer * (2.55 - ambient)

    assert math.isclose(float(np.dot(rhs, volumes)), expected_per_length, rel_tol=1e-14)

def test_solver_preserves_uniform_state_at_matching_ambient_conditions():
    history = ChamberHistory(
        time_s=np.array([0.0, 60.0]),
        temperature_c=np.array([28.0, 28.0]),
        moisture_concentration=np.array([2.55, 2.55]),
    )

    solution = solve_problem1(
        history,
        radial_intervals=8,
        output_times_s=np.array([0.0, 30.0, 60.0]),
        max_step_s=5.0,
    )

    np.testing.assert_allclose(solution.temperature_c, 28.0, atol=1e-11)
    np.testing.assert_allclose(solution.moisture_concentration, 2.55, atol=1e-11)

def test_solver_creates_expected_surface_to_center_gradients():
    history = ChamberHistory(
        time_s=np.array([0.0, 120.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.02, 0.02]),
    )

    solution = solve_problem1(
        history,
        radial_intervals=12,
        output_times_s=np.array([0.0, 120.0]),
        max_step_s=2.0,
    )

    final_temperature = solution.temperature_c[-1]
    final_moisture = solution.moisture_concentration[-1]
    assert 28.0 <= final_temperature[0] < final_temperature[-1] < 50.0
    assert 0.02 < final_moisture[-1] < final_moisture[0] <= 2.55

def test_output_can_start_after_initial_time_without_resetting_initial_state():
    history = ChamberHistory(
        time_s=np.array([0.0, 10.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.02, 0.02]),
    )

    with_initial = solve_problem1(
        history,

```

```

        radial_intervals=6,
        output_times_s=np.array([0.0, 1.0]),
        max_step_s=0.2,
    )
    without_initial = solve_problem1(
        history,
        radial_intervals=6,
        output_times_s=np.array([1.0]),
        max_step_s=0.2,
    )

    np.testing.assert_allclose(without_initial.temperature_c[0], with_initial.temperature_c[1])
    np.testing.assert_allclose(
        without_initial.moisture_concentration[0], with_initial.moisture_concentration[1]
    )

def test_load_chamber_history_csv_reads_chinese_headers(tmp_path: Path):
    source = tmp_path / "attachment1.csv"
    source.write_text(
        "时间,温度,水分浓度\n0,28,0.01963\n60,28.528,0.02002\n",
        encoding="utf-8-sig",
    )

    history = load_chamber_history_csv(source)

    np.testing.assert_array_equal(history.time_s, [0.0, 60.0])
    np.testing.assert_allclose(history.temperature_c, [28.0, 28.528])
    np.testing.assert_allclose(history.moisture_concentration, [0.01963, 0.02002])

def test_sample_solution_interpolates_to_requested_physical_radii():
    history = ChamberHistory(
        time_s=np.array([0.0, 10.0]),
        temperature_c=np.array([40.0, 40.0]),
        moisture_concentration=np.array([0.02, 0.02]),
    )
    solution = solve_problem1(
        history,
        radial_intervals=8,
        output_times_s=np.array([0.0, 10.0]),
        max_step_s=1.0,
    )

    sampled = sample_solution(solution, radius_cm=np.array([0.0, 0.5, 1.0, 1.5, 2.0]))

    assert sampled.temperature_c.shape == (2, 5)
    assert sampled.moisture_concentration.shape == (2, 5)
    np.testing.assert_allclose(sampled.radius_m, np.array([0.0, 0.005, 0.01, 0.015, 0.02]))

def test_result_payload_uses_seconds_centimetres_and_four_decimal_values():
    solution = solve_problem1(
        ChamberHistory(
            time_s=np.array([0.0, 1.0]),
            temperature_c=np.array([28.0, 28.0]),
            moisture_concentration=np.array([2.55, 2.55]),
        ),
        radial_intervals=2,
        output_times_s=np.array([1.0]),
        max_step_s=0.2,
    )

    payload = result_payload(solution)

    assert payload["time_s"] == [1]
    assert payload["radius_cm"] == [0.0, 1.0, 2.0]
    assert payload["temperature_c"] == [[28.0, 28.0, 28.0]]
    assert payload["moisture_concentration"] == [[2.55, 2.55, 2.55]]

```

```

def test_richardson_extrapolation_removes_second_order_spatial_error():
    time = np.array([1.0, 2.0])
    radius = np.array([0.0, 0.02])
    exact_temperature = np.array([[28.0, 29.0], [30.0, 31.0]])
    exact_moisture = np.array([[2.5, 2.4], [2.3, 2.2]])
    fine_error_temperature = np.array([[0.03, -0.06], [0.09, -0.12]])
    fine_error_moisture = np.array([[0.003, -0.006], [0.009, -0.012]])
    coarse = Problem1Solution(
        time_s=time,
        radius_m=radius,
        temperature_c=exact_temperature + 4.0 * fine_error_temperature,
        moisture_concentration=exact_moisture + 4.0 * fine_error_moisture,
    )
    fine = Problem1Solution(
        time_s=time,
        radius_m=radius,
        temperature_c=exact_temperature + fine_error_temperature,
        moisture_concentration=exact_moisture + fine_error_moisture,
    )

    extrapolated = richardson_extrapolate_solutions(coarse, fine, order=2)

    np.testing.assert_allclose(extrapolated.temperature_c, exact_temperature)
    np.testing.assert_allclose(extrapolated.moisture_concentration, exact_moisture)
    np.testing.assert_array_equal(extrapolated.time_s, time)
    np.testing.assert_array_equal(extrapolated.radius_m, radius)

def test_richardson_extrapolation_rejects_different_output_grids():
    coarse = Problem1Solution(
        time_s=np.array([1.0]),
        radius_m=np.array([0.0, 0.02]),
        temperature_c=np.array([[28.0, 29.0]]),
        moisture_concentration=np.array([[2.5, 2.4]]),
    )
    fine = Problem1Solution(
        time_s=np.array([2.0]),
        radius_m=np.array([0.0, 0.02]),
        temperature_c=np.array([[28.0, 29.0]]),
        moisture_concentration=np.array([[2.5, 2.4]]),
    )

    with np.testing.assert_raises(ValueError):
        richardson_extrapolate_solutions(coarse, fine, order=2)

```

tests/test_problem2.py

```

import numpy as np

from drying_model.problem1 import (
    ChamberHistory,
    Problem1Parameters,
    diffusivity_q1,
    solve_problem1,
)
from drying_model.problem2 import (
    density_q2,
    diffusivity_q2,
    heat_capacity_q2,
    solve_problem2,
    solve_problem2_sampled_in_chunks,
    solve_variable_property_fixed_cylinder,
    thermal_conductivity_q2,
)

def test_q2_material_laws_match_appendix_3_exactly():

```

```

concentration = np.array([0.15, 1.0, 2.55])
temperature_c = np.array([28.0, 40.0, 50.0])

np.testing.assert_allclose(density_q2(concentration), 650.0 + 128.0 * concentration)
np.testing.assert_allclose(
    heat_capacity_q2(concentration),
    1450.0 + 2736.0 * concentration / (concentration + 1.0),
)
np.testing.assert_allclose(
    thermal_conductivity_q2(concentration),
    0.21 + 0.38 * concentration / (concentration + 1.0),
)
np.testing.assert_allclose(
    diffusivity_q2(concentration, temperature_c),
    2.4e-3
    * np.exp(-0.45 / concentration)
    * np.exp(-3850.0 / (temperature_c + 273.15)),
)

def test_q2_material_laws_reject_nonphysical_inputs():
    for law in (density_q2, heat_capacity_q2, thermal_conductivity_q2):
        with np.testing.assert_raises(ValueError):
            law(np.array([-1.0]))
        with np.testing.assert_raises(ValueError):
            diffusivity_q2(np.array([0.0]), np.array([28.0]))
        with np.testing.assert_raises(ValueError):
            diffusivity_q2(np.array([1.0]), np.array([-273.15]))

def test_q2_uniform_equilibrium_remains_constant():
    history = ChamberHistory(
        time_s=np.array([0.0, 20.0]),
        temperature_c=np.array([28.0, 28.0]),
        moisture_concentration=np.array([2.55, 2.55]),
    )

    solution = solve_problem2(
        history,
        radial_intervals=8,
        output_times_s=np.array([0.0, 10.0, 20.0]),
        max_step_s=1.0,
    )

    np.testing.assert_allclose(solution.temperature_c, 28.0, atol=1.0e-11)
    np.testing.assert_allclose(solution.moisture_concentration, 2.55, atol=1.0e-11)

def test_generic_coupled_solver_degenerates_to_problem1_laws():
    history = ChamberHistory(
        time_s=np.array([0.0, 30.0]),
        temperature_c=np.array([45.0, 45.0]),
        moisture_concentration=np.array([0.02, 0.02]),
    )
    parameters = Problem1Parameters()
    output_times = np.array([0.0, 10.0, 30.0])
    expected = solve_problem1(
        history,
        radial_intervals=10,
        output_times_s=output_times,
        parameters=parameters,
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )

    actual = solve_variable_property_fixed_cylinder(
        history,
        radial_intervals=10,
        output_times_s=output_times,
        density_law=lambda concentration: np.full_like(concentration, parameters.density_kg_m3),
    )

```

```

        heat_capacity_law=lambda concentration: np.full_like(
            concentration, parameters.heat_capacity_j_kg_k
        ),
        conductivity_law=lambda concentration: np.full_like(
            concentration, parameters.thermal_conductivity_w_m_k
        ),
        diffusivity_law=lambda concentration, temperature_c: diffusivity_q1(concentration),
        parameters=parameters,
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )

    np.testing.assert_allclose(actual.temperature_c, expected.temperature_c, atol=2.0e-8)
    np.testing.assert_allclose(
        actual.moisture_concentration,
        expected.moisture_concentration,
        atol=2.0e-8,
    )

def test_q2_hot_dry_environment_creates_expected_surface_gradients():
    history = ChamberHistory(
        time_s=np.array([0.0, 120.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.02, 0.02]),
    )

    solution = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=np.array([120.0]),
        max_step_s=1.0,
    )

    assert solution.temperature_c[0, -1] > solution.temperature_c[0, 0]
    assert solution.moisture_concentration[0, -1] < solution.moisture_concentration[0, 0]
    assert np.all(solution.moisture_concentration > 0.0)

def test_coupled_solver_appends_exact_maximum_moisture_threshold_event():
    history = ChamberHistory(
        time_s=np.array([0.0, 1000.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.05, 0.05]),
    )
    parameters = Problem1Parameters(radius_m=1.0e-4)

    solution = solve_problem2(
        history,
        radial_intervals=8,
        output_times_s=np.arange(0.0, 1001.0, 60.0),
        parameters=parameters,
        relative_tolerance=1.0e-9,
        max_step_s=1.0,
        maximum_moisture_threshold=0.2,
    )

    assert solution.time_s[-1] % 60.0 != 0.0
    np.testing.assert_allclose(
        np.max(solution.moisture_concentration[-1]),
        0.2,
        atol=1.0e-9,
    )
    assert np.all(np.max(solution.moisture_concentration[:-1], axis=1) > 0.2)

def test_coupled_solver_reports_when_threshold_is_not_reached():
    history = ChamberHistory(
        time_s=np.array([0.0, 100.0]),
        temperature_c=np.array([50.0, 50.0]),

```

```

        moisture_concentration=np.array([0.05, 0.05]),
    )

    with np.testing.assert_raises_regex(RuntimeError, "not reached"):
        solve_problem2(
            history,
            radial_intervals=8,
            output_times_s=np.arange(0.0, 101.0, 20.0),
            maximum_moisture_threshold=0.01,
        )

def test_q2_solver_can_continue_from_a_previous_chunk_without_resetting_state():
    history = ChamberHistory(
        time_s=np.array([0.0, 120.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.05, 0.05]),
    )
    monolithic = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=np.array([60.0, 120.0]),
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )
    first_chunk = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=np.array([60.0]),
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )
    second_chunk = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=np.array([120.0]),
        initial_time_s=60.0,
        initial_temperature_c=first_chunk.temperature_c[-1],
        initial_moisture_concentration=first_chunk.moisture_concentration[-1],
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )

    np.testing.assert_allclose(
        second_chunk.temperature_c[-1],
        monolithic.temperature_c[-1],
        atol=2.0e-7,
    )
    np.testing.assert_allclose(
        second_chunk.moisture_concentration[-1],
        monolithic.moisture_concentration[-1],
        atol=2.0e-9,
    )

def test_chunked_q2_sampling_matches_monolithic_solution():
    history = ChamberHistory(
        time_s=np.array([0.0, 120.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.05, 0.05]),
    )
    output_times = np.arange(1.0, 121.0)
    sample_radius_cm = np.array([0.0, 1.0, 2.0])
    monolithic = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=output_times,
        relative_tolerance=1.0e-9,
        max_step_s=0.5,
    )

```

```

)

actual = solve_problem2_sampled_in_chunks(
    history,
    radial_intervals=16,
    output_times_s=output_times,
    sample_radius_cm=sample_radius_cm,
    chunk_duration_s=30.0,
    relative_tolerance=1.0e-9,
    max_step_s=0.5,
)

np.testing.assert_array_equal(actual.time_s, output_times)
np.testing.assert_allclose(actual.radius_m, sample_radius_cm * 0.01)
np.testing.assert_allclose(
    actual.temperature_c,
    monolithic.temperature_c[:, [0, 8, 16]],
    atol=3.0e-7,
)
np.testing.assert_allclose(
    actual.moisture_concentration,
    monolithic.moisture_concentration[:, [0, 8, 16]],
    atol=3.0e-9,
)

```

tests/test_problem3.py

```

import numpy as np
import drying_model.problem3 as problem3

from drying_model.problem1 import ChamberHistory, Problem1Parameters, Problem1Solution
from drying_model.problem2 import solve_problem2
from drying_model.problem3 import (
    Problem3Result,
    extend_chamber_history_to_plateau,
    problem3_payload,
    richardson_extrapolate_drying_time,
    solve_problem3,
    truncate_solution_at_threshold,
)

def test_plateau_extension_preserves_observations_and_uses_tail_mean():
    history = ChamberHistory(
        time_s=np.array([0.0, 60.0, 120.0, 180.0]),
        temperature_c=np.array([20.0, 30.0, 40.0, 50.0]),
        moisture_concentration=np.array([0.4, 0.3, 0.2, 0.1]),
    )

    extended = extend_chamber_history_to_plateau(
        history,
        end_time_s=600.0,
        averaging_window_s=120.0,
        transition_s=60.0,
    )

    np.testing.assert_array_equal(extended.time_s[:4], history.time_s)
    np.testing.assert_array_equal(extended.temperature_c[:4], history.temperature_c)
    np.testing.assert_array_equal(
        extended.moisture_concentration[:4],
        history.moisture_concentration,
    )
    np.testing.assert_array_equal(extended.time_s[-2:], np.array([240.0, 600.0]))
    np.testing.assert_allclose(extended.temperature_c[-2:], 40.0)
    np.testing.assert_allclose(extended.moisture_concentration[-2:], 0.2)

def test_problem3_solver_stops_on_full_field_threshold():

```

```

history = ChamberHistory(
    time_s=np.array([0.0, 100.0]),
    temperature_c=np.array([50.0, 50.0]),
    moisture_concentration=np.array([0.05, 0.05]),
)
parameters = Problem1Parameters(radius_m=1.0e-4)

result = solve_problem3(
    history,
    radial_intervals=8,
    horizon_s=1000.0,
    output_interval_s=60.0,
    threshold=0.2,
    averaging_window_s=100.0,
    parameters=parameters,
    relative_tolerance=1.0e-9,
    max_step_s=1.0,
)

assert result.drying_time_s == result.solution.time_s[-1]
assert result.strict_completion_time_s is None
assert result.drying_time_s % 60.0 != 0.0
np.testing.assert_allclose(result.maximum_moisture, 0.2, atol=1.0e-9)
assert result.maximum_radius_m in result.solution.radius_m
assert np.all(
    np.max(result.solution.moisture_concentration[:-1], axis=1) > 0.2
)

def test_problem3_payload_keeps_exact_endpoint_and_rounds_only_outputs():
    solution = Problem1Solution(
        time_s=np.array([60.0, 120.0, 135.567891, 180.0]),
        radius_m=np.array([0.0, 0.01, 0.02]),
        temperature_c=np.full((4, 3), 50.0),
        moisture_concentration=np.array(
            [
                [0.20006, 0.19004, 0.18003],
                [0.16006, 0.15504, 0.15103],
                [0.15, 0.14994, 0.14991],
                [0.1499, 0.1497, 0.1495],
            ]
        ),
    )
    result = Problem3Result(
        solution=solution,
        drying_time_s=135.567891,
        strict_completion_time_s=180.0,
        maximum_moisture=0.15,
        maximum_radius_m=0.0,
        plateau_temperature_c=50.0,
        plateau_moisture_concentration=0.05,
        threshold=0.15,
        output_interval_s=60.0,
    )

    payload = problem3_payload(result)

    assert payload["time_s"] == [60, 120, 135.567891, 180]
    assert payload["radius_cm"] == [0.0, 1.0, 2.0]
    assert payload["moisture_concentration"] == [
        [0.2001, 0.19, 0.18],
        [0.1601, 0.155, 0.151],
        [0.15, 0.1499, 0.1499],
        [0.1499, 0.1497, 0.1495],
    ]
    assert payload["drying_time_s"] == 135.567891
    assert payload["drying_time_h"] == round(135.567891 / 3600.0, 10)
    assert payload["strict_completion_time_s"] == 180
    assert payload["strict_completion_time_h"] == 0.05

```

```

def test_problem3_payload_distinguishes_nearby_critical_and_regular_times():
    solution = Problem1Solution(
        time_s=np.array([206880.0, 206939.5, 206940.0]),
        radius_m=np.array([0.0, 0.02]),
        temperature_c=np.full((3, 2), 50.0),
        moisture_concentration=np.array(
            [[0.1501, 0.05], [0.15, 0.05], [0.1499, 0.05]]
        ),
    )
    result = Problem3Result(
        solution=solution,
        drying_time_s=206939.5,
        strict_completion_time_s=206940.0,
        maximum_moisture=0.15,
        maximum_radius_m=0.0,
        plateau_temperature_c=50.0,
        plateau_moisture_concentration=0.05,
        threshold=0.15,
        output_interval_s=60.0,
    )

    payload = problem3_payload(result)

    assert payload["time_s"][-2:] == [206939.5, 206940]

def test_richardson_extrapolates_second_order_drying_time_error():
    assert richardson_extrapolate_drying_time(104.0, 101.0, order=2) == 100.0

    with np.testing.assert_raises(ValueError):
        richardson_extrapolate_drying_time(104.0, 101.0, order=0)

def test_radial_moisture_validation_rejects_an_outward_increase():
    valid = Problem1Solution(
        time_s=np.array([60.0, 120.0]),
        radius_m=np.array([0.0, 0.01, 0.02]),
        temperature_c=np.full((2, 3), 50.0),
        moisture_concentration=np.array(
            [[0.30, 0.20, 0.10], [0.15, 0.14, 0.05]]
        ),
    )
    invalid = Problem1Solution(
        time_s=valid.time_s,
        radius_m=valid.radius_m,
        temperature_c=valid.temperature_c,
        moisture_concentration=np.array(
            [[0.30, 0.10, 0.20], [0.15, 0.14, 0.05]]
        ),
    )

    assert problem3.validate_radially_nonincreasing_moisture(valid) <= 0.0
    with np.testing.assert_raises_regex(ValueError, "outward increase"):
        problem3.validate_radially_nonincreasing_moisture(invalid)

def test_full_solution_moisture_change_matches_integrated_surface_flux():
    history = ChamberHistory(
        time_s=np.array([0.0, 600.0]),
        temperature_c=np.array([50.0, 50.0]),
        moisture_concentration=np.array([0.05, 0.05]),
    )
    parameters = Problem1Parameters()
    solution = solve_problem2(
        history,
        radial_intervals=16,
        output_times_s=np.arange(0.0, 601.0),
        parameters=parameters,
        relative_tolerance=1.0e-9,
    )

```

```

        max_step_s=0.5,
    )

    diagnostics = problem3.moisture_balance_diagnostics(
        solution,
        history,
        parameters=parameters,
    )

    assert diagnostics["relative_max_residual"] < 1.0e-6
    assert abs(diagnostics["final_residual"]) < 1.0e-8

def test_threshold_interpolation_keeps_crossing_and_first_strict_regular_row():
    times = np.array([60.0, 120.0, 130.0, 140.0, 180.0])
    moisture = np.array(
        [
            [0.20, 0.18],
            [0.16, 0.14],
            [0.155, 0.135],
            [0.145, 0.13],
            [0.10, 0.09],
        ]
    )
    solution = Problem1Solution(
        time_s=times,
        radius_m=np.array([0.0, 0.02]),
        temperature_c=np.column_stack([times, times + 1.0]),
        moisture_concentration=moisture,
    )

    truncated = truncate_solution_at_threshold(
        solution,
        threshold=0.15,
        output_interval_s=60.0,
    )

    np.testing.assert_allclose(
        truncated.time_s,
        np.array([60.0, 120.0, 135.0, 180.0]),
    )
    np.testing.assert_allclose(truncated.moisture_concentration[-2, 0], 0.15)
    assert np.max(truncated.moisture_concentration[-1]) < 0.15
    np.testing.assert_allclose(truncated.temperature_c[-2], np.array([135.0, 136.0]))

```

tests/test_problem4.py

```

import numpy as np
import pytest

from drying_model.problem1 import ChamberHistory, Problem1Parameters
from drying_model.problem4 import (
    RadiusHistory,
    density_q4,
    density_shrinkage_compatibility,
    diffusivity_q4,
    heat_capacity_q4,
    interpolate_radius,
    moisture_balance_diagnostics,
    sample_moving_solution,
    solve_problem4,
    thermal_conductivity_q4,
    Problem4Solution,
)

def test_q4_material_laws_match_appendix_4():
    c = np.array([0.15, 1.0, 2.55])

```

```

t = np.array([28.0, 40.0, 50.0])
np.testing.assert_allclose(density_q4(c), 760.0 + 90.0 * c)
np.testing.assert_allclose(heat_capacity_q4(c), 1850.0 + 2150.0 * c / (c + 1.0))
np.testing.assert_allclose(thermal_conductivity_q4(c), 0.12 + 0.20 * c / (c + 1.0))
np.testing.assert_allclose(
    diffusivity_q4(c, t), 4.2e-4 * np.exp(-0.30 / c) * np.exp(-3850.0 / (t + 273.15))
)

def test_radius_history_interpolation_is_monotone_and_holds_tail():
    history = RadiusHistory(np.array([0.0, 10.0, 20.0]), np.array([0.02, 0.018, 0.018]))
    values = interpolate_radius(np.array([0.0, 5.0, 30.0]), history)
    np.testing.assert_allclose(values, [0.02, 0.019, 0.018])

def test_uniform_equilibrium_remains_constant_with_shrinking_radius():
    chamber = ChamberHistory(np.array([0.0, 100.0]), np.array([28.0, 28.0]), np.array([2.55, 2.55]))
    radius = RadiusHistory(np.array([0.0, 50.0, 100.0]), np.array([0.02, 0.018, 0.016]))
    solution = solve_problem4(chamber, radius, 8, np.array([0.0, 50.0, 100.0]), max_step_s=2.0)
    np.testing.assert_allclose(solution.temperature_c, 28.0, atol=2.0e-10)
    np.testing.assert_allclose(solution.moisture_concentration, 2.55, atol=2.0e-10)

def test_hot_dry_boundary_creates_expected_gradients():
    chamber = ChamberHistory(np.array([0.0, 120.0]), np.array([50.0, 50.0]), np.array([0.02, 0.02]))
    radius = RadiusHistory(np.array([0.0, 120.0]), np.array([0.02, 0.019]))
    solution = solve_problem4(chamber, radius, 16, np.array([120.0]), max_step_s=1.0)
    assert solution.temperature_c[0, -1] > solution.temperature_c[0, 0]
    assert solution.moisture_concentration[0, -1] < solution.moisture_concentration[0, 0]
    assert np.all(solution.moisture_concentration > 0.0)

def test_sampling_uses_nan_for_fixed_radii_outside_shrunken_body():
    solution = Problem4Solution(
        time_s=np.array([0.0, 1.0]), xi=np.array([0.0, 0.5, 1.0]),
        temperature_c=np.full((2, 3), 30.0),
        moisture_concentration=np.array([[1.0, 0.8, 0.6], [1.0, 0.8, 0.6]]),
    )
    radius = RadiusHistory(np.array([0.0, 1.0]), np.array([0.02, 0.01]))
    sampled = sample_moving_solution(solution, radius, np.array([0.0, 1.0, 1.5, 2.0]))
    assert np.isfinite(sampled[0, :]).all()
    assert np.isfinite(sampled[1, :2]).all()
    assert np.isnan(sampled[1, 2:]).all()

def test_threshold_event_reports_exact_crossing():
    chamber = ChamberHistory(np.array([0.0, 200.0]), np.array([50.0, 50.0]), np.array([0.02, 0.02]))
    radius = RadiusHistory(np.array([0.0, 200.0]), np.array([0.0001, 0.0001]))
    result = solve_problem4(
        chamber, radius, 8, np.arange(20.0, 201.0, 20.0), maximum_moisture_threshold=0.2,
        parameters=Problem1Parameters(radius_m=0.0001), max_step_s=0.5,
    )
    assert result.time_s[-1] % 20.0 != 0.0
    np.testing.assert_allclose(np.max(result.moisture_concentration[-1]), 0.2, atol=1.0e-8)

def test_moving_domain_moisture_change_matches_surface_flux():
    chamber = ChamberHistory(np.array([0.0, 600.0]), np.array([50.0, 50.0]), np.array([0.05, 0.05]))
    radius = RadiusHistory(np.array([0.0, 600.0]), np.array([0.02, 0.018]))
    solution = solve_problem4(
        chamber, radius, 24, np.arange(0.0, 601.0), relative_tolerance=1.0e-9, max_step_s=0.5
    )
    diagnostics = moisture_balance_diagnostics(solution, chamber, radius)
    assert diagnostics["relative_max_residual"] < 2.0e-6

def test_constant_radius_reduces_to_fixed_cylinder_equations():
    from drying_model.problem2 import solve_variable_property_fixed_cylinder

    chamber = ChamberHistory(np.array([0.0, 600.0]), np.array([45.0, 50.0]), np.array([0.05, 0.03]))

```

```

radius = RadiusHistory(np.array([0.0, 600.0]), np.array([0.02, 0.02]))
times = np.array([0.0, 100.0, 600.0])
moving = solve_problem4(chamber, radius, 24, times, relative_tolerance=1e-9, max_step_s=1.0)
fixed = solve_variable_property_fixed_cylinder(
    chamber, 24, times, density_law=density_q4, heat_capacity_law=heat_capacity_q4,
    conductivity_law=thermal_conductivity_q4, diffusivity_law=diffusivity_q4,
    relative_tolerance=1e-9, max_step_s=1.0,
)
np.testing.assert_allclose(moving.temperature_c, fixed.temperature_c, rtol=1e-8, atol=1e-7)
np.testing.assert_allclose(moving.moisture_concentration, fixed.moisture_concentration, rtol=1e-8,
atol=1e-9)

@pytest.mark.parametrize("method", ["linear", "pchip"])
def test_solver_holds_radius_tail_but_requires_explicit_chamber_extension(method):
    chamber = ChamberHistory(np.array([0.0, 100.0]), np.array([50.0, 50.0]), np.array([0.02, 0.02]))
    radius = RadiusHistory(np.array([0.0, 20.0, 40.0]), np.array([0.02, 0.019, 0.018]),
interpolation=method)
    result = solve_problem4(chamber, radius, 8, [40.0, 100.0])
    assert result.time_s[-1] == 100.0
    assert np.isfinite(result.moisture_concentration).all()
    np.testing.assert_allclose(interpolate_radius([40.0, 100.0], radius), 0.018)
    with pytest.raises(ValueError, match="chamber history"):
        solve_problem4(chamber, radius, 8, [101.0])

def test_pchip_preserves_radius_observations_and_monotonicity():
    radius = RadiusHistory(np.array([0.0, 10.0, 20.0, 30.0]), np.array([0.02, 0.016, 0.015, 0.015]),
interpolation="pchip")
    np.testing.assert_allclose(interpolate_radius(radius.time_s, radius), radius.radius_m, rtol=0,
atol=1e-15)
    samples = interpolate_radius(np.linspace(0.0, 60.0, 601), radius)
    assert np.max(np.diff(samples)) <= 1e-14
    assert np.min(samples) >= 0.015 - 1e-14

def test_density_compatibility_detects_mass_loss_despite_uniform_moisture():
    solution = Problem4Solution(
        time_s=np.array([0.0, 100.0]), xi=np.linspace(0.0, 1.0, 9),
        temperature_c=np.full((2, 9), 28.0), moisture_concentration=np.full((2, 9), 2.55),
    )
    radius = RadiusHistory(solution.time_s, np.array([0.02, 0.01]))
    result = density_shrinkage_compatibility(solution, radius)
    np.testing.assert_allclose(result["mass_ratio_to_initial"], [1.0, 0.25])
    expected_initial = np.pi * 0.02**2 * (760.0 + 90.0 * 2.55) / 3.55
    np.testing.assert_allclose(result["implied_dry_mass_per_length_kg_m"][0], expected_initial)

```

tests/test_problem4_independent.py

"""Independent finite-volume checks against analytic geometry and balances."""

from dataclasses import replace

import numpy as np

from drying_model.problem1 import ChamberHistory, Problem1Parameters

from drying_model.problem4 import RadiusHistory

from drying_model.problem4_independent import (
 cell_flux_divergence,
 reconstruct_centre,
 solve_independent_problem4,
 surface_from_series_resistance,
)

def test_quadratic_cylindrical_laplacian_and_axis_reconstruction():

n = 40
 radius = 0.017

```

xi = (np.arange(n) + 0.5) / n
values = 2.0 + 0.3 * xi**2
diffusion = 2.0e-9
divergence, _ = cell_flux_divergence(values, np.full(n, diffusion), radius, 0.0, 0.0)
# Exclude the imposed zero-flux outer face, which this polynomial does not satisfy.
np.testing.assert_allclose(divergence[:-1], 4 * 0.3 * diffusion / radius**2, rtol=2e-11)
np.testing.assert_allclose(reconstruct_centre(values), 2.0, atol=1e-15)

def test_arbitrary_field_has_exact_discrete_surface_flux_balance():
    n = 35
    radius = 0.016
    values = 0.2 + np.exp(-np.linspace(0.0, 2.0, n))
    diffusion = np.linspace(1e-9, 2e-9, n)
    exchange = 8e-7
    ambient = 0.05
    divergence, surface = cell_flux_divergence(values, diffusion, radius, exchange, ambient)
    faces = np.arange(n + 1) / n
    physical_volumes_per_two_pi = 0.5 * radius**2 * np.diff(faces**2)
    np.testing.assert_allclose(
        np.dot(divergence, physical_volumes_per_two_pi),
        -radius * exchange * (surface - ambient), rtol=1e-13,
    )
    # The reported surface satisfies both pieces of the series resistance.
    inner_flux = diffusion[-1] * (values[-1] - surface) / (radius / (2 * n))
    np.testing.assert_allclose(inner_flux, exchange * (surface - ambient), rtol=1e-13)

def test_shrinking_domain_preserves_uniform_equilibrium():
    times = np.array([0.0, 100.0, 200.0])
    chamber = ChamberHistory(times, np.full(3, 28.0), np.full(3, 2.55))
    radius = RadiusHistory(times, np.array([0.02, 0.015, 0.012]))
    result = solve_independent_problem4(chamber, radius, 12, times, threshold=None)
    np.testing.assert_allclose(result.moisture, 2.55, atol=1e-12)
    np.testing.assert_allclose(result.temperature_c, 28.0, atol=1e-12)

def test_zero_exchange_keeps_initial_fields_during_shrinkage():
    times = np.array([0.0, 100.0, 200.0])
    chamber = ChamberHistory(times, np.full(3, 70.0), np.full(3, 0.05))
    radius = RadiusHistory(times, np.array([0.02, 0.015, 0.012]))
    parameters = replace(Problem1Parameters(), heat_transfer_coefficient_w_m2_k=0.0,
                        mass_transfer_coefficient_m_s=0.0)
    result = solve_independent_problem4(chamber, radius, 12, times, threshold=None,
    parameters=parameters)
    np.testing.assert_allclose(result.moisture, 2.55, atol=1e-12)
    np.testing.assert_allclose(result.temperature_c, 28.0, atol=1e-12)

```

tests/test_result4_workbook.py

"""Round-trip and deliberate-corruption checks for the submission workbook."""

from __future__ import annotations

```

import importlib.util
import json
import math
import subprocess
import sys
from pathlib import Path

```

```

import openpyxl
import pytest
from openpyxl.comments import Comment

```

```

BUILDER_PATH = Path(__file__).resolve().parents[1] / "tools" / "build_result4.py"
SPEC = importlib.util.spec_from_file_location("result4_workbook_builder", BUILDER_PATH)

```

```

builder = importlib.util.module_from_spec(SPEC)
SPEC.loader.exec_module(builder)

def _fixture(tmp_path: Path, critical: float = 119.9999):
    radius_path = tmp_path / "radius.csv"
    radius_path.write_text("time_s,radius_cm\n0,2\n300,0.5\n", encoding="utf-8")
    strict = 60 * (math.floor(critical / 60) + 1)
    times = sorted(set([*range(60, strict + 1, 60), critical]))
    radii = [index / 10 for index in range(21)]
    matrix = []
    for time in times:
        maximum = 0.15 if time == critical else 0.14998 if time == strict else 0.25
        actual_radius = 2 - time / 200
        matrix.append([
            round(maximum - index * 0.002, 4) if radius <= actual_radius + 1e-10 else None
            for index, radius in enumerate(radii)
        ])
    payload = {
        "time_s": times,
        "radius_cm": radii,
        "moisture_concentration": matrix,
        "surface_moisture_concentration": [0.04] * len(times),
        "drying_time_s": critical,
        "strict_completion_time_s": strict,
        "threshold": 0.15,
        "critical_maximum_moisture_unrounded": 0.15,
        "strict_completion_maximum_moisture_unrounded": 0.14998,
    }
    template = tmp_path / "template.xlsx"
    workbook = openpyxl.Workbook()
    workbook.active.title = "Sheet1"
    workbook.active["A1"] = builder.HEADER
    workbook.active["A20"] = "placeholder"
    workbook.properties.title = "Official numeric result template"
    workbook.save(template)
    workbook.close()
    return template, payload, tmp_path / "result4.xlsx", radius_path

@pytest.mark.parametrize("critical", [119.9999, 120.0, 120.0001, 30.0])
def test_roundtrip_preserves_nearby_critical_and_minute_rows(tmp_path, critical):
    template, payload, output, radius = _fixture(tmp_path, critical)
    summary = builder.build_result4(template, payload, output, radius)
    assert summary["status"] == "passed"
    assert summary["rows"] == len(payload["time_s"]) + 1
    assert summary["columns"] == 23
    workbook = openpyxl.load_workbook(output)
    sheet = workbook["Sheet1"]
    assert [cell.value for cell in sheet["A"][1:]] == payload["time_s"]
    assert sheet["A1"].comment is None
    assert sheet["B2"].number_format == "0.0000"
    assert workbook.properties.title == "Official numeric result template"
    # Strict compliance relies on unrounded evidence: both display as 0.1500.
    assert sheet.cell(sheet.max_row, 2).value == 0.15
    assert summary["strict_completion_maximum_moisture_unrounded"] < 0.15
    workbook.close()

@pytest.mark.parametrize(
    "corruption",
    [
        "inside_blank", "outside_zero", "surface_blank", "numeric_change",
        "radius_header", "surface_header", "time_change", "missing_minute",
        "formula", "error", "comment", "hidden_row", "hidden_column",
        "hidden_sheet", "extra_sheet", "extra_column", "boolean", "hidden_by_default",
    ],
)
def test_audit_rejects_workbook_corruption(tmp_path, corruption):
    template, payload, output, radius = _fixture(tmp_path)

```

```

builder.build_result4(template, payload, output, radius)
workbook = openpyxl.load_workbook(output)
sheet = workbook["Sheet1"]
if corruption == "inside_blank":
    sheet["B2"] = None
elif corruption == "outside_zero":
    sheet["V2"] = 0
elif corruption == "surface_blank":
    sheet["W2"] = None
elif corruption == "numeric_change":
    sheet["C2"] = sheet["C2"].value + 0.0001
elif corruption == "radius_header":
    sheet["C1"] = 0.11
elif corruption == "surface_header":
    sheet["W1"] = "2 cm"
elif corruption == "time_change":
    sheet["A3"] = 120.0 # Replaces the nearby critical time.
elif corruption == "missing_minute":
    sheet.delete_rows(2)
elif corruption == "formula":
    sheet["B2"] = "=0.25"
elif corruption == "error":
    sheet["B2"] = "#VALUE!"
elif corruption == "comment":
    sheet["A1"].comment = Comment("Unexpected annotation", "Reviewer")
elif corruption == "hidden_row":
    sheet.row_dimensions[2].hidden = True
elif corruption == "hidden_column":
    sheet.column_dimensions["C"].hidden = True
elif corruption == "hidden_by_default":
    sheet.sheet_format.zeroHeight = True
elif corruption == "hidden_sheet":
    workbook.create_sheet("hidden").sheet_state = "hidden"
elif corruption == "extra_sheet":
    workbook.create_sheet("extra")
elif corruption == "extra_column":
    sheet["X1"] = "unexpected"
elif corruption == "boolean":
    sheet["B1"] = False # Python equality alone would equate this to 0.
workbook.save(output)
workbook.close()
with pytest.raises(ValueError):
    builder.audit_result4(output, payload, radius)

@pytest.mark.parametrize(
    "corruption",
    [
        "inside_blank", "outside_zero", "surface_nan", "nonfinite_inside",
        "bad_headers", "missing_minute", "nearby_critical", "critical_error",
        "strict_not_below", "strict_rounding_mismatch", "wrong_threshold",
        "duplicate_time", "skipped_strict_minute", "wrong_geometry",
    ],
)
def test_audit_rejects_invalid_payload_even_when_workbook_could_match(tmp_path, corruption):
    _, payload, _, radius = _fixture(tmp_path)
    if corruption == "inside_blank":
        payload["moisture_concentration"][0][0] = None
    elif corruption == "outside_zero":
        payload["moisture_concentration"][0][-1] = 0
    elif corruption == "surface_nan":
        payload["surface_moisture_concentration"][0] = float("nan")
    elif corruption == "nonfinite_inside":
        payload["moisture_concentration"][0][0] = float("inf")
    elif corruption == "bad_headers":
        payload["radius_cm"][1] = 0.11
    elif corruption == "missing_minute":
        for key in ("time_s", "moisture_concentration", "surface_moisture_concentration"):
            payload[key].pop(0)
    elif corruption == "nearby_critical":

```

```

        payload["time_s"][1] = 119.9998
    elif corruption == "critical_error":
        payload["critical_maximum_moisture_unrounded"] = 0.1500002
    elif corruption == "strict_not_below":
        payload["strict_completion_maximum_moisture_unrounded"] = 0.15
    elif corruption == "strict_rounding_mismatch":
        payload["strict_completion_maximum_moisture_unrounded"] = 0.14
    elif corruption == "wrong_threshold":
        payload["threshold"] = 0.2
    elif corruption == "duplicate_time":
        payload["time_s"][-1] = payload["time_s"][-2]
    elif corruption == "skipped_strict_minute":
        payload["strict_completion_time_s"] += 60
    elif corruption == "wrong_geometry":
        radius.write_text("time_s,radius_cm\n0,2\n300,2\n", encoding="utf-8")
    with pytest.raises(ValueError):
        builder.validate_payload(payload, radius)

def test_original_cli_and_readonly_audit_mode(tmp_path):
    template, payload, output, radius = _fixture(tmp_path)
    payload_path = tmp_path / "payload.json"
    payload_path.write_text(json.dumps(payload), encoding="utf-8")
    command = [sys.executable, str(BUILDER_PATH), str(template), str(payload_path), str(output), "--radius-history", str(radius)]
    created = subprocess.run(command, capture_output=True, text=True, check=True)
    assert json.loads(created.stdout)["status"] == "passed"
    original_bytes = output.read_bytes()
    checked = subprocess.run([*command, "--audit-only"], capture_output=True, text=True, check=True)
    assert json.loads(checked.stdout)["status"] == "passed"
    assert output.read_bytes() == original_bytes

def test_radius_extension_holds_the_last_observed_value(tmp_path):
    template, payload, output, radius = _fixture(tmp_path)
    radius.write_text("time_s,radius_cm\n0,2\n60,1.7\n", encoding="utf-8")
    for time, row in zip(payload["time_s"], payload["moisture_concentration"]):
        maximum = 0.15 if time == payload["drying_time_s"] else 0.14998 if time ==
payload["strict_completion_time_s"] else 0.25
        for index, position in enumerate(payload["radius_cm"]):
            row[index] = round(maximum - index * 0.002, 4) if position <= 1.7 else None
    summary = builder.build_result4(template, payload, output, radius)
    assert summary["status"] == "passed"
    assert summary["rows_after_radius_observations"] == 2
    assert summary["radius_extension"] == "hold_last_value"
    assert builder._radius_at(90, [0, 60], [2, 1.7]) == 1.7
    assert builder._radius_at(60, [0, 60], [2, 1.7]) == 1.7
    with pytest.raises(ValueError, match="precedes"):
        builder._radius_at(-1, [0, 60], [2, 1.7])
    # A tail row must use the held radius, so its 1.7 cm cell cannot be blank.
    payload["moisture_concentration"][-1][17] = None
    with pytest.raises(ValueError, match="Inside-radius"):
        builder.validate_payload(payload, radius)

@pytest.mark.parametrize("audit_only", [False, True])
def test_cli_can_archive_audit_json(tmp_path, audit_only):
    template, payload, output, radius = _fixture(tmp_path)
    payload_path = tmp_path / "payload.json"
    payload_path.write_text(json.dumps(payload), encoding="utf-8")
    archive = tmp_path / "reports" / "data" / "workbook_audit.json"
    command = [sys.executable, str(BUILDER_PATH), str(template), str(payload_path), str(output), "--radius-history", str(radius), "--audit-output", str(archive)]
    if audit_only:
        builder.build_result4(template, payload, output, radius)
        previous_workbook = output.read_bytes()
        command.append("--audit-only")
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    assert json.loads(archive.read_text(encoding="utf-8")) == json.loads(result.stdout)
    assert json.loads(result.stdout)["status"] == "passed"

```

```

if audit_only:
    assert output.read_bytes() == previous_workbook

def test_audit_archive_cannot_overwrite_the_workbook(tmp_path):
    template, payload, output, radius = _fixture(tmp_path)
    payload_path = tmp_path / "payload.json"
    payload_path.write_text(json.dumps(payload), encoding="utf-8")
    builder.build_result4(template, payload, output, radius)
    previous_workbook = output.read_bytes()
    command = [sys.executable, str(BUILDER_PATH), str(template), str(payload_path), str(output), "--radius-history", str(radius), "--audit-output", str(output), "--audit-only"]
    result = subprocess.run(command, capture_output=True, text=True)
    assert result.returncode != 0
    assert "must be distinct" in result.stderr
    assert output.read_bytes() == previous_workbook

```

tools/audit_all_results.py

```

"""Read-only audit of original inputs, four result workbooks and computed payloads."""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import re
from pathlib import Path

import numpy as np
import openpyxl

from build_result4 import audit_result4

HEADER = "时间\\到药材中心的距离"
RADII = [i / 10 for i in range(21)]

def file_hash(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def check_original_inputs(attachment_dir: Path) -> dict:
    results = {}
    for number, columns in ((1, 3), (2, 2)):
        source = attachment_dir / f"附件{number}.xlsx"
        book = openpyxl.load_workbook(source, read_only=True, data_only=True)
        try:
            original = np.asarray([row[:columns] for row in book.worksheets[0].iter_rows(min_row=2, values_only=True)
                                   if row[0] is not None], dtype=float)
            finally:
                book.close()
            csv_path = Path(f"data/raw/attachment{number}.csv")
            with csv_path.open(encoding="utf-8-sig", newline="") as stream:
                reader = csv.reader(stream)
                next(reader)
                exported = np.asarray(list(reader), dtype=float)
            if original.shape != exported.shape or not np.array_equal(original, exported):
                raise ValueError(f"Attachment {number} CSV differs from original workbook")
            results[str(number)] = {"rows": len(original), "columns": columns,
                                   "exact_numeric_match": True, "original_sha256": file_hash(source),
                                   "csv_sha256": file_hash(csv_path), "first_row": original[0].tolist(),
                                   "last_row": original[-1].tolist()}
    return results

```

```

def audit_fixed_workbook(path: Path, payload: dict, number: int) -> dict:
    times = payload["time_s"]
    if payload["radius_cm"] != RADII:
        raise ValueError(f"Question {number}: invalid fixed radii")
    if number in (1, 2):
        expected_count = 1800 if number == 1 else 10800
        if times != list(range(1, expected_count + 1)):
            raise ValueError(f"Question {number}: incomplete one-second output")
        fields = [("温度", "temperature_c"), ("水分浓度", "moisture_concentration")]
    else:
        critical = payload["drying_time_s"]
        strict = payload["strict_completion_time_s"]
        expected_strict = (np.floor(critical / 60) + 1) * 60
        expected_times = sorted(set(list(range(60, int(expected_strict) + 1, 60)) + [critical]))
        if strict != expected_strict or times != expected_times:
            raise ValueError("Question 3: missing minute, critical or first strict row")
        if abs(payload["critical_maximum_moisture_unrounded"] - 0.15) > 1e-8:
            raise ValueError("Question 3: invalid unrounded critical maximum")
        if not 0 <= payload["strict_completion_maximum_moisture_unrounded"] < 0.15:
            raise ValueError("Question 3: invalid unrounded strict maximum")
        fields = [("Sheet1", "moisture_concentration")]
    book = openpyxl.load_workbook(path, data_only=False)
    summaries = []
    mismatches = []
    mismatch_count = 0
    try:
        if book.sheetnames != [name for name, _ in fields]:
            raise ValueError(f"Question {number}: invalid sheets")
        for name, field in fields:
            sheet = book[name]
            if sheet.sheet_state != "visible" or sheet.sheet_format.zeroHeight:
                raise ValueError(f"Hidden content in {path}:{name}")
            if any(dim.hidden for dim in [*sheet.row_dimensions.values(),
*sheet.column_dimensions.values()]):
                raise ValueError(f"Hidden rows or columns in {path}:{name}")
            expected_shape = (len(times) + 1, 22)
            if (sheet.max_row, sheet.max_column) != expected_shape:
                raise ValueError(f"Question {number}: invalid dimensions {sheet.max_row,
sheet.max_column}")
            data = np.asarray(payload[field], dtype=float)
            if data.shape != (len(times), 21) or not np.isfinite(data).all():
                raise ValueError(f"Question {number}: invalid payload field")
            if field == "moisture_concentration" and np.min(data) < 0:
                raise ValueError("Negative moisture in payload")
            if not np.array_equal(data, np.round(data, 4)):
                raise ValueError("Output concentrations/temperatures must be rounded to four decimals")
            expected = [[HEADER, *RADII], *[time, *row] for time, row in zip(times, payload[field])]
            for actual_row, expected_row in zip(sheet.iter_rows(), expected):
                for cell, value in zip(actual_row, expected_row):
                    if cell.data_type in ("f", "e") or cell.comment is not None:
                        raise ValueError(f"Unexpected formula, error or comment at
{name}!{cell.coordinate}")
                    if cell.value != value or isinstance(cell.value, bool):
                        mismatch_count += 1
                        if len(mismatches) < 12:
                            mismatches.append({"sheet": name, "cell": cell.coordinate, "workbook":
cell.value, "recomputed": value})
                        if cell.row > 1 and cell.column > 1 and cell.number_format != "0.0000":
                            raise ValueError(f"Incorrect number format at {name}!{cell.coordinate}")
            summaries.append({"name": name, "rows": sheet.max_row, "columns": sheet.max_column,
"checked_cells": sheet.max_row * sheet.max_column})
    finally:
        book.close()
    return {"status": "passed" if mismatch_count == 0 else "mismatch", "sheets": summaries,
"mismatch_count": mismatch_count, "first_mismatches": mismatches,
"workbook_sha256": file_hash(path)}

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)

```

```

    parser.add_argument("--attachment-dir", type=Path, required=True)
    parser.add_argument("--payload-dir", type=Path, default=Path("tmp"))
    parser.add_argument("--output", type=Path,
default=Path("reports/data/final_all_questions_audit.json"))
    args = parser.parse_args()
    results = {"inputs": check_original_inputs(args.attachment_dir), "workbooks": {},
"numerical_evidence": {}}
    payloads = {}
    for number in range(1, 5):
        payload_path = args.payload_dir / f"problem{number}_result.json"
        payload = json.loads(payload_path.read_text(encoding="utf-8"))
        payloads[number] = payload
        workbook = Path(f"outputs/result{number}.xlsx")
        audit = audit_result4(workbook, payload) if number == 4 else audit_fixed_workbook(workbook,
payload, number)
        audit["payload_sha256"] = file_hash(payload_path)
        audit["workbook_sha256"] = file_hash(workbook)
        results["workbooks"][str(number)] = audit
        excluded = {"time_s", "radius_cm", "temperature_c", "moisture_concentration",
"surface_moisture_concentration"}
        results["numerical_evidence"][str(number)] = {k: v for k, v in payload.items() if k not in
excluded}
        print(f"Question {number}: {audit['status']}", flush=True)
        q2 = np.asarray(payloads[2]["moisture_concentration"])[59::60]
        q3 = np.asarray(payloads[3]["moisture_concentration"])[180]
        results["q2_q3_shared_first_three_hours"] = {
            "same_continuum_model": True, "comparison_points": int(q2.size),
            "maximum_absolute_rounded_difference": float(np.max(np.abs(q2 - q3))),
            "rounded_mismatch_count": int(np.count_nonzero(q2 != q3)),
            "note": "Q2 and Q3 use different time-integration tolerances; four-decimal boundary flips are
assessed separately.",
        }
        tables = []
        for number, fields, hours in ((1, ["temperature_c", "moisture_concentration"], False),
(2, ["temperature_c", "moisture_concentration"], True),
(3, ["moisture_concentration"], True), (4,
["moisture_concentration"], True)):
            report = Path(f"reports/problem{number}.md").read_text(encoding="utf-8")
            payload = payloads[number]
            for table_index, field in enumerate(fields):
                table_number = (1 if number == 1 else 3 if number == 2 else 5 if number == 3 else 6) +
table_index
                section = re.split(rf"\*表 {table_number}\s", report, maxsplit=1)[1]
                rows = []
                started = False
                for line in section.splitlines():
                    if line.startswith("|"):
                        started = True
                        cells = [value.strip() for value in line.strip("|").split("|")]
                        if re.fullmatch(r"\d+(?:\.\d+)?", cells[0]):
                            rows.append(cells)
                    elif started:
                        break
                for cells in rows:
                    time = float(cells[0]) * (3600 if hours else 1)
                    index = payload["time_s"].index(time)
                    expected = [payload[field][index][i] for i in ([0, 5, 10] if number == 4 else [0, 5,
10, 15, 20])]
                    if number == 4:
                        expected.append(payload["surface_moisture_concentration"][index])
                    if [float(value) for value in cells[1:]] != expected:
                        raise ValueError(f"Table {table_number} differs from recomputed values at {time}
s")
                    expected_rows = 7 if number == 1 else 6 if number == 2 else 9 if number == 3 else 8
                    if len(rows) != expected_rows:
                        raise ValueError(f"Table {table_number} is incomplete")
                    tables.append({"table": table_number, "checked_regular_rows": len(rows), "status":
"passed"})
            results["paper_tables"] = tables
            results["status"] = "passed" if all(row["status"] == "passed" for row in

```

```

results["workbooks"].values()) else "mismatch"
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(results, indent=2, ensure_ascii=False, allow_nan=False) + "\n",
encoding="utf-8")
    print(json.dumps({"status": results["status"], "shared_Q2_Q3":
results["q2_q3_shared_first_three_hours"]}, ensure_ascii=False))
    if results["status"] != "passed":
        raise SystemExit(1)

if __name__ == "__main__":
    main()

```

tools/build_result4.py

```

"""Fill and audit the official Problem 4 workbook with numeric results."""

from __future__ import annotations

import argparse
import csv
import json
import math
from bisect import bisect_left
from copy import copy
from pathlib import Path
from typing import Any

import openpyxl

DEFAULT_RADIUS_HISTORY = Path("data/raw/attachment2.csv")
HEADER = "时间\\到药材中心的距离"
FIXED_RADII_CM = [index / 10.0 for index in range(21)]
# Match the solver's 1e-12 m geometric tolerance, expressed here in cm.
RADIUS_TOLERANCE_CM = 1.0e-10
CRITICAL_MOISTURE_TOLERANCE = 1.0e-7

def _finite_number(value: Any, label: str) -> float:
    if isinstance(value, bool) or not isinstance(value, (int, float)) or not math.isfinite(value):
        raise ValueError(f"{label} must be a finite numeric value")
    return float(value)

def _radius_history(path: Path) -> tuple[list[float], list[float]]:
    with path.open(encoding="utf-8-sig", newline="") as handle:
        rows = list(csv.DictReader(handle))
        times = [float(row["time_s"]) for row in rows]
        radii = [float(row["radius_cm"]) for row in rows]
        if len(times) < 2 or any(not math.isfinite(value) for value in times + radii):
            raise ValueError("Radius history must contain at least two finite observations")
        if any(right <= left for left, right in zip(times, times[1:])):
            raise ValueError("Radius-history times must be strictly increasing")
        if any(value <= 0.0 for value in radii):
            raise ValueError("Radius-history values must be positive")
    return times, radii

def _radius_at(query: float, times: list[float], radii: list[float]) -> float:
    """Interpolate measured radii, holding the last radius after observation."""
    if query < times[0]:
        raise ValueError("Output time precedes the measured radius-history horizon")
    if query >= times[-1]:
        return radii[-1]
    index = bisect_left(times, query)
    if times[index] == query:
        return radii[index]
    fraction = (query - times[index - 1]) / (times[index] - times[index - 1])

```

```

    return radii[index - 1] + fraction * (radii[index] - radii[index - 1])

def validate_payload(
    payload: dict[str, Any],
    radius_history: Path = DEFAULT_RADIUS_HISTORY,
) -> dict[str, Any]:
    """Validate time, geometry, and unrounded endpoint evidence.

    Time identities use exact equality: relative tolerances can incorrectly
    merge a minute sample and a nearby critical sample at large times.
    Geometry is recomputed from the observations, not the payload blank mask.
    """
    times = payload["time_s"]
    radii = payload["radius_cm"]
    matrix = payload["moisture_concentration"]
    surface = payload["surface_moisture_concentration"]
    if radii != FIXED_RADII_CM or any(isinstance(value, bool) for value in radii):
        raise ValueError("Fixed radius headers must be 0, 0.1, ..., 2.0 cm")
    if not times or len(times) != len(matrix) or len(times) != len(surface):
        raise ValueError("Payload dimensions do not match the result4 template")
    if any(len(row) != 21 for row in matrix):
        raise ValueError("Moisture matrix has an invalid shape")
    numeric_times = [_finite_number(value, "Output time") for value in times]
    if any(right <= left for left, right in zip(numeric_times, numeric_times[1:])):
        raise ValueError("Output times must be strictly increasing")
    critical = _finite_number(payload["drying_time_s"], "Critical time")
    strict = _finite_number(payload["strict_completion_time_s"], "Strict completion time")
    if critical <= 0.0 or strict != 60.0 * (math.floor(critical / 60.0) + 1):
        raise ValueError("Strict completion must be the first minute strictly after the critical time")
    if numeric_times.count(critical) != 1:
        raise ValueError("Exactly one critical time row is required")
    expected_times = sorted(set([float(value) for value in range(60, int(strict) + 1, 60)] +
[critical]))
    if numeric_times != expected_times:
        raise ValueError("Time rows must contain every 60 s sample and exactly one critical row")
    history_times, history_radii = _radius_history(Path(radius_history))
    blank_count = 0
    for index, time in enumerate(numeric_times):
        actual_radius = _radius_at(time, history_times, history_radii)
        for radius, value in zip(FIXED_RADII_CM, matrix[index]):
            if radius > actual_radius + RADIUS_TOLERANCE_CM:
                if value is not None:
                    raise ValueError(f"Outside-radius cell must be blank at time {time}, radius
{radius}")
                blank_count += 1
            else:
                moisture = _finite_number(value, f"Inside-radius moisture at time {time}, radius
{radius}")
                if moisture < 0.0:
                    raise ValueError("Moisture values must not be negative")
                if _finite_number(surface[index], f"Surface moisture at time {time}") < 0.0:
                    raise ValueError("Surface moisture values must not be negative")
                threshold = _finite_number(payload["threshold"], "Moisture threshold")
                if threshold != 0.15:
                    raise ValueError("Problem 4 requires the 0.15 kg/kg moisture threshold")
                critical_maximum = _finite_number(payload["critical_maximum_moisture_unrounded"], "Unrounded
critical maximum")
                strict_maximum = _finite_number(payload["strict_completion_maximum_moisture_unrounded"], "Unrounded
strict maximum")
                if abs(critical_maximum - threshold) > CRITICAL_MOISTURE_TOLERANCE:
                    raise ValueError("Unrounded critical maximum does not meet the threshold tolerance")
                if not 0.0 <= strict_maximum < threshold:
                    raise ValueError("Unrounded strict completion maximum must be strictly below the threshold")
                for time, maximum in ((critical, critical_maximum), (strict, strict_maximum)):
                    index = numeric_times.index(time)
                    displayed_maximum = max([value for value in matrix[index] if value is not None] +
[surface[index]])
                    if abs(displayed_maximum - round(maximum, 4)) > 1.0e-12:
                        raise ValueError("Rounded endpoint values disagree with the unrounded maximum")

```

```

return {
    "data_rows": len(times),
    "rows": len(times) + 1,
    "columns": 23,
    "blank_outside_cells": blank_count,
    "radius_last_observation_time_s": history_times[-1],
    "radius_extension": "hold_last_value",
    "rows_after_radius_observations": sum(time > history_times[-1] for time in numeric_times),
    "critical_time_s": critical,
    "strict_completion_time_s": strict,
    "critical_maximum_moisture_unrounded": critical_maximum,
    "strict_completion_maximum_moisture_unrounded": strict_maximum,
}

def audit_result4(
    workbook_path: Path,
    payload: dict[str, Any] | Path,
    radius_history: Path = DEFAULT_RADIUS_HISTORY,
) -> dict[str, Any]:
    """Reopen the workbook and compare every cell with the verified payload.

    No workbook metadata is removed. Unexpected comments, hidden content,
    formulas, errors, extra sheets, and changed numeric values fail the audit.
    """
    if not isinstance(payload, dict):
        payload = json.loads(Path(payload).read_text(encoding="utf-8"))
    summary = validate_payload(payload, radius_history)
    workbook = openpyxl.load_workbook(workbook_path, data_only=False)
    try:
        for sheet in workbook.worksheets:
            if sheet.sheet_state != "visible":
                raise ValueError(f"Hidden worksheet: {sheet.title}")
            if sheet.sheet_format.zeroHeight or any(dimension.hidden for dimension in
sheet.row_dimensions.values()):
                raise ValueError(f"Hidden rows in {sheet.title}")
            if any(dimension.hidden for dimension in sheet.column_dimensions.values()):
                raise ValueError(f"Hidden columns in {sheet.title}")
            for row in sheet.iter_rows():
                for cell in row:
                    if cell.comment is not None:
                        raise ValueError(f"Unexpected comment in {sheet.title}!{cell.coordinate}")
                    if cell.data_type in ("f", "e"):
                        raise ValueError(f"Unexpected formula or error in
{sheet.title}!{cell.coordinate}")
                    if workbook.sheetnames != ["Sheet1"]:
                        raise ValueError("Result4 must contain only the official Sheet1")
                    sheet = workbook["Sheet1"]
                    if (sheet.max_row, sheet.max_column) != (summary["rows"], summary["columns"]):
                        raise ValueError("Workbook dimensions do not match the validated payload")
                    expected = [[HEADER, *FIXED_RADII_CM, "药材表面"]]
                    expected.extend(
                        [time, *row, surface]
                        for time, row, surface in zip(payload["time_s"], payload["moisture_concentration"],
payload["surface_moisture_concentration"])
                    )
                    for actual_row, expected_row in zip(sheet.iter_rows(), expected):
                        for cell, value in zip(actual_row, expected_row):
                            if cell.value != value or isinstance(cell.value, bool):
                                raise ValueError(f"Workbook/payload mismatch at {cell.coordinate}:
{cell.value!r} != {value!r}")
                    return {"status": "passed", **summary, "checked_cells": summary["rows"] * summary["columns"]}
    finally:
        workbook.close()

def build_result4(
    template: Path,
    payload: dict[str, Any] | Path,
    output: Path,

```

```

    radius_history: Path = DEFAULT_RADIUS_HISTORY,
) -> dict[str, Any]:
    """Populate the template, preserving its numeric-output conventions."""
    if not isinstance(payload, dict):
        payload = json.loads(Path(payload).read_text(encoding="utf-8"))
    summary = validate_payload(payload, radius_history)
    times = payload["time_s"]
    radii = payload["radius_cm"]
    matrix = payload["moisture_concentration"]
    surface = payload["surface_moisture_concentration"]
    workbook = openpyxl.load_workbook(template)
    if "Sheet1" not in workbook.sheetnames:
        raise ValueError("Official result4 template must contain Sheet1")
    sheet = workbook["Sheet1"]
    sheet.cell(1, 1).value = HEADER
    for col, radius in enumerate(radii, start=2):
        sheet.cell(1, col).value = radius
    sheet.cell(1, len(radii) + 2).value = "药材表面"
    required_rows = summary["rows"]
    if sheet.max_row > required_rows:
        sheet.delete_rows(required_rows + 1, sheet.max_row - required_rows)
    for row_index, time in enumerate(times, start=2):
        for col_index, value in enumerate([time, *matrix[row_index - 2], surface[row_index - 2]],
start=1):
            sheet.cell(row_index, col_index).value = value

    sheet.column_dimensions["A"].width = 24
    for col in range(2, len(radii) + 3):
        sheet.column_dimensions[openpyxl.utils.get_column_letter(col)].width = 11
    for row in sheet.iter_rows(min_row=1, max_row=required_rows, min_col=1, max_col=len(radii) + 2):
        for cell in row:
            alignment = copy(cell.alignment)
            alignment.horizontal = "center"
            alignment.vertical = "center"
            cell.alignment = alignment
            if cell.row == 1:
                font = copy(cell.font)
                font.name = "宋体"
                font.size = 10
                cell.font = font
            elif cell.column == 1:
                cell.number_format = "0.#####
            else:
                cell.number_format = "0.0000"
    sheet.freeze_panes = "B2"
    output.parent.mkdir(parents=True, exist_ok=True)
    workbook.save(output)
    workbook.close()
    return audit_result4(output, payload, radius_history)

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("template", type=Path)
    parser.add_argument("payload", type=Path)
    parser.add_argument("output", type=Path)
    parser.add_argument("--radius-history", type=Path, default=DEFAULT_RADIUS_HISTORY)
    parser.add_argument("--audit-only", action="store_true", help="Audit an existing output; do not
write a workbook")
    parser.add_argument("--audit-output", type=Path, help="Write the successful audit summary as JSON")
    args = parser.parse_args()
    if args.audit_output is not None and args.audit_output.resolve() in {
        path.resolve() for path in (args.template, args.payload, args.output, args.radius_history)
    }:
        parser.error("--audit-output must be distinct from the template, payload, workbook, and radius
history")
    if args.audit_only:
        summary = audit_result4(args.output, args.payload, args.radius_history)
    else:
        summary = build_result4(args.template, args.payload, args.output, args.radius_history)

```

```

report = {"workbook": str(args.output), **summary}
if args.audit_output is not None:
    args.audit_output.parent.mkdir(parents=True, exist_ok=True)
    args.audit_output.write_text(json.dumps(report, ensure_ascii=False, indent=2, allow_nan=False)
+ "\n", encoding="utf-8")
    print(json.dumps(report, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

tools/compress_problem2_full_process.py

```

"""Convert the large full-process workbook to a compact compressed NPZ archive."""
from pathlib import Path
import numpy as np
from openpyxl import load_workbook

root = Path(__file__).resolve().parents[1]
src = root / 'deliverables' / 'supporting_materials' / 'outputs' / 'result2.xlsx'
dst = src.with_name('result2_full_process.npz')
wb = load_workbook(src, read_only=True, data_only=True)
arrays = {}
for ws in wb.worksheets:
    rows = ws.iter_rows(values_only=True)
    header = next(rows)
    data = np.asarray([[np.nan if v is None else v for v in row] for row in rows], dtype=np.float64)
    arrays[ws.title] = data
    arrays[f'{ws.title}_header'] = np.asarray([str(v) for v in header])
np.savez_compressed(dst, **arrays)
print({'source_bytes': src.stat().st_size, 'npz_bytes': dst.stat().st_size,
      'sheets': list(arrays), 'rows': {k: v.shape for k, v in arrays.items() if not
k.endswith('_header')}}})

```

tools/extend_problem2_delivery.py

```

"""Add a full-process one-second workbook without changing the original results."""
from pathlib import Path
import json
import numpy as np
from openpyxl import Workbook, load_workbook
from openpyxl.cell import WriteOnlyCell
from drying_model.problem1 import load_chamber_history_csv, sample_solution
from drying_model.problem2 import solve_problem2
from drying_model.problem3 import extend_chamber_history_to_plateau

def main():
    out = Path('deliverables/supporting_materials')
    (out / 'outputs').mkdir(parents=True, exist_ok=True)
    Path('tmp/paper').mkdir(parents=True, exist_ok=True)
    end = int(json.loads(Path('tmp/problem3_result.json').read_text(encoding='utf-
8'))['strict_completion_time_s'])
    history = extend_chamber_history_to_plateau(load_chamber_history_csv('data/raw/attachment1.csv'),
end)
    radii = np.arange(21) / 10
    for n in (2560, 5120):
        cache = Path(f'tmp/paper/q2full_{n}.npz')
        if cache.exists():
            continue
        T = np.empty((end, 21)); C = np.empty_like(T)
        ti = ci = None
        for start in range(0, end, 600):
            stop = min(start + 600, end)
            times = np.arange(start + 1, stop + 1, dtype=float)
            sol = solve_problem2(history, radial_intervals=n, output_times_s=times,
relative_tolerance=2e-10, max_step_s=2 if start < 14400 else 60,

```

```

        initial_time_s=float(start) if start else None,
        initial_temperature_c=ti, initial_moisture_concentration=ci)
    sample = sample_solution(sol, radii)
    T[start:stop] = sample.temperature_c; C[start:stop] = sample.moisture_concentration
    ti = sol.temperature_c[-1].copy(); ci = sol.moisture_concentration[-1].copy()
    if start % 21600 == 0:
        print(f'grid={n}, time={stop}/{end}', flush=True)
    np.savez_compressed(cache, temperature=T, moisture=C)
    coarse = np.load('tmp/paper/q2full_2560.npz')
    fine = np.load('tmp/paper/q2full_5120.npz')
    values = {key: (4*fine[key]-coarse[key])/3 for key in ('temperature', 'moisture')}
    p2 = json.loads(Path('tmp/problem2_result.json').read_text(encoding='utf-8'))
    p3 = json.loads(Path('tmp/problem3_result.json').read_text(encoding='utf-8'))
    audit = {'time_start_s':1, 'time_end_s':end, 'radial_intervals':[2560,5120], 'output_interval_s':1}
    for field, key in [('temperature', 'temperature_c'), ('moisture', 'moisture_concentration')]:
        difference = np.abs(np.round(values[field][:10800],4)-np.array(p2[key]))
        audit[field+'_first_3h_max_difference'] = float(difference.max())
        if difference.max() > 1e-8:
            raise ValueError('First three hours differ from audited original')
    minute_rows = [(int(t)-1, i) for i,t in enumerate(p3['time_s']) if float(t).is_integer() and
int(t)%60 == 0]
    diff = np.array([np.round(values['moisture'][a],4)-np.array(p3['moisture_concentration'])[b]) for
a,b in minute_rows])
    audit['q3_minute_max_rounded_difference'] = float(np.abs(diff).max())
    audit['strict_final_maximum_unrounded'] = float(values['moisture'][-1].max())
    if audit['strict_final_maximum_unrounded'] >= .15 or np.abs(diff).max() > .00010001:
        raise ValueError('Full-process cross-check failed')
    book = Workbook(write_only=True)
    for name, field in [('温度', 'temperature'), ('水分浓度', 'moisture')]:
        sheet = book.create_sheet(name)
        sheet.append(['时间\\到药材中心的距离']+list(radii))
        for i, row in enumerate(np.round(values[field],4)):
            cells = [i+1]
            for v in row:
                cell = WriteOnlyCell(sheet, float(v)); cell.number_format='0.0000'; cells.append(cell)
            sheet.append(cells)
    dest = out/'outputs/result2.xlsx'
    book.save(dest)
    check = load_workbook(dest, read_only=True, data_only=True)
    checked=0
    for sheet, field in zip(check.worksheets, ('temperature', 'moisture')):
        for i,row in enumerate(sheet.iter_rows(min_row=2, values_only=True)):
            if row[0] != i+1 or not np.allclose(row[1:], np.round(values[field][i],4), rtol=0, atol=1e-
12):
                raise ValueError('Workbook readback mismatch')
            checked+=len(row)
            if i+1 != end: raise ValueError('Missing output rows')
    check.close()
    audit['readback_checked_cells']=checked
    audit['workbook_bytes']=dest.stat().st_size
    audit['status']='passed'
    (out/'reports/data').mkdir(parents=True, exist_ok=True)

(out/'reports/data/problem2_full_process_audit.json').write_text(json.dumps(audit, ensure_ascii=False, in
dent=2), encoding='utf-8')
    print(json.dumps(audit), flush=True)

if __name__=='__main__': main()

```

tools/rebuild_delivery_workbooks.py

```

"""Build the four workbooks from the independently reproduced JSON payloads."""
from pathlib import Path
import json
from openpyxl import Workbook

def main():

```

```

out=Path('outputs/recomputed'); out.mkdir(parents=True,exist_ok=True)
for number in (1,2,3,4):
    payload=json.loads(Path(f'tmp/problem{number}_result.json').read_text(encoding='utf-8'))
    book=Workbook(); book.remove(book.active)
    for name,key in (('温度','temperature_c'),('水分浓度','moisture_concentration')) if number<3
else [('Sheet1','moisture_concentration')]]:
    ws=book.create_sheet(name)
    ws.append(['时间\\到药材中心的距离']+payload['radius_cm']+(['药材表面'] if number==4 else
[]))

    for i,(t,row) in enumerate(zip(payload['time_s'],payload[key],strict=True)):
        ws.append([t]+row+([payload['surface_moisture_concentration'][i]] if number==4 else
[]))

    for row in ws.iter_rows(min_row=2,min_col=2):
        for c in row: c.number_format='0.0000'
    dest=out/f'result{number}.xlsx'
    if dest.exists(): raise FileExistsError(dest)
    book.save(dest)
    print(dest)

if __name__=='__main__': main()

```

tools/restore_full_result2.py

```

"""Restore the complete, rounded, per-second result2.xlsx from the lossless NPZ."""
from pathlib import Path
import argparse
import numpy as np
from openpyxl import Workbook
from openpyxl.cell import WriteOnlyCell

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--input',type=Path,default=Path('outputs/result2_full_process.npz'))
    parser.add_argument('--output',type=Path,default=Path('outputs/result2_full_process.xlsx'))
    args = parser.parse_args()
    if args.output.exists():
        raise FileExistsError(f'Refusing to overwrite {args.output}; choose another --output')
    wb = Workbook(write_only=True)
    with np.load(args.input,allow_pickle=False) as archive:
        for name in ('温度','水分浓度'):
            ws = wb.create_sheet(name)
            ws.append(['时间\\到药材中心的距离']+ [i/10 for i in range(21)])
            for row in archive[name]:
                cells = [int(row[0])]
                for value in row[1:]:
                    cell = WriteOnlyCell(ws,float(value)); cell.number_format='0.0000';
            cells.append(cell)
            ws.append(cells)
    args.output.parent.mkdir(parents=True,exist_ok=True)
    wb.save(args.output)
    print(args.output)

if __name__=='__main__': main()

```